

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250278524

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

AUH; Yoon Ho

STRUCTURED DATA FOLDING WITH TRANSMUTATIONS

Abstract

A method of processing data includes at least one processor accessing a data storage unit, the data storage unit providing at least one input data object and at least one transmutation command to be performed on the at least one input data object. The at least one transmutation command operates in a forward mode on the at least one input data object to produce at least one output data object to be stored in a data storage unit.

Inventors:	AUH; Yoon Ho (Glencoe, IL)
Applicant:	NUTS HOLDINGS, LLC (Glencoe, IL)
Family ID:	61560000
Assignee:	NUTS HOLDINGS, LLC (Glencoe, IL)
Appl. No.:	19/209443
Filed:	May 15, 2025

Related U.S. Application Data

parent US continuation 17232929 20210416 parent-grant-document US 11720716 child US 18331366
parent US continuation 16582649 20190925 parent-grant-document US 11010496 child US 17232929
parent US continuation 15692648 20170831 parent-grant-document US 10503933 child US 16582649
parent US division 18331366 20230608 PENDING child US 19209443
us-provisional-application US 62395084 20160915

Publication Classification

Int. Cl.: G06F21/78 (20130101); G06F16/25 (20190101); G06F21/60 (20130101); G06F21/62 (20130101); H04L9/06 (20060101); H04L9/08 (20060101); H04L9/14 (20060101); H04L9/32 (20060101); H04L9/40 (20220101)

U.S. Cl.:

CPC G06F21/78 (20130101); G06F16/258 (20190101); G06F21/602 (20130101); G06F21/6227 (20130101); H04L9/0637 (20130101); H04L9/0643 (20130101); H04L9/085 (20130101); H04L9/0861 (20130101); H04L9/14 (20130101); H04L9/3226 (20130101); H04L9/3242 (20130101); H04L9/3247 (20130101); H04L63/0281 (20130101); H04L63/0442 (20130101)

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS [0001] This application is a divisional of U.S. patent application Ser. No. 18/331,366, filed Jun. 8, 2023, which is a continuation of U.S. patent application Ser. No. 17/232,929, filed Apr. 16, 2021, now U.S. Pat. No. 11,720,716 issued Aug. 8, 2023, which is a continuation of U.S. patent application Ser. No. 16/582,649, filed Sep. 25, 2019, now U.S. Pat. No. 11,010,496 issued, May 18, 2021, which is a divisional of U.S. patent application Ser. No. 15/692,648, filed Aug. 31, 2017, now U.S. Pat. No. 10,503,933 issued Dec. 10, 2019, which is based on and derives priority from U.S. Provisional Patent Application No. 62/395,084, filed Sep. 15, 2016, the entire contents of each of which are incorporated herein by reference.

BRIEF SUMMARY

[0002] A data centric model of computer software design is where user data may be prioritized over applications. A data centric software design may allow for data to be secured at the point of storage. The containerization of data may be an embodiment of a data centric design. In order to show how various concepts may be implemented within this disclosure, a series of drawings from different perspectives highlight the specific concepts being explored and integral drawings show how several of these processes and structures may work together.

[0003] The containerization of data may be presented in a layered approach and, if preferred, each layer may build upon, or work in conjunction with, the previous layers in part or whole. The concepts, methods, apparatus, embodiments and/or specifications described herein for a first layer may be collectively called Structured Data Folding with Transmutations or SDFT. The concepts, methods, apparatus, embodiments and/or specifications described herein for a second layer, which may be inclusive of the first layer, may be collectively called eNCrypted Userdata Transit & Storage or NUTS. Any combination of each layer may be deployed in part or whole to construct a container for data called a Nut, and each layer may be deployed in part or whole in isolation. The interplay and/or interweaving of these two layers may be significant and/or complex and may pose challenges for the clear demarcation of such layers. Therefore, these layers are presented together in this specification. The Nut container may then be infused with various data centric characteristics which may allow for logical operations on the data contained therein. Upon the unit of storage called a Nut, various embodiments may be described to show how certain common data oriented logical operations may be re-defined and restructured to offer users privacy, security, convenience and/or capabilities.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0004] Various embodiments may be disclosed in the following detailed description and the accompanying drawings:

[0005] FIG. **1** shows a table of symbols used to represent different cipher key types.

[0006] FIG. **2** shows a set of simplified flowcharts showing the data inputs, data outputs and logical operations that may be typically performed by different cipher methods.

[0007] FIG. **3** shows an illustration of a general network layout where embodiments of this disclosure may function.

[0008] FIG. **4** shows an illustration of a computing device where embodiments of this disclosure may function.

[0009] FIG. **5** shows an illustration of a transmutation in forward mode or normal operation

[0010] FIG. **6** shows a table of common data operations and their transmutation classifications

[0011] FIG. **7** shows an illustration of a transmutation in reverse mode.

[0012] FIG. **8** shows an illustration of an irreversible transmutation

[0013] FIG. **9** shows an illustration of a conditionally reversible transmutation

[0014] FIG. **10** shows a table of common data operations and functions grouped by transmutation type

[0015] FIG. **11** shows a table of codecs defined in Python v3.

[0016] FIG. **12** shows a table listing additional transmutation definitions

[0017] FIG. **13** shows a transmutation reversibility matrix

[0018] FIG. **14** shows a transmutation modal action matrix

[0019] FIG. **15** shows a detailed example of a serialize transmutation

[0020] FIG. **16** shows a detailed example of a digest transmutation

[0021] FIG. **17** shows a detailed example of a digest transmutation in reverse mode also known as a verification

[0022] FIG. **18** shows an illustration of a scipher transmutation

[0023] FIG. **19** shows an illustration of a salsa20 (scipher) transmutation

[0024] FIG. **20** shows a detailed example of a salsa20 (scipher) transmutation.

[0025] FIG. **21** shows a table of command specifications for serialize and compress transmutations and a set of sample transmutation commands showing its usage.

[0026] FIG. **22** shows a table of command specifications for an encode transmutation and a set of sample transmutation commands showing its usage.

[0027] FIG. **23** shows a table of command specifications for a digest transmutation and a set of sample transmutation commands showing its usage.

[0028] FIG. **24** shows a table of command specifications for an acipher and dign transmutations and a set of sample transmutation commands showing its usage.

[0029] FIG. **25** shows a table of command specifications for a derive transmutation and a set of sample transmutation commands showing its usage.

[0030] FIG. **26** shows a table of command specifications for a scipher transmutation **2602** and a set of sample transmutation commands showing its usage **2604**.

[0031] FIG. **27** shows the output structure format for a scipher output string in a sequence of two steps where Step 1 illustrates the input format and Step 2 illustrates the output format. “Header” is the variable length key-value utf8 encoded parameter string of the scipher transmutation on the output message.

[0032] FIG. **28** shows a table of parameter keywords and specifications for the header string in the output structure format of a scipher transmutation.

[0033] FIG. **29** shows an illustration of iterative embedded message encapsulations for an AEAD mode scipher transmutation.

[0034] FIG. **30** shows a table of command specifications for a lock transmutation **3002** and a set of

sample transmutation commands showing its usage **3010**.

[0035] FIG. **31** shows the specifications of various transmutation structures in tabular format.

[0036] FIG. **32** shows a table of command specifications for a mobius transmutation. Its usage is shown and a graphic shows the structural changes it can enact on various structures. A matrix shows the structure type/mode valid operations that the mobius transmutation can operate on.

[0037] FIG. **33** shows a table of command specifications for press, clean and key transmutations **3302**, **3304** and a set of sample transmutation commands showing its usage **3310**.

[0038] FIG. **34** shows a table for the Key Interchange Specification Structure or KISS.

[0039] FIG. **35** shows a table for KISS modes of operation **3502**, a matrix showing key types/field generation mappings **3504** and key type definitions **3506**.

[0040] FIG. **36** shows the structure of a TAR and examples of TAR definitions.

[0041] FIG. **37** shows block diagrams illustrating where transmutation related attributes are persisted and a table listing the type and locations of the attributes.

[0042] FIG. **38** shows block diagrams of SDFT operations ravel and unravel (or reversal of ravel).

[0043] FIG. **39** shows a flowchart of a SDFT ravel operation.

[0044] FIG. **40** shows a flowchart of a SDFT unravel operation.

[0045] FIG. **41** shows how a TAR reversal is performed on a generic TAR.

[0046] FIG. **42** shows examples of TAR reversals.

[0047] FIG. **43** shows a table of transmutations mapped to a key type template it may generate or require during TAR processing.

[0048] FIG. **44** shows TAR examples and the key templates generated from each.

[0049] FIG. **45** shows TAR examples and the key templates generated from each and the expected list of KISS structures to be input (put) or generated (gen). The list of KISSes is also referred to as the keystack.

[0050] FIG. **46** show the three modes of keystack operation within SDFT TAR processing: generate (gen), input (put) and injection (mixed).

[0051] FIG. **47** shows an illustration of how keystacks may be generated and used in the life cycle of data and its TAR.

[0052] FIG. **48** shows an illustration of operations which may occur on data stored in an NSstr structure.

[0053] FIG. **49** shows a flow diagram of SDFT usage to iteratively fold data.

[0054] FIG. **50** shows a flow diagram of SDFT usage to iteratively unfold data.

[0055] FIG. **51** shows an illustration of the SDFT API/Library and the various types of TAR definition files it may have access to.

[0056] FIG. **52** shows an example Python script to perform manual data folding.

[0057] FIG. **53** shows an SDFT example of a TAR definition and its usage in a Python script.

[0058] FIG. **54** shows block diagrams of dynamic TAR switching within a single communication session.

[0059] FIG. **55** shows a flowchart of an example process for generating a Nut ID.

[0060] FIG. **56** shows a block diagram showing where Nut IDs and Lock IDs may be used within a Nut.

[0061] FIG. **57** shows example relationships between Nut IDs, path names and payload data.

[0062] FIG. **58** shows a block diagram of an embodiment of a Nut or Lock Graph comprising logical sections: Nut Lock and Nut Parts.

[0063] FIG. **59** shows a block diagram of an alternative embodiment of a Nut Lock in a Nut comprising three Keyhole Lock Nodes.

[0064] FIG. **60** shows a block diagram of the internal data structures of a Lock Node.

[0065] FIG. **61** shows a block diagram of the internal data structures of an Input Section of a Lock Node shown in FIG. **60**.

[0066] FIG. **62** shows a data flow diagram showing the relationship of the internal data structures

of a Primary Keyhole of an Input Section shown in FIG. 61 when a valid Primary Key may be inserted into the keyhole.

[0067] FIG. 63 shows a flowchart of the key insertion process for any Lock Node and for any cipher key.

[0068] FIG. 64 shows an example where three Primary Keys may be inserted into a Primary Keyhole.

[0069] FIG. 65 shows a data flow diagram of a Variable Lock decrypt operation continuing from the example shown in FIG. 64.

[0070] FIG. 66 shows a data flow diagram of a Variable Lock encrypt operation continuing from the example shown in FIG. 64.

[0071] FIG. 67 shows a table of Variable Lock Types available in any Lock Node and their characteristics.

[0072] FIG. 68 shows a data flow diagram of an ORLOCK decryption operation.

[0073] FIG. 69 shows a data flow diagram of an ORLOCK encryption operation by a Nut owner.

[0074] FIG. 70 shows a data flow diagram of an MATLOCK decryption operation.

[0075] FIG. 71 shows a data flow diagram of an MATLOCK encryption operation by a Nut owner.

[0076] FIG. 72 shows a data flow diagram of an XORLOCK decryption operation.

[0077] FIG. 73 shows a data flow diagram of an XORLOCK encryption operation by a Nut owner.

[0078] FIG. 74 shows a data flow diagram of an HASHLOCK decryption operation.

[0079] FIG. 75 shows a data flow diagram of an HASHLOCK encryption operation by a Nut owner.

[0080] FIG. 76 shows a data flow diagram of an SSLOCK decryption operation.

[0081] FIG. 77 shows a data flow diagram of an SSLOCK encryption operation by a Nut owner.

[0082] FIG. 78 shows a block diagram of a Nut highlighting Stratum Keys.

[0083] FIG. 79 shows a flowchart of how a Stratum Key may be inserted within a Nut.

[0084] FIG. 80 shows a table of Key Based Permissions for two roles and four role players.

[0085] FIG. 81 shows a table listing the variety of Nut Parts in an example Nut where each part may be represented by a Lock Node.

[0086] FIG. 82 shows a table listing Key Based Permissions access roles defined for a typical Nut.

[0087] FIG. 83 shows a block diagram of how the initial set of Nut Access Control access keys called Access Attribute Key Set Unlock Keys (AAKSUK) may be inserted into the Access Keyhole for each valid Primary Key.

[0088] FIG. 84 shows a block diagram of the propagation of NAC access attributes from External Lock Nodes to Internal Lock Nodes.

[0089] FIG. 85 shows a block diagram of the propagation of NAC access attributes from External Lock Nodes to Internal Lock Nodes and the insertion of the Output Linking Key into the Primary Keyhole of the linked Lock Node.

[0090] FIG. 86 shows a flowchart for inserting keys into an Access Keyhole.

[0091] FIG. 87 shows a table of Key Based Permissions for an alternate embodiment.

[0092] FIG. 88 shows a data flow diagram of the internal decryption data flows of a Lock Node.

[0093] FIG. 89 shows a flowchart to unlock a Nut.

[0094] FIG. 90 shows a block diagram of an embodiment of a NUTS based system and how a document stored in a Nut may be unlocked.

[0095] FIG. 91 shows an illustration of the common usage in NUTS parlance to refer to a Nut's payload by the Nut ID of the Nut holding it. Here a cipher key may be referred to by the Nut ID of the Nut holding it.

[0096] FIG. 92 shows a simplified embodiment of a list of recipients locking model.

[0097] FIG. 93 shows a simplified embodiment of an ordered locking model.

[0098] FIG. 94 shows a simplified embodiment of an ordered locking model with master key.

[0099] FIG. 95 shows a simplified embodiment of a locking model with master key.

[0100] FIG. **96** shows a simplified embodiment of a locking model with master key.

[0101] FIG. **97** shows a simplified embodiment of a safe deposit box locking model.

[0102] FIG. **98** shows a simplified embodiment of a secret sharing locking model with master key.

[0103] FIG. **99** shows a simplified embodiment of a “PrivaTegrity” type locking model.

[0104] FIG. **100** shows a simplified embodiment of a multi-Nut configuration where multiple payloads may be stored within a single Nut.

[0105] FIG. **101** shows a simplified embodiment of a multi-Nut configuration where multiple payloads may be stored within a single Nut.

[0106] FIG. **102** shows a simplified embodiment of a direct locking model with multiple payloads.

[0107] FIG. **103** shows a simplified embodiment of an ordered message passing exhibiting collusion resistant design.

[0108] FIG. **104** shows a block diagram of typical components of Modular I/O.

[0109] FIG. **105** shows an illustration of simple reading and writing operations using the MIOR.

[0110] FIG. **106** shows the data transformations and transfers that may be involved in a typical MIO file reading operation.

[0111] FIG. **107** illustrates how backward compatibility of file formats may be facilitated using Modular I/O.

[0112] FIG. **108** illustrates how forward compatibility of file formats may be facilitated using Modular I/O.

[0113] FIG. **109** illustrates how modular display may be facilitated using Modular I/O.

[0114] FIG. **110** illustrates how modular application may be facilitated using Modular I/O.

[0115] FIG. **111** illustrates the progressive changes to a Nut History over two edits and at three points in time.

[0116] FIG. **112** illustrates the progressive changes to a Nut Log over the course of events from FIG. **111**.

[0117] FIG. **113** shows how Relationship Based Keys may be represented in Alice and Bob's contact cards.

[0118] FIG. **114** shows a flowchart of how SPAM may be detected using well known email addresses and/or RBKs.

[0119] FIG. **115** shows a flowchart of how SPAM may be detected using anonymous email addresses and/or RBKs.

[0120] FIG. **116** shows a Deterministic Context Based Status Matrix of an Alice-Bob RBK communication channel.

[0121] FIG. **117** shows a Deterministic Context Based Status Matrix of an Alice-vendor RBK communication channel.

[0122] FIG. **118** shows a Deterministic Context Based Status Matrix of a vendor-Alice RBK communication channel.

[0123] FIG. **119** illustrates the isolation of RBK relationships in a compromised system for Bob.

[0124] FIG. **120** shows a block diagram of Pre-Packaged Data Nuts.

[0125] FIG. **121** charts the sequence of events in an Automated Registration process using RBKs.

[0126] FIG. **122** charts the sequence of events in an Automated Registration process using RBKs and anonymous email addresses.

[0127] FIG. **123** shows a table listing the NUTS Core Applications and their descriptions.

[0128] FIG. **124** shows a block diagram of the NUTS Core Applications running in a computer device.

[0129] FIG. **125** shows a block diagram of the NUTserver running in a user device.

[0130] FIG. **126** shows block diagram of the internal components comprising a NUTserver and their functional connections to the environment of the user device.

[0131] FIG. **127** shows an alternative embodiment of the NUTserver shown in FIG. **126** using a NoSQL database as a caching mechanism.

[0132] FIG. **128** shows a block diagram of a MIOR Server network layout.

[0133] FIG. **129** shows a block diagram of a MIOR Server application layout.

[0134] FIG. **130** shows a flowchart for fetching MIO modules from a MIOR Server.

[0135] FIG. **131** shows a block diagram illustrating the organization of a MIOR Cache.

[0136] FIG. **132** shows a block diagram of a NUTbrowser application in a user device environment.

[0137] FIG. **133** shows a block diagram of a NUTbook application in a user device environment.

[0138] FIG. **134** shows a block diagram of a Nut processing application framework in a user device environment.

[0139] FIG. **135** shows a block diagram illustrating the internal components comprising a NUTbook.

[0140] FIG. **136** shows a block diagram illustrating the internal organization of a NUTbook Catalog Cache from FIG. **135**.

[0141] FIG. **137** shows a diagram showing the organization of Hierarchical Passwords.

[0142] FIG. **138** shows how the Main Password opens a Personal Document as per the Hierarchical Passwords of FIG. **137**.

[0143] FIG. **139** shows how the Master Password opens a Personal Document as per the Hierarchical Passwords of FIG. **137** and the document in FIG. **138**.

[0144] FIG. **140** shows how the Main and Work Passwords opens a Work Document as per the Hierarchical Passwords of FIG. **137**.

[0145] FIG. **141** shows how the Master Password opens a Work Document as per the Hierarchical Passwords of FIG. **137** and the document in FIG. **140**.

[0146] FIG. **142** shows a block diagram illustrating the internal organization of a NUTbook Key Cache from FIG. **135**.

[0147] FIG. **143** shows a flowchart for how a NUTbook may view a Card Catalog.

[0148] FIG. **144** shows a table of NUTS Based Services.

[0149] FIG. **145** shows an illustration of the network layout of NUTS Based Services.

[0150] FIG. **146** shows an illustration of the network layout of a NUTmail Server.

[0151] FIG. **147** charts the sequence of events in automated registration process to an anonymous email service such as NUTmail using RBKs.

[0152] FIG. **148** charts the sequence of events when adding a communication channel in a NUTmail Server.

[0153] FIG. **149** charts the sequence of events when Alice and Bob send emails to each other via NUTmail.

[0154] FIG. **150** shows an illustration of the network layout of a NUTchat Server.

[0155] FIG. **151** shows a data flow diagram of three chat sessions hosted by a NUTchat Server.

[0156] FIG. **152** shows a data flow diagram of chat history persistence and replication across NUTservers.

[0157] FIG. **153** shows a data flow diagram for three separate chat sessions using different chat IDs or chat services.

[0158] FIG. **154** shows a data flow diagram for a path agnostic Dialogue managed by a NUTchat client using three different chat pathways from FIG. **153**.

[0159] FIG. **155** shows an illustration of the network layout of a NUTcloud Server.

[0160] FIG. **156** shows an illustration of the network layout of a NUTnet Server.

[0161] FIG. **157** shows an illustration of the network layout of a NUThub Server for the Internet of NUTS (IoN).

[0162] FIG. **158** shows an illustration of a Direct IoN network topology.

[0163] FIG. **159** shows an illustration of an Indirect IoN network topology.

[0164] FIG. **160** shows an illustration of a NUTserver hub and its connections to the NUThub and IoN devices from FIG. **159**.

[0165] FIG. **161** shows a block diagram of the NUThub/IoN Interface in a NUTserver hub from FIG. **160**.

[0166] FIG. **162** shows a block diagram of the NUThub/NUTserver/IoT Interface in an IoN Device from FIG. **160**.

[0167] FIG. **163** shows a flowchart of the registration and configuration process for IoN/IoT devices.

[0168] FIG. **164** shows a flowchart of how the Remote Control Interface may process Command Nuts from FIG. **161** and FIG. **162**.

[0169] FIG. **165** shows an illustration of the network layout of a NUTS Certification Server.

[0170] FIG. **166** shows a block diagram highlighting the functionalities of a NUTS Certification Server from FIG. **165**.

[0171] FIG. **167** shows an illustration of the network layout of a NUTS Based WiFi/Ethernet router.

[0172] FIG. **168** shows a flowchart of how messages may be processed in a NUTS Based WiFi/Ethernet router from FIG. **167**.

[0173] FIG. **169** shows a table of device categories for a NUTS Based WiFi/Ethernet router.

[0174] FIG. **170** shows a table of example device category attributes on a NUTS Based WiFi/Ethernet router.

[0175] FIG. **171** shows a block diagram of how Application Wrapping enables device backups and duplication in an automated manner.

[0176] FIG. **172** shows a block diagram of the Event Processing Service (EPS) in two devices.

[0177] FIG. **173** shows a block diagram of a typical vendor network setup which may use tracking cookies and session histories stored on Big Data servers.

[0178] FIG. **174** shows a block diagram of a vendor network setup which may use App Nuts to record a copy of session histories locally as well as being stored on Big Data servers from FIG. **173**.

[0179] FIG. **175** shows a block diagram of the Contextual Computing that may be done locally utilizing the App Nut from FIG. **173** and FIG. **174**.

[0180] FIG. **176** shows an illustration of a personal home network topology comprising IoT devices and service providers.

[0181] FIG. **177** shows an illustration of a personal home network comprising two IoN devices and their respective service providers in an Indirect IoN network topology to control the flow of data going out to the vendors.

[0182] FIG. **178** shows a block diagram of how Contextual Analysis Apps may automatically filter outgoing IoN messages to protect the user's privacy in the NUTserver from FIG. **177**.

DETAILED DESCRIPTION

TABLE-US-00001 Table of Contents Smbols & Abbreviations Ciphers & One Way Hashes
 Network Diagram Device Diagram Transmutations Transmutation Types Transmutation Structures
 Transmutation Audit Records (TAR) Structured Data Folding with Transmutations (SDFT) Nut ID
 Lock Graphs and Lock Nodes Keyholes Variable Locks Stratum Nut Access Control (NAC) Lock
 Node Traversal Modular I/O Reading and Writing Backward Compatibility Forward Compatibility
 Display Application Nut History Nut Log Relationship Based Keys (RBK) Anonymous
 Relationships NUTS Core Applications NUTserver MIOR Server NUTbrowser/NUTshell
 NUTbook NUTS Based Services NUTmail NUTchat NUTcloud NUTnet NUThub NUTS
 Certification Server NUTS Based Wifi/Ethernet Router Application Wrapping Event Processing
 Service Contextual Computing Conclusion and Philosophy
 Symbols & Abbreviations

[0183] The following symbols and abbreviations may be used throughout the descriptions and drawings. Those marked with a (*) may be NUTS specific: [0184] AAKS *Access Attribute Key Set [0185] AAKSUK *Access Attribute Key Set Unlock Key [0186] AAPK *Access Attribute

Propagation Key [0187] acipher Asymmetric Cipher [0188] AEAD Authenticated Encryption with Associated Data [0189] AES Advanced Encryption Standard; also Rijndael [0190] API Application Programming Interface [0191] AKS *Access Key Set [0192] ARK *Access Role Key [0193] BIOS Basic Input/Output System [0194] bz2 bzip2, Burrows-Wheeler compression algorithm [0195] CA Certificate Authority [0196] CAC Cryptographic Access Control [0197] ChaCha20 symmetric key based stream cipher by Bernstein [0198] CLI Command Line Interface [0199] CMAC Cipher-based Message Authentication Code [0200] CODEC COder/DECoder; encoding scheme for character data [0201] COM Component Object Model [0202] COR *Class of Readers; or Reader [0203] CORBA Common Object Request Broker Architecture [0204] COW *Class or Writers; or Writer [0205] CPU Central Processing Unit [0206] CRC Cyclic Redundancy Check [0207] dign * (noun) a digital signature generated using an asymmetric private key [0208] dign *(verb) to create a digital signature using an asymmetric private key [0209] DK *Derived Key [0210] DRM Digital Rights Management [0211] DVD Digital Video Disk [0212] DSA Digital Signature Algorithm [0213] ECC Elliptic Curve Cryptography [0214] eDK *encrypted Derived Key [0215] EPS *Event Processing Service [0216] FIPS Federal Information Processing Standards [0217] HMAC Hash based Message Authentication Code [0218] GPS Global Positioning System [0219] GPU Graphics Processing Unit [0220] GUI Graphical User Interface [0221] GUID Globally Unique Identifier [0222] gzip GNU zip compression [0223] HKDF HMAC based Key Derivation Function [0224] ikm Initial key material [0225] IMEI International Mobile station Equipment Identity [0226] IoN *Internet of Nuts [0227] IoT Internet of Things [0228] IPC Inter Process Communication [0229] IPv4 Internet Protocol version 4 [0230] IPv6 Internet Protocol version 6 [0231] I/O Input/Output [0232] ima *KISS field name short for “I am a” or “I’m a”: determines KISS mode [0233] iv Initialization Vector: random number for cryptographic use [0234] JSON JavaScript Object Notation [0235] KBP *Key Based Permissions [0236] Keccak SHA3 hash family [0237] KISS *Key Interchange Specification Structure [0238] LAN Local Area Network [0239] lock *Implementation of Variable Locks as a class of transmutations [0240] Izma Lempel-Ziv-Markov chain Algorithm [0241] MAC Media Access Control (w.r.t. Ethernet) [0242] MAC Message Authentication Code [0243] MD5 Message Digest #5 by Rivest [0244] MIO *Modular I/O [0245] MIOR *Modular I/O Repository [0246] MMS Multimedia Messaging Service [0247] NAC *Nut Access Control [0248] NCS *NUTS Certification Server [0249] NFC Near Field Communication [0250] NIST National Institute of Standards and Technology [0251] NoSQL Non Standard Query Language; also non-relational Standard Query Language [0252] nonce Number only used ONCE: random number for cryptographic use [0253] NTFS New Technology File System (Microsoft) [0254] NUTS *eNcrypted Userdata Transit & Storage [0255] OAEP Optimal Asymmetric Encryption Padding by Bellare and Rogaway [0256] OS Operating System [0257] PBKDF2 Password Based Key Derivation Function #2 by RSA (PKCS) [0258] PGP Pretty Good Privacy [0259] PIM Personal Information Manager [0260] PKCS Public Key Cryptography Standards by RSA Laboratories [0261] PKCS1_V1.5 Version 1.5 of PKCS #1 [0262] PKI Public Key Infrastructure [0263] PSS Probabilistic Signature Scheme [0264] PUID Practically Unique ID [0265] QA Quality Assurance [0266] QUOPRI Quoted-Printable or QP encoding [0267] RAM Random Access Memory [0268] RAT *Root Access Tier, owner/creator of Nut; also RAT Writer, owner [0269] RBAC Role Based Access Control [0270] RBCAC Role Based Cryptographic Access Control [0271] RBK *Relationship Based Keys [0272] ROM Read Only Memory [0273] RSA Rivest-Shamir-Adleman public key cryptosystem [0274] SAC *Stratum Access Control [0275] Salsa20 symmetric key based stream cipher by Bernstein [0276] salt random number for cryptographic use [0277] scipher Symmetric Cipher [0278] SCP *Structured Cryptographic Programming [0279] SCRYPT a password based key derivation function by Percival [0280] SDF *Structured Data Folding [0281] SDFT *Structured Data Folding with Transmutations [0282] SHA Secure Hash Algorithm—Keccak hash variant [0283] Shake Keccak hash variant [0284] SMS Short Message Service [0285] SOAP Simple Object Access Protocol [0286] SPAM unsolicited

bulk email; also junk email [0287] SSD Solid State Drive [0288] SSID Service Set Identifier [0289] SSO Single Sign On [0290] tar Tape Archive: Unix command to store data onto tape or disk [0291] TAR *Transmutation Audit Record [0292] TOP *Transmutations Organizing Principle [0293] tine *Shamir's Secret Sharing share, like tines on a fork [0294] TMX *Transmutation [0295] TOP *Transmutations Organizing Principle [0296] URL Uniform Resource Locator [0297] UTF Unicode Transformation Format [0298] UTI Uniform Type Identifier [0299] UUID Universally Unique Identifier [0300] VPN Virtual Private Network [0301] WAN Wide Area Network [0302] WiFi WLAN protocol [0303] WLAN Wireless LAN [0304] XML eXensible Markup Language [0305] Zlib zlib compression algorithm

[0306] FIG. 1 shows a table of cipher key types and their respective symbols which may be used throughout the descriptions and drawings of this disclosure. A variable length text based password or passphrase may be represented by symbol **102**. Symbol **104** represents a key for a symmetric cipher comprising AES-256 or alternative cipher. Symbol **106** represents a key pair for an asymmetric cipher comprising RSA-2048 or alternative cipher. The public portion of an asymmetric key pair **106** may be depicted as symbol **108** and the private portion may be shown as symbol **110**. A person having ordinary skill in the art may readily recognize that these ciphers may be well known and well tested algorithms and that other suitable methods may be substituted where these methods may be specified in this disclosure when standards change or an alternative may be desired.

Ciphers & One Way Hashes

[0307] FIG. 2 depicts the basic operations which may be performed by various types of ciphers. A symmetric cipher **208** in an encrypting mode may accept a symmetric key **202** and data **204** to produce encrypted data **206** or ciphertext. A symmetric cipher **208** in a decrypting mode may accept the same symmetric key **202** and ciphertext **206** to produce the original data **204**. In implementations of a symmetric cipher, the encryption and decryption methods may be two separately named function calls or may be a singular call with a mode parameter as part of the inputs. A characteristic of a symmetric cipher may be that both encryption and decryption processes may utilize the same secret key **202**.

[0308] An asymmetric cipher **214** in an encrypting mode may accept the public portion of an asymmetric key pair **210** and data **204** to produce encrypted data **212** or ciphertext. An asymmetric cipher **214** in a decrypting mode may accept the private portion of an asymmetric key pair **216** and ciphertext **212** to produce the original data **204**. In implementations of an asymmetric cipher, the encryption and decryption methods may be two separately named function calls or may be a singular call with a mode parameter as part of the inputs. A characteristic of an asymmetric cipher may be that the encryption and decryption processes may utilize different parts of a key pair. In an implementation such as RSA-2048, a public key may be derived from the private key using a mathematical relationship therefore an RSA-2048 private key may be synonymous with the key pair and the public key may be extracted from it.

[0309] A digital signature method **222** in a signing mode may accept the private portion of an asymmetric key pair **216** and ciphertext **218** to produce a digital signature **220**. The digital signature method **222** in an authentication mode may accept the public portion of an asymmetric key pair **210**, digital signature **220** and ciphertext **218** to authenticate **224** whether the digital signature was created using the said ciphertext **218** and private portion of an asymmetric key pair **216**. In implementations of a digital signature method, the signing and authentication methods may be two separately named function calls or may be a singular call with a mode parameter as part of the inputs. A characteristic of a digital signature method may be that the signing and authentication processes may utilize different parts of a key pair. In an implementation such as a digital signature method based on RSA-2048 key pairs, a public key may be derived from the private key using a mathematical relationship therefore an RSA-2048 private key may be synonymous with the key pair and the public key may be extracted from it. For brevity and conciseness, this document may

interchangeably refer to a digital signature as a dign; an act of digitally signing a piece of data may be interchangeably referred to as digning; having digitally signed a piece of data may be interchangeably referred to as digned.

[0310] A digital signature method may be a type of message authentication code or MAC. MACs may be created with one way hash algorithms on data. A hash method such as SHA-512 may accept data content to produce a message digest of it which may be up to 512 bits in length.

Authentication of MACs using methods such as SHA-512 entails recalculating the MAC on the said piece of data and comparing the provided MAC and the calculated MAC for equality. A technique known as keyed hash message authentication code or HMAC may take in an additional input of a cryptographic key along with the data content to produce an HMAC value.

[0311] Digital signature methods and/or hashing methods may be used in various parts of this disclosure to produce message digests that may be representative of the respective data.

Network Diagram

[0312] FIG. 3 represents a simplified network diagram in which various embodiments of this disclosure may be applied to in part or whole. A wide area network WAN **302** may be represented as a network cloud which may comprise many servers, routers and switching systems at various telecommunication centers working together to provide a simplified network cloud view to a user or company. Cloud Services **304** may be also pictorially simplified as a cloud which represents various commercial systems that may provide network services such as cloud storage and cloud processing; these services may be implemented to their own specifications but may comprise many instances of server farms, storage arrays and routers. A personal router **306** may be connected to the WAN **302** which may give individual users access to the Internet among a plurality of network connectible devices **308-318** that the user may have. A user may not be limited to the devices depicted on the diagram but may use any device that may utilize a router to access other devices on the same network or across the Internet. The router **306** may be a dedicated routing device to the Internet service provider or it may be a combination device providing routing and/or LAN and/or WLAN capabilities and may be referred to as a gateway. A corporate router **320** may be connected to the WAN **302** which may give institutional users access to the Internet among a plurality of network connectible devices **302-330** that the company may have. A company may not be limited to the devices depicted on the diagram but may use any device that may utilize a router to access other devices on the same network or across the Internet. The router **320** may be a dedicated routing device to the Internet service provider or it may be a set of interconnected and managed routers providing routing and/or LAN and/or WLAN capabilities and may be referred to as a gateway and/or intranet. The system and method described herein in various embodiments, may be used and applied to some or all parts of this network diagram.

Device Diagram

[0313] A generic computing device **400** is depicted in FIG. 4. A processing unit **404** may be connected to a system bus **402** which may facilitate some or all internal communications and data transfers within the device. There may be several different types of system buses available but for simplicity they may be collectively referred to as the system bus **402**. The processing unit may represent a single or multi-cored processor as well as arrays of processors such as those found in various specialized processing boards such as GPU boards and blade servers. Other components serviced by the system bus may be Network Adapters **412**; I/O Interfaces **410**; Display Interfaces **414**; Read Only Memory ROM **406** which may store a BIOS program **408**; Volatile Memory RAM **416** which may ephemerally store the running Operating System **418**, running applications **420** and/or application data **422**; and Non-Volatile Memory **424** such as hard drives, SSD and flash drives **426** which collectively may persistently store the installed Operating System **428**, Applications **430**, and/or data files **432**.

[0314] Not all components of the depicted computing device may be necessary for some or all embodiments of this disclosure to be applicable and functional. For example, devices may not have

any physical displays nor I/O Interfaces as those found on some IoT devices; routers and gateways may have very little in the way of physical hard disks. A necessary requirement for NUTS support and compatibility may be the ability to run NUTS compatible software which may comprise a processing unit, some form of storage and a system bus.

Transmutations

[0315] Transmutations may be a preferred method of organizing the many known data manipulation operations found in computer programming. NUTS may designate this as the Transmutations Organizing Principle or TOP. Furthermore, any systematic data manipulation operation may be analyzed using TOP and may be classified as a type of transmutation. Then, the transmutation may be categorized, normalized, structured, integrated and/or adapted to work cooperatively within the framework of TOP which may be called Structured Data Folding with Transmutations or SDFT. The insightful perspectives of TOP and/or operating on data with SDFT may allow for better and/or complex data designs to be implemented in a conceptually simpler and/or programmatically efficient manner. TOP and SDFT may be the preferred lower level implementation mechanisms for NUTS components.

[0316] The analyses, methods and/or structures based on the transmutation of data may show how layering such concepts and designing their associated methods may define an implementable set of integrated data structures and algorithmic methods which may allow for the facile and systematic transmutations of data in a modular, portable, storable and/or self-describing way. Due to the layered and intertwining nature of such analyses, the descriptions of transmutations may have forward and backward references and may require the reader to reference different sections in order to gain a better appreciation of certain characteristics. Structured Data Folding with Transmutations (SDFT) builds upon transmutations using data structures and methodologies and may help enable the storability, transmissibility, modularity, portability, encapsulability and/or time compatibility of the transmuted data.

[0317] Within the NUTS design, SDFT is a set of low level operations and may be considered a fundamental building block to more easily construct a Nut. However, SDFT may be used independently, in part or whole, to simplify certain tedious and/or repetitive data transmutations within an application. SDFT may enable computer communication protocols to dynamically switch transmutation sequences and/or transmutation parametric variances within the same session between two different applications. Currently, such single session dynamic switching may be a non-trivial programmatic exercise. It may not be a necessary requirement to use SDFT in order to build a Nut but its features may help build a Nut more expediently, clearly and flexibly. SDFT may be further described as a data state transition methodology that allows for infinite variations of transition events with well-defined behaviors on the reversibility of the state transition sequences and may provide an iterative encapsulation technique to persist the necessary attributes and data in a simple context sensitive way. SDFT accepts and embraces the messiness of everyday programming issues and may present a pragmatic set of organizing principles where theoretical proofs may be subordinate to empirical proofs.

[0318] FIG. 5 shows how the Transmutations Organizing Principle may view any data operation as a data transmutation **510** which may require an input data source **502** and attributes **504** and which may output the transmuted data **512** and associated attributes **514**. Any well-defined manipulation, operation, conversion and/or transformation of data may be classified as a type of transmutation **510**. TOP may allow one to systematically construct a consistent set of methods of transmuting data in a modular, portable and/or self-describing way.

[0319] The table in FIG. 6 shows a sample set of common data operations and how they may be classified using TOP. Transmutations may encompass a class of fundamental data operations which may have been traditionally segregated in perception and in practice. Such may be the case when programmers discuss cryptography and data compressions, these two classes of data operations may typically be considered as two very separate and distinct operations on data. Beyond the

algorithmic differences of each operation, through the perspective of TOP, these operations may be viewed as a type of ciphering transmutation and a compression transmutation. In the table, a 'BON serialization' may be classified as a 'serialize' transmutation with an operation of 'j son', therefore an executable transmutation command may be stated as 'serialize j son'. An AES symmetric cipher encryption call on a piece of data may be classified as a 'scipher' transmutation with an operation of 'aes', therefore an executable transmutation command may be stated as 'scipher aes'. A person having ordinary skill in the art may readily recognize all the other types of data operations listed in the table and follow the organizing pattern of transmutation classification and operation categorization.

[0320] FIG. 7 shows a diagram of a transmutation in reverse mode or reversal operation. This figure is identical to FIG. 5 except for the data flow arrows which flow in the opposite direction. A transmutation **510** may have a well-defined reversible algorithm as illustrated by block **710**. A reversible transmutation **710** may require as input a transmuted data source **712** and attributes **714** and may output the original data **702** and associated attributes **704**. A field of computing may exist called Reversible Computing which may exhibit similar concepts as a reversible transmutation. There may be some differences in the goals of each organizing principle. Reversible Computing may theorize the existence of a generalized reversible computing language whose operations may be implemented down to the silicon layers for a possible energy efficiency of general computations. Reversible transmutations may aim to concretely implement TOP for benefits such as, but not limited to, minimizing written code, minimizing programmatic errors, convenient key management, simplifying key generation, structuring portable self-describing data, normalizing data manipulation concepts, introducing programming language independent methods of performing transmutations, and/or simplifying the building of complex cryptographic data structures.

[0321] FIG. 8 shows a pictorial representation of an irreversible transmutation. A transmutation **810** in forward mode may perform a transmutation on data **802** and attributes **804** which may produce transmuted data **812** and attributes **814** but these outputs along with the type of manipulations the transmutation may perform on the inputs may be of an irreversible nature. Such irreversible transmutations may be exemplified by hashes, MACs, lossy data compressions and other one way functions or data manipulations. TOP may introduce analysis techniques which may peripherally augment the characteristics of such irreversible transmutations and may produce operations which may define their reverse transmutation characteristics.

[0322] FIG. 9 shows a block diagram of a conditionally reversible transmutation. Such a transmutation **910** may have a well-defined reversible algorithm but may require additional inputs and/or attributes **914** for the reversal operation to succeed. A conditionally reversible transmutation **910** may require as input a transmuted data source **912** and/or attributes **914** and may output the original data **902** and/or associated attributes **904**, if and when, the requisite conditions **916** are satisfied, otherwise it may fail with an error condition **920**. Ciphers which may require keys may be classified as conditionally reversible transmutations since the absence of the correct key (an attribute) may hinder the decryption of the ciphertext.

[0323] FIG. 10 is a table listing common data operations and functions and their respective transmutation classifications. A person having ordinary skill in the art may recognize some or all of the data operations and/or functions listed in the table. For exemplary purposes, the material presented in this document may reference the programming language called Python version 3.6 and its syntax, concepts, function and/or methods. Many of the cryptographic functions referenced in this disclosure may be found in the Python Standard, pyCryptodome, secretsharing and/or pyCrypto libraries. A person having ordinary skill in the art may find equivalent syntax, concepts, function and/or methods in most modern programming languages and their respective libraries. Note that a 'dign' is a transmutation for digital signatures; other mnemonics may be listed in the SYMBOLS & ABBREVIATIONS section of this document. Further detailed descriptions of each transmutation may be found in the table in FIG. 12.

[0324] FIG. **11** is a table of Codecs defined in Python v3.6. This list may not be complete due to the numerous Codecs in existence, the proliferation of new Codecs and/or the limitations of those defined in Python v3.6. A 'Codec' is short for Code/Decode and is a mapping for a character set in computing. A character is assigned an arbitrary but unique binary value within a single Codec; thus a complete set of characters may define a single Codec. Not all characters within a given Codec may be human readable or printable. A common usage of Codecs is for the proper representation of different character sets of different languages. The 'encode' transmutation may be able to perform any of these Codec encoding operations on a given data string. The Codecs with names starting with 'utf_' may specify those conforming to the Unicode Transformation Format (UTF) which may be the organizing principle of international character sets for many internet based standards.

[0325] FIG. **12** is a table listing the transmutations discussed thus far with their detailed descriptions. Additional transmutations may be defined within the framework using TOP as shown by the last six transmutations in the table: key, clean, TAR group, press, lock and mobius. Some of these additional transmutations may be specific to the Structured Data Folding with Transmutations (SDFT) library, some may be language specific and/or some may be operations related to NUTS. This may illustrate the flexible nature of TOP by allowing new transmutation types to be defined and classified to expand its repertoire. This flexible expansion feature is by design so that the SDFT library may accommodate new transmutation operations in the future. The expansion feature may also allow for older versions of transmutation operations to be retroactively added for backwards compatibility. A benefit of such flexibility may be the ability of SDFT processed data to acquire a characteristic of better time compatibility. Time compatibility of data may be defined as those characteristics which may enable stored data to be easily processed by the same or different application at some future point in time. Time incompatibilities may arise from, but are not limited to, application file format version differences, different character encodings, obsolete application file formats, data operation method differences, data operations sequencing differences and/or data operation specific parametric variances.

[0326] FIG. **13** shows a transmutation reversibility matrix. Each transmutation may be designated as being reversible, irreversible and/or conditionally reversible. The criteria for making such a designation may be based on the ideal intentions of the transmutation rather than its implemented and/or theoretical shortfalls in practice. In other cases, the designation may be arbitrary. This may be illustrated by a digest operation called MD4 which may produce 128 bit long hashes of source data. The MD4 hash may be considered a severely weak hashing algorithm as compared to a hash operation such as 512 bit SHA2 due to its susceptibility to collisions which may be an undesirable trait in hashing algorithms. The TOP perspective may be to recognize one of MD4's original intentions to be an irreversible unique hash and categorize it in that fashion. Such categorization may not exclude this type of transmutation from gaining a well-defined, engineered reversibility characteristic through additional TOP analysis as will be shown in a later section. The compress transmutation may fall under both reversible and irreversible designations based on the particular compression operation being performed. Many image and/or audio compression techniques may exhibit irreversibility due to their sampling nature; a 12 MB digital image may be compressed down to 360 KB for efficient transmission via a chat application, but due to the nature of human visual perception, the general impression of the image may be properly conveyed despite the permanent data loss. Such a compressed image may be irreversibly modified due to the tremendous amount of original data that may have been discarded during the transformation.

[0327] A reversible compression transmutation may be exemplified by a gzip compression; it may operate on the principle of identifying and reducing repetitive bit patterns within the binary data but it may maintain enough information to reverse the process and reproduce the original data in its entirety. A conditionally reversible transmutation may be exemplified by the AES symmetric cipher; it may operate on the principle of taking in cleartext and a symmetric key and producing ciphertext. The decryption process may take the key and ciphertext to produce the original

cleartext. Thus, the presentation of the correct symmetric key for the ciphertext may be the necessary condition which must be satisfied to decrypt the ciphertext or reverse the encryption process.

[0328] TOP may define a transmutation mode which may indicate the direction of a given transmutation operation as either Forward or Reverse. The forward mode of a transmutation may perform its normal process and/or its engineered forward process. The reverse mode of a transmutation may perform its inherent reverse process and/or its engineered reverse process. The table in FIG. 14 shows a matrix indicating the type of operation a transmutation may perform internally based on its transmutation mode. As a reference, the table lists the commonly known operation names such as ‘serialize’ and ‘deserialize’, or ‘encrypt’ and ‘decrypt’. Note the engineered reverse processes of digest and dign: ‘digest’ and ‘verification’, ‘sign’ and ‘authentication’. For the ‘clean’ transmutation where it may delete various internal data associated with its transmutation data structure, it may be impossible to reinstate such deleted data without proper additional data and/or the rerunning of the forward transmutation process on the original data to reproduce the deleted transitory data. The ‘key’ transmutation may perform key generation and/or management operations related to performing transmutations. As such, due to the inherent random nature of key generation, it may be impossible to theoretically and/or algorithmically reverse such a process in a deterministic fashion in finite time. The key management aspect of the ‘key’ transmutation will be discussed in detail in a later section when we address how transmutations may work within the context of Structured Data Folding (SDF); the key management aspect of a key transmutation may be difficult to engineer a reversible counterpart due to its characteristic of setting up the proper key structures for a successful processing in an SDF context.

[0329] FIG. 15 shows a series three diagrams, each of which further details a serialize transmutation **1504** example which may be designated a reversible transmutation. In computer programming, a serialize and/or marshalling technique may take a complex language specific internal data structure and may systematically deconstruct and/or organize its contents in a linear fashion to produce an equivalent data string or set of data strings (henceforth referred to as a data string). The data string form may be more suitable for permanent storage, data transmission and/or further transmutations. A serialization by definition may require it to be completely reversible in a logical fashion to reconstruct the original content in the originating language or its equivalent. A Python structure **1512** may be transmuted using the JSON operation **1514** to produce an equivalent JSON string **1516** and the reverse process may be possible as shown by the bidirectional process flow arrows. A simple tree data structure is shown in **1522** which may exemplify a complex Python data structure. The serialize transmutation **1524** may produce the equivalent string **1526** from **1522**. This output string **1526** may now be stored, transmitted and/or transmuted as the program progresses.

[0330] FIG. 16 shows a series three diagrams, each of which further details a digest transmutation **1606** example which may be designated an irreversible transmutation. This example shows the SHA2 hash operation as a digest transmutation **1616** which may require as inputs data **1612** and a digest length **1614** as the attribute **1604**. The SHA2 hash digest transmutation **1616** may produce a hash of specified length **1618**. A Python data string **1622** and the desired digest length **1624** of 256 bits may be inputs to the SHA2 hash transmutation **1626** to produce a 256 bit long hash string **1628**.

[0331] FIG. 17 shows a detailed example of a digest transmutation in reverse mode also known as a verification. In TOP, this may be referred to as an engineered reversal of a transmutation. A digest transmutation **1710** may accept as inputs data D1 **1702** and attributes A1 **1704** to perform a forward mode digest transmutation **1710** which may produce as output **1708** a digest string DG1 **1712**. The reverse mode of this transmutation **1720** may accept as inputs **1736** data D1 **1722**, attributes A1 **1724** and digest string DG1 **1728** to perform a reverse mode digest transmutation **1720** which may

produce as output **1738** a flag and/or value indicating whether the digest string DG1 **1728** was Verified **1732** or has Failed Verification **1734**. The process of verification **1740** may produce a digest string DG2 **1722** by performing a forward mode digest transmutation **1720** on inputs D1 **1722** and A1 **1724**. The output digest string DG2 **1722** may then be compared for equality **1730** versus the input digest string DG1 **1728**. The result of the comparison **1738** may be presented in some form to show whether the reverse digest transmutation succeeded or not. In this way, the engineering of this digest reversal may require the forward mode of the transmutation to be re-processed and to comparing outputs rather than relying on any workarounds to finding a logical reversibility of such operations which may be difficult, time consuming and/or unattainable.

[0332] FIGS. **18**, **19** and **20** show a detailed example of a scipher transmutation in forward and reverse mode also known as a symmetric cipher. A scipher transmutation in forward mode **1806** may accept as input cleartext **1802** and attributes **1804** to produce as output ciphertext **1810** and/or attributes **1812**. A scipher transmutation in reverse mode **1826** may accept as input ciphertext **1830** and attributes **1832** to produce as output cleartext **1822** and/or attributes **1824**. FIG. **19** illustrates how the salsa20 symmetric cipher may operate as a scipher transmutation. A scipher salsa20 transmutation in forward mode **1906** may accept as input cleartext **1902** and attributes **1904** to produce as output ciphertext **1910**. The attributes for this particular cipher in forward mode may comprise a binary symmetric key, the length of the key and/or a salt value. This salsa20 forward (encryption) implementation may be a streaming cipher and no additional output attributes may be generated during the encryption process besides whatever hidden attributes the function may embed within its own output string. A scipher salsa20 transmutation in reverse mode **1926** may accept as input ciphertext **1930** and attributes **1932** to produce as output cleartext **1922**. The attributes for this particular cipher in reverse mode may comprise a binary symmetric key, the length of the key and/or a salt value. This salsa20 reverse implementation (decryption) may be a streaming cipher and no additional output attributes may be generated during the decryption process. FIG. **20** illustrates how the salsa20 symmetric cipher may operate as a transmutation on sample data as may be represented within Python v3.6 and the PyCryptodome library function. A scipher salsa20 transmutation in forward mode **2006** may accept as input a data string **2002** and attributes **2004** comprising a 256 bit symmetric secret key and a nonce (as salt) to produce as output ciphertext **2010**. In Python, the symmetric key may be represented as a 'bytes' datatype string and thus a key length attribute may be easily derived by the len() function on the key bytes string. A scipher salsa20 transmutation in reverse mode **2026** may accept as input ciphertext **2030** and attributes **2032** comprising a 256 bit symmetric secret key and a nonce to produce as output cleartext **2022**. In Python, the symmetric key may be represented as a 'bytes' datatype string and thus a key length attribute may be easily derived by the len() function on key bytes string. The attributes **2032** may be required to be equivalent to the attributes **2004** in order for this conditionally reversible transmutation to properly process the ciphertext **2030** in reverse mode (decrypt) to recover the original cleartext **2002**.

Transmutation Types

[0333] In the following tables and examples presented in FIG. **21** through FIG. **35**, each transmutation may not be limited to the operations specified in this table; any suitable operation may be analyzed through TOP and may then be integrated into the framework to extend the operational capabilities of the particular transmutation. Python v3.6 syntax and constructs may be used to illustrate examples in more detail. Equivalent data types, structures, syntax and/or methods may be found and substituted in different programming languages by a person having ordinary skill in the art. In some cases, a key/value option may not be relevant to a particular language or library and it may be ignored or modified as needed as long as the processing may produce equivalent results.

Serialize/Compress Transmutation

[0334] FIG. **21** shows a table of command specifications for serialize and compress transmutations

2102 and a set of sample transmutation commands showing its usage **2104**. The table **2102** lists the transmutation name and the acceptable operation types for each transmutation. Any column header with a trailing '=' indicates that the value presented beneath it may be presented in a key/value format in the command syntax construction. The 'Input' and 'Output' columns may specify the expected data types/structures for the transmutation/operation within the context of Python v3.6. For example, the command 'serialize json sortkeys=t' may perform the following data manipulation sequences: take as input any Python data structure, perform a json.dumps() on it with the sort_keys' flag set to True, then output a Python string with the serialized version of the data. The reverse mode of this command may expect a JSON formatted Python string as input, perform a json.loads() on it, then output a Python data structure. The 'sort_keys' flag informs the json.dumps() function to process the keys of any Python dictionary structure in ascending order. Python v3.6 may not guarantee a consistent processing order for a dictionary structure when processing by keys, therefore the resulting JSON strings may be inconsistent between multiple runs of this transmutation on the same data structure. Sorting the keys in a particular order within the serialization transmutation may provide a consistency in the processing sequence resulting in identical JSON strings as output between multiple runs on the same data structure. This may become very important for the purposes of determining whether two JSON strings are equivalent and as such may represent two equivalent pre-serialization data structures.

[0335] The compress transmutation in table **2102** shows several different lossless compression operations or reversible compressions. Any irreversible or lossy compression operations may extend the compression transmutation repertoire but for the purposes of discussing reversible transmutations, it may be neither interesting nor constructive to discuss a one way function which may not provide a cryptographic purpose much beyond data size reduction. From a TOP perspective, lossy compressions may be analyzed and treated the same way as a digest transmutation which will be discussed in a later section. In the example in **2104**, the command compress bz2' may perform a bz2 compression on a binary string input and may produce a binary string output which may or may not be smaller in size than the input string. Some data may no longer be compressible using a particular compression scheme; an example of this may be where a bz2 compressed string may be processed again and no further data size reduction may be achieved.

Encode Transmutation

[0336] FIG. **22** shows a table of command specifications for an encode **2202** transmutation and a set of sample transmutation commands showing its usage **2204**. There may be numerous encoding schemes in computer science and the references in this table does not represent all known encoding schemes. The encoding schemes which are listed under the 'encoding=' column may be found in Python v3.6 and its associated standard libraries. A person having ordinary skill in the art may recognize the utility of having access to all these types of encodings towards solving an issue related to an application which may manipulate data. 'Codecs(98)' refers to the list of supported Codecs in Python v3.6 as of this writing and listed previously in the table in FIG. **11**. The transmutation command 'encode strbin utf_8' may take as input a Python string, perform a utf_8 Unicode encoding on it and output the results as a Python bytes string. The transmutation command 'encode utf utf_16' may take as input a Python string, perform a utf_16 Unicode encoding on it and output the results as a Python string. The transmutation command 'encode binascii hex' may take as input a Python bytes string, perform a hexadecimal encoding on it and output the results as a Python string. The transmutation command 'encode base 64' may take as input a Python bytes string, perform a base 64 binary encoding on it and output the results as a Python string. The transmutation command 'encode utf_8' is equivalent to 'encode utf utf_8'. These explanations may illustrate the consistency and types of permutations allowed in the encode transmutation command syntax.

Digest Transmutation

[0337] FIG. **23** shows a table of command specifications for a digest transmutation **2302** and a set

of sample transmutation commands showing its usage **2304**. A digest transmutation as shown in table **2302** defines three types of operations but are not limited to them: hash, hmac and/or cmac. The transmutation command ‘digest hash md5 128’ may take as input a source data string, perform an MD5 hash function on it and produce an output digest bytes string that is 128 bits in length. Note that the input source data string may not be modified and not overwritten during a digest transmutation call; the output digest bytes string may be additional data generated from a digest transmutation call and may be provided a separate memory space. The transmutation command ‘digest hash sha2 512’ may take as input a source data string, perform a SHA2 hash function on it and produce an output digest bytes string that is 512 bits in length. The transmutation command ‘digest hash shake256 digestlen=332’ may take as input a source data string, perform a SHAKE256 hash function on it and produce an output digest bytes string that is 332 bits in length. The transmutation command ‘digest hmac sha2 256’ may take as input a source data string, perform a HMAC function on it using a SHA2 hash and produce an output digest bytes string that is 256 bits in length. The transmutation command ‘digest cmac aes 256’ may take as input a source data string and a 256 bit symmetric key, perform a CMAC function on it using AES256 cipher and produce an output digest bytes string that is 128 bits in length. All of these digest transmutation example operations and types may be found in the standard Python library and/or the PyCryptodome library and may not represent all the variety of operations, types, digest lengths, key lengths and/or other parameters that may exist in a theoretical and/or implemented sense outside of these sample libraries. Any additional variations may be properly analyzed through TOP and integrated into a transmutation form. Such integrations for any transmutation form may require refactoring and retesting of the existing transmutation operations.

Acipher/Dign Transmutations

[0338] FIG. **24** shows a table of command specifications for an acipher and dign transmutations **2402, 2404, 2406** and a set of sample transmutation commands showing its usage **2410**. The transmutation command ‘acipher pkcs1_oaep 2048’ may take as input a bytes string and an 2048 bit long RSA asymmetric public key, perform a RSA PKCS #1 OAEP cipher operation on it utilizing a 512 bit SHA2 hash, and may produce as output a ciphered bytes string that is 2048 bits in length. The transmutation command ‘acipher pkcs1_v1_5 3072’ may take as input a bytes string and an 3072 bit long RSA asymmetric public key, perform a RSA PKCS #1 v1.5 cipher operation on it, and may produce as output a ciphered bytes string that is 3072 bits in length. The reverse mode of these acipher transmutations may require as input the ciphertext as a bytes string and the private portion of the appropriate RSA key in order to produce the original cleartext.

[0339] The transmutation command ‘dign pkcs1_v1_5 2048’ may take as input a bytes source string and an 2048 bit long RSA asymmetric private key, perform an RSA PKCS #1 v1.5 digital signature operation on it utilizing a 512 bit SHA2 hash, and may produce as output a digest bytes string that is 2048 bits in length. Note the term ‘digest bytes string’ may be used interchangeably with ‘digital signature bytes string’ because TOP may view these outputs as providing a similar functionality and thus may store such a bytes string referred to by a ‘digest’ variable name. The transmutation command ‘dign dss 1024 hashtyp=sha2’ may take as input a bytes source string and an 1024 bit long DSA asymmetric private key, perform a DSS digital signature operation on it in a FIPS-186-3 mode utilizing a 512 bit SHA2 hash, and may produce as output a digest bytes string that is 1024 bits in length. The transmutation command ‘dign dss 256’ may take as input a bytes source string and an 256 bit long ECC asymmetric private key, perform a DSS digital signature operation on it in a FIPS-186-3 mode utilizing a 512 bit SHA2 hash, and may produce as output a digest bytes string that is 256 bits in length. The reverse mode of these dign transmutations may require as input the digest bytes string (digital signature), the source bytes string and the public portion of the appropriate asymmetric key in order to authenticate it.

Derive Transmutation

[0340] FIG. **25** shows a table of command specifications for a derive transmutation **2502, 2504,**

2506 and a set of sample transmutation commands showing its usage **2510**. The sample operations pbkdf2, hkdf and scrypt may also be known as key derivation functions and/or key stretching functions. The basic functionality of a derive transmutation may be to derive a symmetric key or keys of a desired length from a binary or character data string that may be known to the user; a common usage of a key derivation function may be to derive a properly formed symmetric cryptographic key(s) from a password or passphrase. The transmutation command ‘derive pbkdf2 keylen=256 iterations=100000’ may take as input a character data string (password or passphrase), perform a PBKDF2 operation on it using a SHA2 512 bit hash function, a randomly generated 512 bit initialization vector as a salt and an iteration count parameter set to 100,000, and may produce a corresponding symmetric key that is a 256 bits long bytes data string. The transmutation command ‘derive hkdf keylen=256 numkeys=4’ may take as input a bytes data string, perform a HKDF operation on it using a SHA2 512 bit hash function, a randomly generated 512 bit initialization vector as a salt, and may produce a corresponding set of four related symmetric keys each of which is a 256 bits long bytes data string. The transmutation command ‘derive scrypt keylen=128 mode=login’ may take as input a data string, perform a login mode SCRYPT operation on it using a randomly generated 512 bit initialization vector as a salt, and may produce a corresponding symmetric key that may be a 256 bits long bytes data string. The login mode of a derive scrypt transmutation may be shorthand for setting the three parameters n, r, and p to the values indicated in table **2506**. These parameter values may be the suggested settings of the author of the SCRYPT algorithm.

[0341] The TOP approach to derive transmutations may suggest a bimodal operation. Data mode: if the transmutation may be engaged with no keystack (to be discussed in detail in a later section) and only a data source string of some type, it may transmute this input data source string and replace it with the output of the transmutation which may be in the form of a symmetric key(s). Key mode: if the transmutation may be engaged with a keystack and a data source of some type, it may transmute the corresponding key source material present in the keystack and may replace the key source material thereby deriving a cryptographically usable symmetric key(s) and placing it in the keystack. These statements may be clarified further in a later section when keystacks and key management are discussed within the context of a Transmutation Audit Record or TAR and dependent transmutations.

Scipher Transmutation

[0342] Using TOP, symmetric cipher operations may be classified as scipher transmutations, and as a group, these transmutations may present a set of associated attributes which may be extensive both in number and/or variety. The next three figures illustrates how TOP may systematically normalize and encapsulate each scipher transmutation with all its attributes into the same output string. This type of attribute embedding techniques may be found in various functions and libraries for many types of operations. However, there may be very few widely accepted standards for such embedding techniques. TOP may propose a consistent methodology which may apply to all scipher transmutations for the distinct purposes of supporting a feature called Structured Data Folding with Transmutations or SDFT. Whether such a methodology may become a widely used standard may be beyond the scope of this document, but the reader may recognize the possible benefits of its usage within the TOP framework especially when we later discuss TAR and SDFT constructs and methods.

[0343] FIG. **26** shows a table of command specifications for a scipher transmutation **2602** and a set of sample transmutation commands showing its usage **2604**. The table shows three types of scipher operations: aes, chacha20 and salsa20. This is not a complete list of known symmetric ciphers but it may present a relevant variety of them to illustrate how TOP may organize them and propose their usage. Symmetric ciphers may have the following attributes associated with them more or less: key length (keylen), operating mode (mode), salt type (salttyp), salt length (saltlen), block size (Block), cipher operation type (type), and/or padding (pad). A key length may specify the length of the

secret symmetric key which may be used in the cipher to produce ciphertext from cleartext. For AES ciphers, they may have at least ten different operating modes as shown in the table. Most symmetric ciphers may require the input of a salt (random value) of a particular type (iv or nonce) and particular salt length whose use may promote better semantic security. The symmetric cipher may provide at least three different operation types: block, stream and/or AEAD ciphers. Newer modes may be proposed and they may be integrated using TOP as an additional transmutation variant. Block mode ciphers may necessitate additional attributes comprising padding methodology, pad positioning, pad type and/or pad length.

[0344] In the examples in section **2604**, a transmutation command ‘scipher aes 256 mode=ofb’ may take as inputs a bytes data string and a 256 bit symmetric key, encrypt the input data string using the AES-256 OFB mode streaming cipher with the presented key and a randomly generated 128 bit initialization vector, and produce an output string that may be composed of the ciphertext and all the associated attributes involved in the process embedded in the header of the output bytes string formatted in a consistent key/value format as specified in FIG. 27 (to be discussed in a later section). A transmutation command ‘scipher aes 128 mode=gcm’ may take as inputs a bytes data string and a 128 bit symmetric key, encrypt the input string using the AES-256 GCM mode AEAD streaming cipher with the presented key and a 128 bit nonce, and produce an output bytes string that may be composed of the ciphertext and all the associated attributes involved in the process embedded in the header of the output string formatted in a consistent key/value format as specified in FIG. 27. AEAD is an acronym for Authenticated Encryption with Associated Data and may be a standardized or well-known method of embedding an authentication functionality along with the ciphering capability of the symmetric cipher within a single function call. A transmutation command ‘scipher chacha20 256’ may take as inputs a bytes data string and a 256 bit symmetric key, encrypt the input string using the CHACHA20 streaming cipher with the presented key and a 64 bit nonce, and produce an output string that may be composed of the ciphertext and all the associated attributes involved in the process embedded in the header of the output string formatted in a consistent key/value format as specified in FIG. 27. A transmutation command ‘scipher salsa20 128’ may take as inputs a bytes data string and a 128 bit symmetric key, encrypt the input string using the SALSA20 streaming cipher with the presented key and a 64 bit nonce, and produce an output string that may be composed of the ciphertext and all the associated attributes involved in the process embedded in the header of the output bytes string formatted in a consistent key/value format as specified in FIG. 27.

[0345] FIG. 27 shows the output structure format for a scipher output string in a sequence of two steps where Step 1 illustrates the input format and Step 2 illustrates the output format. “Header” is the variable length key-value utf8 encoded parameter string of the scipher transmutation on the output message. In Step 1, a scipher may accept as input a message bytes string ‘Message’ of variable length with an optional padding of pad length usually placed at the end of the Message as needed. A Message may have been prepended with a salt value as may be recommended by the chosen cipher. The padding may be a necessary requirement of a block mode symmetric cipher. If no particular padding methodology is specified by the programmer, a default padding methodology may be used and appended to the end of the Message. This Message and padding may be referred to as the Plaintext. The chosen cipher may now process the input Plaintext and produce an output which may be called the Encrypted Message as shown in Step 2. The chosen scipher transmutation may now prepare the embedded Header as a printable key/value pairs in a character string format where the keys may represent the parameter type and the values represent their respective settings. The details of the key/value will be discussed in the next section. Once a Header string may be produced, the transmutation may calculate the length of this Header string referred to as the Header Size and may be formatted as a two byte long unsigned big-endian integer. Two bytes may range in value from 0 to 2^{sup.16} (65,536) and may be sufficient to describe all the attributes for any symmetric ciphers for the foreseeable future in this particular format. Then Step 2 may proceed to

create a Packed Message comprising Header Size, Header and Encrypted Message. This Packed Message may be the actual output string from the scipher transmutation therefore it may be considered to have successfully encapsulated and embedded all the attributes associated with the transmuted data. The data flows of a reverse scipher transmutation may follow this process in reverse: the transmutation command may specify the exact scipher transmutation to perform, a matching symmetric key and the Packed Message may be provided as inputs. The scipher transmutation may then unpack the Packed Message, read and store all attributes found in the Header, then prepare to decipher the Encrypted Message. A symmetric cipher may have no deterministic method to convey a successful decryption. A verification method may be used in an overlaid manner to determine such results. An example of a rudimentary method may be to extract the prepended salt value from the decrypted Message and compare it to the saved salt value from the Header. Matching salt values may indicate a successful decryption but may not guarantee it. AEAD mode symmetric ciphers may address this issue a degree better by embedding a MAC (CMAC, HASH or HMAC) of the data string (before or after the encryption) within the Encrypted Message and performing a comparison. More sophisticated methods may require the authentication of digitally signed hashes of some form of the Message using different keys. As may be shown in a later section, the use of SDFT and TARs may allow such sophistication in a procedurally simple and logical way. In all of these hash based methodologies, it may be deterministically impossible to fully state the condition of a decryption attempt due to the weaknesses inherent in a hashing scheme to uniquely identify data universally. One deterministic method may be to compare the decrypted Message with the original Message for equality but there may be trade-offs of efficiency for lengthy messages.

[0346] FIG. **28** shows a table of parameter keywords and specifications for the header string in the output structure format of a scipher transmutation. The keywords chosen for this attribute table may be sufficiently self-describing and/or self-explanatory to a person having ordinary skill in the art. Examples of attribute values are shown in the column on the right. The first attribute listed, Header Size, may be the only attribute that may be presented as a 16 bit binary unsigned big endian integer value and may be the first field present in the Header. This Header Size may indicate the number of bytes that follow which may describe the attributes of this particular scipher transmutation in a printable format. The attribute format may have been chosen to be printable to allow for the variability of attribute value ranges and lengths. All attribute values which may exist naturally as a binary string in the running program comprising salt values (salt_val) and MAC strings (mac_val) may be encoded into base64 to satisfy the preference of printable characters.

[0347] In this manner, the output string of a scipher transmutation may comprise one or more encapsulating layers of attributes depending on the particulars of the chosen scipher. FIG. **29** shows an illustration of iterative embedded message encapsulations for an AEAD mode scipher transmutation. An AEAD mode AES cipher may output the following layers listed from inner to outer layers. A message preparation layer **2910** may comprise the cleartext message to be ciphered **2914** combined with an appropriate salt value **2912**. This prepared message **2910** may be encrypted with the chosen AEAD cipher which may then additionally produce a MAC value and additional data **2922** as part of the cipher process and we may refer to this combined message as the AEAD cipher output **2920**. This AEAD cipher output **2920** may also be referred to as the Encrypted Message **2934**. The Encrypted Message **2934** may have associated attributes from the scipher process which may be parameterized using the keyword/value Header method from FIG. **28** to produce a Header **2932** and this combination may be referred to as the scipher Packed Message **2930**. This scipher Packed Message **2930** may be the output of the chosen scipher transmutation which may be stored into the obj pointer or variable **2944** of the NSstr structure **2940** that may be associated with the TAR that called the scipher transmutation. The structure of the NSstr will be discussed more fully in a later section. Also other attributes **2942** may be stored in this data storage unit called the NSstr structure comprising the TAR, keystack, digest and/or status flags. The obj

pointer or variable **2944** in the NSstr structure **2940** may have been the starting point of the cleartext message **2914**, thus an iterative path **2950** may be possible and may exist for the object **2944** for as many nested encapsulations as needed by the TAR it may be processing which itself may be stored in the attributes **2942** of the NSstr structure **2940**.

[0348] In the Header **2932** of the scipher Packed Message **2930**, parameters comprising the description of the symmetric cipher, its mode and attribute values used may be completely and exactly described by the keywords listed in FIG. **28**. In this regard, the TOP approach may not rely on the obfuscation and hiding of non-cryptographic processes and procedures for securing data but rather only on the theoretical and implemented security of the cipher being used as a transmutation. This may not seem significant on initial observation but it may be shown later that such clarity of associated details of data transformations embedded into the output of the transformation itself may eventually lend itself to novel methodologies and designs which may rely more on self-describing data than hardwired programs to properly process it. This approach may help formulate one of the fundamental primitives in Data Centric designs and Data Centric models of operating on data at some of the lowest layers of data storage science. NUTS may rely heavily on Data Centric designs and models as may be shown in a later section.

[0349] FIG. **30** shows a table of command specifications for a lock transmutation **3002** and a set of sample transmutation commands showing its usage **3010**. A lock transmutation is one of the additional transmutations as listed in the table in FIG. **12** and it may be an embodiment of a Variable Lock from NUTS as will be described in detail in FIG. **65-77**. Variable Locks may present several different methods of cryptographically locking a secret message comprising sslock, orlock, matlock, xorlock and hashlock. A feature of Variable Locks may be the ability to input and use several cryptographic keys in the process of ciphering a secret message in a normalized, systematic and orderly way. The TOP approach may allow a compact method of implementing such locking techniques in a simple and convenient fashion. In the examples of section **3010**, a transmutation command 'lock orlock 128 numkeys=10 scipherkeylen=128' may take as inputs a bytes data string and up to ten 128 bit identical symmetric keys, encrypt the input string using a 'scipher aes 128 mode=eax' transmutation command, and produce an output string comprising the ciphertext and associated embedded attributes. A transmutation command 'lock matlock 256 numkeys=5' may take as inputs a bytes data string and five 256 bit symmetric keys, iteratively encrypt the input string using a 'scipher aes 256 mode=eax' transmutation command for each key, and produce an output string comprising the ciphertext and associated embedded attributes. A transmutation command 'lock sslock 256 numkeys=4 threshold=2' may take as inputs a bytes data string and at least two 256 bit times256 secret sharing keys, encrypt the input string using an implementation of Shamir's Secret Sharing method with the supplied secret shares, and produce an output string comprising the ciphertext and associated embedded attributes. A transmutation command 'lock sslock_b 256 numkeys=6 threshold=3' may take as inputs a bytes data string and at least three 256 bit timesidx128 secret sharing keys, encrypt the input string using an implementation of Shamir's Secret Sharing method with the supplied secret shares, and produce an output string comprising the ciphertext and associated embedded attributes. A transmutation command 'lock xorlock 128 numkeys=6' may take as inputs a bytes data string and six 128 bit symmetric keys, derive a calculated key by iteratively performing XOR operations on the supplied keys, encrypt the input string using a 'scipher aes 128 mode=eax' transmutation command, and produce an output string comprising the ciphertext and associated embedded attributes. A transmutation command 'lock hashlock 192 numkeys=7' may take as inputs a bytes data string and seven 192 bit symmetric keys, derive a calculated key by performing a hash on the ordered concatenation of the supplied keys, encrypt the input string using a 'scipher aes 192 mode=eax' transmutation command, and produce an output string comprising the ciphertext and associated embedded attributes.

[0350] Each Variable Lock type description and mode of operation may be found in later sections on Variable Locks starting with FIG. **60**. The TOP analysis and methods may allow for complex

iterative locking variations potentially utilizing a plurality of cryptographic keys to be done in a concise logical manner and may allow for facile extensions of different types of locking algorithms in the future. And, it may be shown later that the key management aspect of SDFT may allow a programmer to conveniently generate and manage such plurality of cryptographic keys with relative ease.

[0351] As presented in FIGS. **12**, **13** and **14**, TOP analysis and methods may allow a person having ordinary skill in the art to take a given data manipulation function and determine its suitability for normalization into a transmutation operation and type. The table in FIG. **12** may show a sampling of very well known data manipulations and may very well be considered adequate for use by a wide audience of developers. However, in such cases where a data manipulation function may not be found in this table, it may be possible to analyze and tailor the function to operate withing the SDFT framework using TOP methods; functions such as but not limited to lossy compression, bit scattering, message dispersals, erasure coding (ECC) and message level RAID encoding and structuring. In most cases of such transmutation extensions, it may be unnecessary to recode or rewrite the actual data manipulation function. In fact it may be counterproductive and procedurally weak to do that in most circumstances. The library containing the data manipulation function may be accessed by the Transmutation library and the TOP method may allow a developer to provide a normalizing wrapper function around the particular data manipulation function to behave well withing the SDFT framework.

[0352] Transmutation Structures, Transmutation Audit Records (Tar) and Structured Data Folding with Transmutations (Sdft)

[0353] FIG. **31** shows the specifications of various transmutation structures in tabular format. The structural definition relies on a nested key based approach similar to how structures are defined within Javascript; this may be an intentional design choice so that its representation may be readily duplicated in a wide variety of programming languages and may not rely on the peculiarities of a particular language. For example, Python v3.6 allows for classes to be defined whereas some languages such as Javascript may not, therefore transmutation data structures may not rely on classes to define it for wider applicability. A transmutation structure may provide a well-defined working memory area whereby the inputs and outputs of a transmutation may be prepared, processed and/or stored. The main data storage unit or structure that may be used in most if not all transmutation operations is called the NSstr structure **3108**. There may be at least one instance of NSstr associated with a transmutation call. All transmutation structures may have a 'typ' or structure type field specifying what structure it represents. The NSstr structure further may define a 'state' field specifying the state of the structure and/or its data. The 'obj' or object field may either hold a single value or it may be a pointer that references another area of memory. The obj field may be where the input data to most transmutations may be found. Also the obj field may be where the output data of most transmutations may be found. The 'digest' field, if it exists, may store the digest of the data stored or referenced by the obj field. The manner in which the digest may have been produced may be dependent upon the particular dign or digest transmutation and the supplied parameters and/or attributes to the transmutation command. The 'keystack', if it exists, may be a single instance of a KISS (Key Interchange Specification Structure, to be discussed in a later section) structure or it may be a list of KISS structures in a predetermined order that corresponds to its operating TAR. A keystack basically holds the secret key(s) of various types which may be needed by certain transmutations. The 'tar' field may point to an instance of an NStar structure **3106**.

[0354] The NStar structure **3106** may specify the particular Transmutation Audit Records (TAR) that may be applied to the input data stored in the NSstr structure's obj field. A TAR may be a collection of transmutation commands in a logical order which may have been knowledgeably sequenced to process the data in NSstr in an orderly and well-behaved manner to produce a single 'folding' of the NSstr data. We may refer to this process of performing a TAR on an NSstr data

structure as a ‘ravel’ function call. Conversely, an ‘unravel’ function call may ‘unfold’ a piece of folded data within the NSstr structure using the same TAR relying on the inherent characteristics of reversible transmutations. Therefore, the reversibility of transmutations may become a central feature in Structured Data Folding with Transmutations (SDFT). The SDFT methodology may use TARs on NSstr structures to iteratively transmute the object within much like an assembly line operation on data. Since the analysis may have been done on the reversible behavior of each transmutation command in the TAR, any TAR may therefore be called upon in a reverse mode or unravel function call. This topic may be discussed in more depth as additional necessary ingredients may be presented in the following sections that may make such operations possible.

[0355] The NSbin structure **3102** may serve a particular function that may or may not be only relevant to Python v3.6. In Python v3.6, a distinction may be made in the manner in which a string of data may be stored internally. It may be stored as a ‘bytes’ string or a character string. A bytes string datatype may indicate that the information held within the variable may be a series of binary bytes data. A character string may indicate that the information held within the variable may be a series of bits representing characters encoded in some type of encoding scheme. Python v3.6 may employ a sophisticated internal management scheme to best determine how to store a particular character string since different encodings may require different storage requirements per ‘character’. An example may be that UTF-8 may use 8 bit long code units to represent each character whereas UTF-16 may use 16 bit long code units to represent each character; these variations may be necessary to convey different international character sets where the number of characters in a language may be quite different from the English alphabet and therefore may not fit into the permutations of 8 bits of data. The preferred internal serialization method of transmutations, TARs and SDFT may be JSON and JSON may not have native support to map Python ‘bytes’ datatype to one of its own. If a conversion is attempted, the JSON function call may fail abruptly with some indication that the particular datatype may not be supported. An NSbin structure may be specifically designed for this type of situation and may be substituted for any ‘bytes’ data strings and therefore may make the Python variable JSON compatible. A ‘bytes’ string may be encoded into a base64 character string and stored within the ‘b64’ field of an NSbin structure. The bytes string variable may now be made to point to this NSbin structure, overwriting the original bytes data. These may represent equivalent data but they may be in different encodings and structures. However, an end result may be that the NSbin structure may be entirely JSON compatible and may now be safely serialized using JSON functions without errors due to incompatible datatypes.

[0356] In the TOP approach, this ‘bytes’ data to NSbin structure conversion and substitution may be referred to as a ‘press’ transmutation from FIGS. **12** and **33**. In Python v3.6, a press transmutation as listed in table **3302** may take any valid Python structure or variable and iteratively transmute every bytes string to an equivalent NSbin structure which may result in a Python structure devoid of any bytes datatypes. A person having ordinary skill in the art may customize an appropriate press transmutation for a language other than Python v3.6 and its JSON function call to remove such sources of data serialization errors. The reverse mode of ‘press’ may be referred to as ‘depress’ and may undo the conversion and substitution iteratively so that the data structure including its original datatypes may be restored.

[0357] The NSjson structure **3104** may serve a peculiarly useful function of only holding data that may be entirely JSON compatible. A quick glance at the fields defined for NSstr **3108** may alert one to a potential issue if the structure was directly submitted for JSON serialization due to its digest field potentially holding a digest value of the source obj in a binary string form or a bytes data string in Python v3.6. We refer back to FIG. **12** and reintroduce the ‘mobius’ transmutation for this particular issue. Note that any reasonable definition of the mobius transmutation prior to this point in this description may not be made entirely clearly to the reader due to the intertwining nature of transmutations and the TOP approach. The mobius transmutation in FIG. **32** may

transmute a given structure from one form to another in a circular fashion but with a slight twist as in a mobius strip. The mobius transmutation may be an important enabler of Structured Data Folding with Transmutations by systematically converting a NSstr structure to a JSON serializable structure such as NSjson; the process of conversion may embed the operating TAR for the NSstr in its entirety along with the transmuted data thereby imbuing the resulting storable object a self-describing characteristic. The mobius transmutation may be an embodiment that performs the essence of structured data folding in the SDFT library in a convenient way. A developer may opt to perform SDF manually using a logical combination of transmutation commands excluding the mobius command, but the mobius command adds at least one extra logical step that may require a developer to perform that step outside of the SDFT library: the ability to serialize the NSstr data structure that it is operating on and from into another structure such as NSjson. A mobius transmutation may be the last transmutation command in a TAR. Because of its functionality, this may be the only logical place where the mobius transmutation may be placed. When a mobius transmutation is processed, it may take the NSstr structure that it may be operating from and on, and transmute it to a NSjson structure. The TAR embedded in the NSstr structure may no longer exist in an useful or accessible form therefore the mobius transmutation may be the last transmutation command of a given TAR to be processed. Simply, the mobius transmutation may press the NSstr structure, JSON serialize it, then store it in an NSjson structure which may be stored, transmitted, JSON serialized, folded, or any other valid data operation that may be performed on such structures. There may be a reverse mode to a mobius transmutation but another way to view this transmutation may to state that it is a circular transmutation: regardless of a forward or reverse mode, it performs a specific data transformation depending on the input data structure type. The table **3204** indicates an NSx structure of which NSjson may be a variant. If the need arises in the future for additional transmutation structures other than those defined in FIG. **31**, and they may need to be accommodated into a mobius transmutation, this table illustrates how the mobius transmutation may behave for any transmutation structure other than NSstr. It may not be entirely obvious without actually programming with SDFT, but the mobius transmutation may logically imply that there may be no TAR processing possible from a recovered NSjson structure unless a mobius transmutation may be operated on it to convert it to its original NSstr structure which may hold the TAR that may have folded it. To initiate this mobius spin cycle with a NSjson structure, a mobius (reversal) may be kickstarted with a mobius function call from the SDFT library to produce an NSstr structure, access the embedded TAR and process the embedded TAR in reverse. This may further imply that the mobius transmutation command in the TAR, which by definition will be the first command to be processed in the reverse mode, may be safely ignored during processing since it may have been already performed by the kickstarting function call thereby it may not perform the mobius functionality more than once during such reversals. In this sequencing, failure to ignore the mobius transmutation in reverse mode may potentially produce an infinite oscillation of mobius calls which continuously convert NSstr to NSjson and back. It may seem a circuitous way of expressing such operations but it may produce fairly compact bidirectional TARs which may be systematically embedded in the output transmuted data thereby imbuing a self-describing characteristic to the folded data. This characteristic may be novel in that it may be acted upon much like interpreted scripts but both in forward or reverse modes to perform operations on the data in a consistent reproducible way across any language and/or operating systems which may support an implementation of an SDFT library.

[0358] FIG. **33** shows a table of command specifications for press, clean and key transmutations **3302**, **3304** and a set of sample transmutation commands showing its usage **3310**. A clean transmutation may be a housekeeping function that deletes transitory or temporary fields from within the NSstr structure. Certain transmutations may have a need for additional temporary fields during processing and may create additional fields within the NSstr structure to store and access them. The creation and use of such transitory fields within the NSstr may be done in a thoughtful

manner after analyzing its coexistence within the TOP approach and minimizing its interference with the proper functioning of any other transmutations. There may be no reversal mode for a clean transmutation due to its functionality therefore it may be safely ignored. This reversal implication may be taken into consideration when proposing a new transitory field within the NSstr structure, the field may not exist in a reversal mode processing of the transmutation therefore no transmutation in reverse may depend on its existence for it to function properly. An important function of the clean transmutation may be to delete an internal copy of the full keystack used in the processing of the TAR; or, it may delete only the secret keys within the keystack and convert the KISS structures to keyholes. This may be one the most critical transmutations in the SDFT TAR processing because the failure to properly clean the NSstr prior to preparing it for storage may result in the storage of any and all cryptographic keys which may have been used in the particular TAR processing and it may be inadvertently stored in cleartext along with the folded data. This situation may reveal the secret keys and compromise some or all the ciphered data within the folded data; this may not be the intended purpose of ciphering data.

[0359] In table **3304**, a key transmutation is shown with some of its operations. This transmutation may be part of the key management functionality of SDFT and may operate primarily on the keystack field by referencing the tar field of an NSstr structure. A key check transmutation may examine the stored TAR and may generate a list of key templates. If a keystack is input, it may be compared against such key templates to determine if the correct key types in the proper sequence have been provided in the input keystack. For example, if a TAR requires two different 256 bit symmetric keys for two key transmutations which may require keys, it may generate two key templates of 'symmetric 256' in a list signifying that the TAR expects the keystack to contain such keys if it may be present. Table **3504** lists some of the various key types. An empty keystack or partially filled input keystack may also be properly processed. When no keystack may be input where a TAR requires some keys, then it may indicate a 'key generate' transmutation. The SDFT may engage in a key generate mode whereby the proper types of keys according to the derived key templates may be created and composed into a keystack for submission into the operating NSstr structure prior to TAR processing on the data stored in the obj field. A partial 'key generate' mode may be engaged when a partially filled keystack may be input. The key check and generate transmutations may cooperatively determine whether the partially supplied keys in the keystack may be of the proper type and in the proper sequence. Then it may proceed to generate the proper keys for the missing keys. This process may be referred to as the 'missing teeth' scenario of SDFT keystack management. There may be very few if any examples of a TAR with key transmutation commands because it may be considered so fundamental to the proper operation of the SDFT library on a NSstr structure utilizing a TAR that it may be implicitly performed by default in every call to ravel/unravel operations rather than make the programmer place it in every TAR. It may turn out that just by having the possibility of processing a TAR which may require a cryptographic key may be sufficient cause to implicitly do the check for proper keystack management consistently, implicitly and/or automatically. The TAR reversal process may process the keystack in an appropriately reverse order. Complications may arise due to the peculiarities of the derive transmutation in keystack mode which will be discussed in a later section on how the SDFT handles such situations referred to as TAR groupings for dependent transmutations.

[0360] FIG. **34** shows a table for the Key Interchange Specification Structure or KISS. This structure may have at least two modes of operation: key or keyhole. The attributes of a key may be specified by some or all of the fields defined in the table and additional fields may be added to extend the structure to support other key attributes as needed. The TOP approach to cryptographic operations may be to assert the view of each cryptographic transmutation to require a matching keyhole specifying the exact type of key required for that transmutation. The attributes may include but are not limited to a practically unique ID for the key itself, a question or hint for a passphrase or password, a description of the key, etc. If the key value may be present in the KISS structure, it

may be referred to as just a key. If the key value may be missing in the KISS structure, it may be referred to as a keyhole. This may be indicated by the value in the 'ima' field. The field name may be a contraction of "I'm a" key/keyhole and may be read that way for clarity. The column titled 'In' may indicate the required values for creating a blank KISS structure and inserting a key into it for the purposes of placing it into the input keystack of an NSstr structure. The column titled 'Gen' may indicate those fields which may be created and filled automatically during a key generate transmutation from within the SDFT library. Throughout the SDFT discussion involving TARs, all key references may be synonymous with KISS structures of the appropriate type. It may be apparent that the keystack may closely correspond to the characteristics of the TAR being processed and that this method of stacking transmutation commands and stacking the necessary cryptographic keys in a specific form and sequence may allow for any input data to be iteratively processed through an infinite number of transmutation variations, transmutations' parametric variances and/or successive data foldings. At this point of the description of TOP, one may begin to understand the intertwined nature of the various components of SDFT and that a full appreciation of any particular part may not be revealed in an entirely linear manner.

[0361] FIG. **35** shows a table for KISS modes of operation **3502**, a matrix showing key types/field generation mappings **3504** and key type definitions **3506**. Table **3506** lists several types of keys recognized by SDFT but it may not be limited to these as new key types may be added and integrated as needed. At least three key types may need some explanation since these may be structured specifically for the SDFT library using well-known base key types. The key type 'symmetriclist' may be an array or list of symmetric keys and may be stored as the key value within a single KISS structure. This key type may support, but are not limited to, such transmutations as lock and derive. The secret sharing lock transmutations called sslock and sslock_b may respectively represent two different implementations of Shamir's Secret Sharing algorithm. The lock sslock transmutation may expect secret shares in a specific format comprising an internal index number and the key share in a 256 bit long key. This may be referred to within the SDFT library as a 'tines256' key type. The lock sslock_b transmutation may expect secret shares in a specific format comprising an internal index number and the key share in a 256 bit long key. This may be referred to within the SDFT library as a 'tinesidx256' key type.

[0362] Table **3502** is a matrix showing what characteristics may apply to a KISS structure in the two modes it can exist: key (or transmutation) or keyhole. In transmutation (key) mode, a KISS structure may be expected to store the actual cryptographic key to produce some version of ciphertext which may include keyed digests and/or digns. Therefore, its storage may be used informationally but needs to be embedded further using cryptographic functions to store it persistently in a secure manner. In keyhole mode, a KISS structure may be expected to have enough details to accept an appropriate cryptographic key as its value to produce some version of ciphertext which may include keyed digests, digns and/or derived keys. Therefore, its storage may be mandatory and may not need to be further secured by any embedding methodology since it may not contain a key value as a keyhole.

[0363] Table **3504** is a matrix showing which fields may be mandatory, relevant, input and/or generated by key type. Upon examining the table, it may be apparent that a KISS structure may hold salts pertaining to various cryptographic operations. This may seem redundant in light of the discussion on scipher embedded headers but that discussion of salts may not present the entire picture on salts. As shown in FIG. **37**, the persistence of attributes **3704**, **3714** associated with a transmutation may be dispersed among several data storage areas **3732**, **3734**, **3736** and **3738**. The TOP approach may have shown that salts may be embedded in certain cryptographic operations along with the resultant output data since it may reveal no additional information about the ciphertext produced. However, when we examine key derivation transmutations processed in a keystack mode, we may find that it may be convenient and logical to store the associated salt value in the KISS structure. A typical method of use of a key derivation function may be to accept a

passphrase as input, combine it with some salt value and produce an appropriately formed cryptographic key such as but not limited to a symmetric key. The usage of the salt in this case may be for semantic security. Therefore, it may be altogether possible that every keyhole that may accept the same passphrase may have a different salt in order that the resultant secret cryptographic key maybe different from each other for whatever rational reason there may be. This derived key may be used in a temporary fashion and discarded after use thereby only leaving the keyhole as evidence of its existence. Since the product of the key derivation may not typically be saved permanently since it may be used as a secret key, it may beg the question, where may we store it? TOP may store it in the corresponding keyhole and may prefer that the SDFT store this keyhole along with the folded data thereby each keyhole that may accept the same passphrase may have the storage appropriated for its own instance of a salt value. The programmer may store the KISS keyholes in an external manner in entirely different way. The simplified transmutation diagram on the top of FIG. 37 which is the same as in FIG. 5, becomes more like the diagram on the bottom of FIG. 37 when the various components of TOP and SDFT may be introduced. Table 3720 summarizes the placement of the attributes.

[0364] Much has been described previously concerning the syntax and variety of transmutation commands analyzed and available via TOP and SDFT, but what does a TAR actually look like in practice? FIG. 36 shows the structure of a TAR and lists several examples of TARs. Section 3602 specifies the general structure of a Transmutation Audit Record or TAR. A 'tar label01' declaration indicates the name or label of the TAR being defined just below it. All TAR commands follow the TAR label declaration and a blank line indicates the end of the current TAR definition. Therefore, many TARs may be declared in a single text file. The TAR definition section may include TAR labels on a line by itself or a transmutation command. This may be similar to a programming language compiler's macro features; it may be used as a convenience feature to combine well-known TAR constructs into a new TAR without having to actually copy the definition into the TAR itself. Transmutation commands may be inserted in a specific sequence to process the target NSstr structure in the desired way. TAR 'test_a01' may just press the Python data object into an equivalent structure devoid of any Python bytes datatypes; for other languages, it may or may not perform the same functions since 'press' may be language and/or environment specific. TAR 'test_a02' performs a press transmutation twice in succession. The second press transmutation may accomplish no functional changes to the data. This shows the TAR expansion at work. TAR 'test_a07' may press the data, serialize it into a JSON string, then convert it into a bytes type binary string using utf_32 encoding. TAR 'test_a17' shows what a terminating mobius transmutation may look like. TAR 'test_a20' presses the data, serializes it into a JSON string, converts it into a utf_8 encoded binary string, ciphers it using chacha20 with a 256 bit symmetric key and then converts the resulting binary ciphertext string into a base64 encoded character string. The symmetric key for the scipher transmutation may be expected in the keystack of the NSstr that may contain a single KISS structure holding a 256 bit symmetric key value. An alternative may be that no keystack may be provided and the ravel function proceeds to generate a valid keystack with a properly generated random 256 bit symmetric key, uses it to perform the scipher transmutation and allows the programmer to fetch a copy of the keystack (thus the key within) upon completion. TAR 'test_a42' shows an example of TAR groups and dependent transmutions: it will press the data, serialize into a JSON string, convert it to a binary string encoded in utf_8, derive a 256 bit symmetric key from a passphrase supplied in the keystack, then perform a chacha20 encryption on the data using the derived symmetric key. The last two transmutions may have a permanent dependency because the cipher relies on the derived key; therefore, this dependency may be grouped within the TAR with leading <tags> marked as such. In a forward mode, there may be no apparent influence of TAR groupings within a TAR definition except to highlight such dependencies in a visual manner. However, TAR groups may play a significant role when it comes to TAR reversals. When a TAR is being prepared for a TAR reversal process, TAR groups may be kept intact as a unit and its

constituents may not be reversed. FIG. 41 and FIG. 42 illustrate several examples of TAR reversals. The TAR 'test_a64' may perform five scipher transmutations and a DSS dign transmutation. This TAR may expect a keystack filled with six keys of various types and lengths in a particular order. Illustrated in section 3610 may be a simplified representation of the key template that may correspond to TAR 'test_a64'. This key template may be used by the implicit key check and/or generate transmutations to validate any input keystacks and/or generate a valid keystack for proper processing of the TAR.

[0365] FIG. 38 shows block diagrams of SDFT operations ravel and unravel (or reversal of ravel). Two central operations in SDFT may be 'ravel' and its inverse, 'unravel'. The ravel operation may process a given NSstr which may comprise some or all of the following items: data, TAR, keystack and/or other attributes. The ravel operation may 'ravel' or fold 3810 the source data within 3802 according to the sequence of transmutations listed in the TAR within 3802 and may eventually produce the output as a component within an NSstr structure 3804 or an NSjson structure 3806. The unravel operation may 'unravel' or unfold 3820 the source NSstr 3804 or NSjson 3806 structure according to the reversed sequence of transmutations listed in the embedded TAR and may eventually produce the output as an NSstr structure 3802. As will be shown, the symmetry of the ravel/unravel may be an interesting aspect of this design. Note the consistency of terminology and perspectives which may be used throughout TOP. A ravel operation in reverse may be equivalent to an unravel. This reversibility principle may not only simplify the analysis of such functions but it may percolate modular organizing methods which may lead to higher order concepts pertaining to the transmutation of data.

[0366] FIG. 39 shows a flowchart of a SDFT ravel operation. Given an NSx structure, the ravel function or method call may perform the following operations on the data contained within utilizing either parameters provided with the call and/or a TAR embedded within the NSx where in this case 'x' stands for any transmutation structure. Similar to the key check/generate transmutations, a mobius transmutation may be considered so fundamental to this algorithm that it may be implicitly performed on any input data structure if the conditions are met 3906. Ravel may only properly perform its core operations on an NSstr structure therefore if an NSx structure may be passed that is not an NSstr, it may attempt a conversion to an NSstr structure 3918. The failure to produce a valid NSstr 3924 may raise an appropriate error code 3978 and abruptly terminate the process 3984. There may be at least three different methods by which the data within may be raveled or folded: first, within a valid NSstr, there may be a TAR contained indicating the transmutation sequences to perform on the data within the NSstr structure; second, the name of a TAR label may be passed into the ravel call as a parameter thereby indicating the preferred set of transmutations to perform on the data within the NSstr structure; third, a customized TAR list may be passed in as a parameter along with its given name in the ravel call thereby indicating the preferred set of transmutations to perform on the data within the NSstr structure. The preparation of the TAR 3912 may comprise expanding other TAR label references and/or properly ordering it for the mode of traversal which may be either forward or reverse. FIG. 41 and FIG. 42 illustrate several examples of TAR reversals. Then a key check transmutation may be effectively performed on the TAR and NSstr structure. A component of a key check transmutation may be to derive a list of key templates by examining the TAR 3930. Using the TAR, input keystack (which may be empty or partially populated) and/or key templates, the process may compose the keystack for proper traversal of the TAR 3936. This may comprise generating the missing keys of the correct type, sequencing keys in the proper order and/or checking input keys for proper structure and type. Any mismatches in input key types and corresponding derived key templates may produce an error condition 3942 leading to raising an appropriate error code 3978 and abruptly terminate the process 3984. The process may now iterate over each transmutation command in the TAR in a proper sequence 3948 and perform the specified transmutation 3954 on the data contained within the NSstr. Any errors that may be encountered during a transmutation command execution 3960 may

raise an appropriate error code **3978** and abruptly terminate the process **3984**. When the end of the TAR sequence is reached **3948** with no errors, then the ravel operation may be deemed a success **3966** and the process may exit gracefully **3972**.

[0367] FIG. **40** shows a flowchart of a SDFT unravel operation. Rather than specifying the unravel process in detail, we may illustrate the symmetry of the reversibility of transmutations by comparing the flowcharts in FIG. **39** and FIG. **40**. The only difference between the two flow charts may be the TAR preparation steps **3912** and **4012**. Since every transmutation may have been analyzed and structured using TOP to perform in a well-behaved manner in a bidirectional way, the unravel process may not need to be very different from the ravel process except for how the TAR may be presented. It may be implemented as the same code but have a slight deviation when a reverse flag may be indicated and perform the proper reverse sequencing of the TAR when encountered. Such a call in Python v3.6 may take the form 'obj.ravel(. . . , reverse=True)'. The symmetry may allow the actual implemented code to be much smaller and/or may present fewer opportunities for programming errors. A conceptual benefit may be the clarity and simplicity of thought when constructing new TARs for specific purposes: the programmer may rely on a proper TAR sequence to be entirely reversible within its limitations and may not have to give much thought to that portion of the application. A benefit may be that the programmer's workload for creating a specific set of data transmutations may be effectively reduced by at least half since he no longer may need to create the reversing code of such data manipulations. The building of complex ciphering and locking mechanisms may require an immense number of data manipulations utilizing a large number of cryptographic keys. The Transmutation Organizing Principle (TOP) methods may help achieve more cohesive and unitized methods of approaching such complexity in discrete, less error-prone ways; thus it may allow for, but may not be limited to, more consistent, reliable, secure, portable, understandable, comprehensive, flexible, extensible and/or complicated code and/or data.

[0368] FIG. **43** shows a table of transmutations mapped to a key type template it may generate or require during TAR processing. Referring back to the discussion on key management, one of the main operations of key management may be to analyze the given TAR and produce a corresponding list of key type templates which may detail the type and specifications of each key that may be necessary in a successful processing of the given TAR. Table **3506** lists at least nine types of key types defined within SDFT. Table **4300** shows a mapping of each transmutation operation which may require a key and the corresponding key type or 'keytyp' it may require by the ravel/unravel process. A key template may have several attributes associated with each key type such as but not limited to key lengths or 'keylen'. For brevity and simplified illustrations sake, we may indicate a 256 bit long symmetric key as having a key template which may be represented as 'symmetric keylen=256' or 'symmetric 256' but in actual implementation may utilize any available data structure mechanisms in the programming language to store such values in an organized manner. In Python v3.6, a possible structure for a key template may be represented by an array of dictionaries where each dictionary entry in the array stores a single key template with each attribute corresponding to a dictionary key and the attribute value corresponding to the value associated with that key in the dictionary. Within SDFT, all key templates may be temporary structures and may be subject to repetitive regenerations via the key check transmutation and it may not be necessary to permanently store such key templates. In this way, SDFT may properly analyze any keys inserted into a keystack for processing prior to letting a cryptographic transmutation to outright fail due to key type/structure incompatibilities. A prevalent theme in TOP and SDFT may be the view that the obfuscation of data manipulation sequences may not be a reliable component in securing any sensitive payloads but rather may be relegated to the strength of the chosen cipher and its operating attributes and/or characteristics.

[0369] FIG. **44** shows TAR examples and the key templates generated from each. The left column in table **4402** lists a TAR example 'A'. The right column indicates the key type templates generated

for each transmutation command that may require a cryptographic key as an attribute input. In this example, TAR 'A' may require two cryptographic keys in the indicated sequence. The left column in table **4404** lists a TAR example 'B'. The right column indicates the key type templates generated for each transmutation command that requires a cryptographic key as an input. In this example, TAR 'B' may require four cryptographic keys in the indicated sequence. This process may be known as key template generation from a TAR.

[0370] FIG. **45** shows TAR examples and the key templates generated from each and the expected list of KISS structures to be input (put) or generated (gen). The list of KISSes is also referred to as the keystack. We can take the two examples from FIG. **44** and show the next step in the key management aspect of a ravel/unravel call. A keystack may be expected or generated in the form of a list of KISS structures corresponding to each key type template as shown by **4510**. When the TAR 'A' processing reaches the 'scipher salsa20 256' transmutation command, the process may expect to find an input 256 bit long symmetric key in the keystack as indicated by KISS A1. When the TAR 'A' processing reaches the 'dign dss 1024 digestlen=512' transmutation command, the process may expect to find an input 1024 bit dsa key in the keystack as indicated by KISS A2. The KISS list for TAR 'B' may be read and understood to be done in a similar manner. If no such expected key may be found in the keystack, the TAR processing may expect a generated key to be found instead. This implicit key generation may be beneficial to the programmer since the only requirement to generate any type of acceptable key for a given keyed transmutation is to be able to declare it within a TAR. There may be no additional steps needed to generate a specific key for a specific cryptographic function. Calling a ravel with an empty keystack may result in the output NSstr structure to hold a fully compliant keystack with appropriately generated keys to match the TAR and be able to fold the data within. It is strongly recommended and advisable that this keystack composed of KISS structures may then be stored separately in a secure manner away from the folded data and/or it may be further modified in some way and be folded again and secured using a TAR with a cryptographic transmutation thus encrypting it further. The repetitive encryption and encapsulation of keystacks may be useful when dealing with many cryptographic keys to manage and secure. TAR 'B' may produce a keystack of four KISSes and it may be convenient to securely store the entire keystack into a key repository; however, the programmer may want to encrypt the keystack of four keys using a single key for convenience. This may be accomplished by creating a new NSstr, inserting the four key keystack into the data obj field, picking an appropriate cryptographic TAR and performing a ravel call on the NSstr structure. This series of steps may produce a keystack with a single KISS structure containing the locking key to the folded NSstr structure holding the four key keystack.

[0371] FIG. **46** show the three modes of keystack operation within SDFT TAR processing: generate (gen), input (put) and injection (mixed). Section **4600** illustrates what may occur when a keystack **4602** is empty in the processing of TAR example 'B'. The ravel process may take the key type template for TAR 'B' **4508** and generate **4606** the appropriate number of randomly generated cryptographic keys of the same type and in the same order as found in the key type template as shown in **4604**. Section **4610** illustrates what may occur when a keystack **4612** is input (put) **4616** into the processing of TAR example 'B'. The ravel process may take the key type template for TAR 'B' **4508** and check it against the provided keystack **4612** to validate the number, type and ordering of keys, and then it may allow its use during the processing of TAR 'B' as shown in **4614**. Section **4620** illustrates what may occur when a keystack **4622** is presented into the processing of TAR example 'B' with only one key provided, KISS B3 or also referred to as a partially filled keystack or the 'missing teeth' scenario. The ravel process may take the key type template for TAR 'B' **4508** and check it against the provided keystack **4622** to validate the number, type and ordering of keys. During the iterative validation of each key type template vs. keystack entry, any empty KISS structure may be deemed to be a special type of validation failure and may be further construed as an implicit key generate transmutation for that key type template. The ravel process may then inject

4626 a newly generated key of the appropriate type into the empty position of the keystack and continue in the key validation iteration. Upon completing this step, a mixed keystack (may be referred to as a mix of input and generated keys, the missing teeth scenario, or key injection) may be presented and used during the processing of TAR 'B' as shown in **4624**.

[0372] FIG. **47** shows an illustration of how keystacks may be generated and used in the life cycle of data and its TAR. The use of SDFT on data **4700** may allow it to be iteratively transmuted in an orderly fashion according to a variable set of transmutations as defined by a specific TAR **4702**. The TAR may be structured in such a way as to allow for cryptographic key type analysis and thus produce key templates detailing the number and type of keys needed by the TAR. The key template may then be referenced in the composition **4704** of the input keystack whether all, some or no necessary keys may be present. When a required cryptographic key may be missing, the composition process may generate a new key for use. The TAR, data and keystack may then be passed into a ravel call **4706** to perform a folding of the structured data according to the TAR. The folded data may then be stored **4708** by any means. The keys in the keystack may be stored **4710** in a separate secure location. When the folded data needs to be referenced, the application may retrieve it from its storage place **4712**, retrieve the keys or keystack from its secure storage **4714**, pass the folded data and keystack into an unravel call **4716**, and access the data in its original form from the unravel output structure **4702**. This may be indicative of one complete cycle of Structured Data Folding with Transmutations. There may be many other pathways for any data structure to be transmuted and folded, but in essence some form of this cycle may be necessary to complete in order to fully retrieve the original data within SDFT.

[0373] The storage of the keys and/or keystack **4710** may involve a folding of the keystack utilizing a cryptographic TAR in order to protect it with fewer keys, just one key and/or different keys. The folded keystack data may become part of another structure which may eventually be folded itself. Data may be folded iteratively in a cascading manner to build internal data structures where precise piecemeal folding may lead to precise piecemeal encryptions. This ability to direct complex cryptographic data transmutations in a precise, organized and/or methodical way may lead to better and/or simpler designs for the protection of sensitive data using more sophisticated transmutation schemes. The simplicity and clarity of TAR syntax may lead to better understanding of the operations being done to the target data by others.

[0374] An important benefit of SDFT may be the systematic handling of key management within the context of combining various cryptographic operations on a given piece of data as in **4704** and **4714**. The programmer may be somewhat relieved of the minutiae of generating each key and manually manipulating its storage and/or sequencing during such processes. In the application of cryptographic functions, these minutiae may quickly add up to become a massive number of small details or attributes that the application (thus the programmer) must track, analyze, store and/or use. The SDFT methods may allow a given application to track, analyze, store and/or use fewer individual attributes of cryptographic functions because it may allow those attributes to be embedded within the context of the data and/or keystack it has operated on and produced as output, thereby it may provide a pairwise coupling of the folded data along with the transmutations which may have folded it. The transplanting of data manipulation instructions from the application to the data may allow for simpler applications and/or applications with more sophisticated uses of cryptographic functions. SDFT may enable a better alternative to express Structured Cryptographic Programming (SCP) methods as will be discussed in the NUTS section.

[0375] FIG. **48** shows an illustration of operations which may occur on data stored in an NSstr structure. Any data referenced by a pointer and/or stored in a variable **4802** may be encapsulated **4812** into an NSstr structure directly, using an instantiation method and/or using a method/function call **4804**. Then the NSstr structure **4810** may encapsulate a TAR **4814** and its associated attributes if necessary **4816**. Attributes may comprise keystacks, digests, transmutation parameters and/or temporary variables. This may provide the minimum complete set of information necessary to

process the NSstr through an SDFT ravel/unravel operation to perform the TAR on the data contained within using the attributes that may have been provided **4810**. In TOP parlance, we may refer to this as a folding of the data. The output of SDFT may be returned as the same NSstr structure **4810** or an NSx structure such as NSjson. This output may then be stored **4820** in some persistent and accessible way, transmitted to another computing device using an Inter Process Communication (IPC) method **4840**, and/or stored into another internal data structure **4830**. The cycle may begin anew for the stored data **4820** and **4830** at a later point in the application. For transmitted data **4840**, the cycle may be initiated by the reception of such data packets **4800**.

[0376] FIG. **49** shows a flow diagram of SDFT usage to iteratively fold data. The series of simplified diagrams shows the systematic folding of data using SDFT ravel calls for N successive data foldings. An NSstr structure **4900** containing at least data **4902** and TAR **4904** may be folded by calling ravel **4906** to produce output data **4908** which may be modified and/or further encapsulated into an NSstr structure **4920** containing at least data **4922** and TAR **4924** which may be folded by calling ravel **4926** to produce output data **4928** which may be modified and/or further encapsulated into an NSstr structure **4940** containing at least data **4942** and TAR **4944** which may be folded by calling ravel **4946** to produce output data **4948** which may be modified and/or further encapsulated . . . this process may be iterated as necessary **4950**. Note that in this complex series of structured data foldings, any TAR in any step may be modified separately from the application code by simply modifying the TAR instructions stored in some text file or its equivalent. An equivalent programmatic expression without SDFT of such iterative encapsulations with the possibility of transmutation sequence and/or parametric variances for each step may be comparatively long, error prone and/or difficult to comprehend.

[0377] FIG. **50** shows a flow diagram of SDFT usage to iteratively unfold data. The series of simplified diagrams shows the systematic unfolding of data using SDFT unravel calls for N successive data unfoldings. It is the exact reverse sequence of flow of FIG. **49** and may thus be understood as such. As shown previously in FIGS. **39** and **40**, the unravel call may be identical to the ravel call except for the preparation of the TAR and the state of the data being fed into it. Note that in this complex series of structured data unfoldings, no additional reverse TARs may be necessary in order to achieve the unfoldings. All necessary TARs necessary to unravel each folded data may be found embedded within the folded construct. A closer examination of the NStar structure **3106** shows an 'expd' field defined as 'List of TAR commands-expanded form'. This may be a crucial feature of the reversibility in SDFT: the output of TAR preparation steps **3912** and **4012** may produce a full set of operable transmutation commands devoid of label references and any other external references, and may be considered a complete description of the transmutations which the transmuted data may have been subjected to. This may be viewed as a static snapshot of the TAR set for the folded data thereby assuring that a proper unfolding may be performed on the folded data regardless of any changes to the TAR definitions in an external location. It may imply that TAR definition files may grow over time with a large number of TAR definitions but the storage of the operating TAR definition may be saved by the SDFT process in such a way as to preserve its reversibility regardless of changes to such external definition files (which may not be a recommended practice). This design may promote a systematic way to address time compatibility of stored data in a better way.

[0378] FIG. **51** shows an illustration of the SDFT API/Library and the various types of TAR definition files it may have access to. A TAR definition may exist in many forms such as but not limited to text files, Nuts, encrypted files, databases, server processes, and/or in running memory. A TAR may be defined at any time by the programmer as a customized TAR definition in a ravel call and may thus be a temporary TAR. For those TAR definitions which may be stored persistently, the diagram **5100** may illustrate the various forms of these but may not be limited by these shown. Standard TARs **5102** may be TAR definitions which may be provided as a package along with the SDFT library installation for the any OS/language pairing. Hidden TARs **5104** may be TAR

definitions which may be customized TAR definitions that may only exist in access restricted locations and/or accessed by expressed permission. These may be the preferred method of TAR definitions within a private network or custom application installation. The use of Hidden TARs may be kept hidden even within the output of a ravel and no expanded form of the TAR may be found embedded in such folded data but just a reference to it by TAR label. The obligation to maintain the Hidden TARs may reside on the administrators of such groups since data folded with Hidden TARs may not necessarily contain the transmutation set needed to unfold it. Hidden TARs may seem familiar as the equivalent method of obfuscating the data manipulation sequences within a program. Local User TARs **5106** may be TAR definitions which may be customized TAR definitions that may only be accessed under the user's or programmer's account privileges. These may be temporary or developmental TARs that a programmer may be formulating for permanent addition to one of the TAR definition storage forms at a later time. Remote TARs **5108** may be TAR definitions which may be accessed with or without permission access from a remote server or storage site. Such topology may be necessary due to limited local storage or due to a policy of centralizing key TAR definitions into a centrally managed area. This may also be a method of constantly checking to see if the Standard TAR definitions may be the most recent versions. Protected TARs **5110** may be TAR definitions which may be located in any appropriate, accessible place but may be encrypted for authorized access only. A separate authentication and/or authorization process may need to be traversed successfully in order to gain access to Protected TARs. Another form of a Protected TAR may be stored within a Nut container which may require a proper key(s) to gain access into it. Embedded Folded TARs **5112** may be the expanded TAR definitions preserved along with the folded data from a ravel call.

[0379] FIG. **52** shows an example Python script to perform manual data folding. FIG. **53** shows an SDFT example of a TAR definition and its usage in a Python script. FIGS. **52** and **53** may together show an example of how SDFT differs from a more straightforward programmatic approach using Python v3.6. These example Python scripts may illustrate the major differences in the basic calling sequences for each task at hand using each methodology. We may start with a sample data set **5210**. The operations to perform on the data may be specified in tasks **5220** expressed in plain language as shown in lines 02-06. Usually these may be entered into the program itself as comment lines for readability. Section **5250** shows the actual Python code to perform the tasks and section **5260** shows the reverse processing of the tasks to recover the original data **5210**.

[0380] Using SDFT, the data set **5310** is the same as **5210**. Section **5320** expresses the tasks **5220** as a TAR definition labeled 'test a70'. Section **5350** ravel the data and writes the folded data to a file. Section **5360** reads the folded data from a file and unravels it.

[0381] There are 18 lines of Python code for FIG. **52** and only 8 lines of code in FIG. **53**. It may be apparent that any changes in the types and number of data transmutations may affect both sections **5250** and **5260**. The method in FIG. **52** requires the programmer to maintain several variables, the sequence of tasks and/or the proper calling of each function or method. The reverse process in **5260** requires the programmer to make sure all operations are called in the correct reverse order and the parameters fed in the correct way for each function or method call. Any changes to the tasks in **5220** may result in programming changes to sections **5250** and **5260**. Any additional tasks in **5220** may result in additional program lines to sections **5250** and **5260**. More temporary variables may be created and used as necessary for these additions or changes to the tasks.

[0382] In the SDFT method in FIG. **53**, any changes in tasks may be directly reflected in the TAR **5320**. Therefore any additional transmutation modifications may only vary the length of this section. The ravel and unravel calling lines 10 and 14 stay unchanged. The reversal process in **5360** of TAR **5320** need not be specified beyond the original TAR definition in **5320**. In fact, sections **5350** and **5360** may stay undisturbed for any TAR definition chosen except for line 10 where the TAR definition label is specified in the ravel method call.

[0383] In terms of readability and comprehensibility of the tasks being performed, the reader may

prefer the TAR **5320** over the actual program code in sections **5250** and **5260**. The tasks specified in **5220** are not code and may usually be expressed as comments within the Python code. Any changes to the program code in sections **5250** and **5260** must be manually coordinated with the comments by the programmer otherwise confusion may ensue if another programmer was to attempt to understand the code with inaccurate comments and vice versa. A TAR **5320** may be considered self-describing in a clear and compact way.

[0384] The data stored by lines 15-16 in section **5250** has no embedded metadata describing how it may have been transmuted. The transmutation methodology is hardwired in sections **5250** and **5260** as actual code. Any such data written in this manner may be completely dependent on the existence of the same or similar code for its proper retrieval and recovery. These code sections or its equivalents must be maintained for all time for the data it transmuted to be recoverable for all time. It may be the equivalent of a Hidden TAR method.

[0385] The data stored by line 11 in section **5350** may contain an embedded, expanded TAR definition which may have transmuted the folded data. The transmutation methodology may be paired with the folded data thereby making it transportable. The recoverability of the folded data may be considered independent of the code that created it **5350** and **5360**. Any code that may properly process the embedded TAR definition in the folded data may recover the original data. This type of functionality may allow for better time compatibility for changing transmutation sequences over time as older folded data may self-describe and thus self-prescribe how it may be recovered.

[0386] FIG. **54** shows block diagrams of dynamic TAR switching within a single communication session. In the TOP approach, a higher level communication protocol may be viewed as the passing of transmuted data from one computing process to another. Since transmutations may allow many of the most frequently used cryptographic functions, it may be used to create secure messages for IPC. Theoretically, each message may be transmuted and folded using a different TAR. Each different TAR definition may be considered as its own protocol by modern standards of protocol definitions. Using SDFT, TARs can be switched dynamically on a per folded message basis between two applications as illustrated in FIG. **54**. Any mixture of TAR sources as shown and described in FIG. **51** may be used as long as each application may have access to those TAR definition sources. The rich set of embedded, folded metadata such as but not limited to KISS structures as keyholes specifying an exact key identifier for each key needed in an embedded cryptographic transmutation may allow SDFT based communication protocols to offer security on a more sophisticated and potentially more secure level.

[0387] TOP analysis and methods which may result in a framework called SDFT may allow stored data to contain its own portable instruction set which may have produced it. This framework may define a data folding and may provide methodologies and/or embodiments to fold data using a conceptually and logically consistent reversible transmutation processing method expressible as a Transmutation Audit Record (TAR) which may be embedded within the stored data in an organized fashion. The resulting folded data may then be modified in some way and may then be repeatedly folded as needed to achieve the desired application or data form result. Short of describing TAR as a programming language, it represents a set of cooperative data manipulations in a concise form which may allow for infinite variations of transmutation sequences and/or the infinite variations of transmutation attributes within a given TAR and/or attributes. SDFT may allow for variable scoping for datasets similar to the way programming languages isolate local variables using scoping concepts and techniques. Through TOP, protocol variances may be viewed in a higher conceptual construct which may lead to data that may be self-describing and possibly may be accessible and readable from a wide variety of applications that may access its methodologies via an available SDFT library adapted for their programming environment. Furthermore, these characteristics which may be imbued into folded data may allow for the dynamic switching of protocols within a single communication session or single stored data object. The TOP approach

may be utilized as a fundamental building block for the NUTS ecosystem and in the composition of a Nut. NUTS may be fully implemented independent of SDFT but that may be inadvisable.

Nut ID

[0388] The NUTS design may enable the identifiability of data regardless of location. This may require a universally unique ID (UUID) but it may not be achievable in a guaranteed manner without some form of centralization, therefore we may settle on the notion of a practically unique ID with sufficient length and entropic properties to provide a low probability of ID collisions. FIG. 55 shows a flowchart of an example of a process for generating a Nut ID 5500. Here a local device 5502 may be running an application which may invoke a function to generate a practically unique ID from data pieces such as but not limited to user attributes 5504, environment attributes 5506 and/or random attributes 5508. User attributes 5504 may include data items such as but not limited to user login information, group ID, company ID, user ID and/or user password hash. Environment attributes 5506 may include data items such as but not limited to MAC address, IP address, device information, system time, OS information, directory paths and/or files, atomic clock synchronized time values, GPS synchronized time values, declared environment variables, thread ID, CPU runtime, IMEI number, phone number, application name and/or process ID. Random attributes 5508 may include data items such as but not limited to session counters, UUID, clock cycle counts, randomly generated numbers, mouse movement, keyboard activity, file system state, partial or complete screen area hashes, process up-time, OS up-time, and/or session duration. These data pieces may be gathered and stored in an ID structure 5510 which may then be serialized using JSON or alternative marshalling techniques. Then the resultant binary string may be hashed 5520 using a hashing algorithm such as SHA-512 (from the SHA-2 family of hash algorithms published in FIPS PUB 180-2 by NIST in 2001) or alternative hashing method which may produce practical uniqueness with a suggested minimum length of 512 bits to lower the probability of ID collisions. The binary hash may be encoded into a base64 (or alternative encoding scheme) text string for portability and readability 5514 which may produce a text string 86 characters long more or less. The encoding scheme may comprise any method that may result in a printable and human readable form and may be accepted by the plurality of programming languages and software systems as a text string. Depending upon the modality in which the function may have been called, the resulting encoded hash string may be checked for duplication against any accessible Nut ID cache 5516. If there may be a collision of ID values then the process may be repeated with new random attributes 5508 until a non-colliding ID may be generated; collisions may be expected to be rare occurrences. The output string of this logical operation may be called a Nut ID 5518.

[0389] This process may be called locally within the running program or may be implemented within a server application residing locally or remotely serving client application requests for new Nut IDs. A possible benefit of a server model implementation may be its ability to access larger caches of existing Nut IDs to check against and may produce a Nut ID with a lower probability of collision. Nut ID duplication checking is not mandatory since the hash length and properly gathered data components in the ID structure 5510 may provide sufficient entropy. There may be a general concept of compartmentalization throughout some or all digital infrastructures such as the Internet with IPv4/IPv6 addresses, domains, directory hierarchies and access control groups. In a similar way, a Nut ID may be practically unique but it likely might be used within the context of a compartment constructed by an external system or relationship and thus the chances of collision may be many orders of magnitude smaller than the mathematical probabilities offered by the permutations in a given length of bits of the Nut ID. In cases where a different length may be desired, it may be accomplished by substituting the SHA-512 hash with an alternative hash algorithm in a modular parameterized fashion by a person having ordinary skill in the art.

[0390] Given the process by which a practically unique ID may be generated in the form of a Nut ID, what may be identified by it? In NUTS parlance, this may be known as Nut ID stamping. There may be at least two structures within NUTS that may be consistently stamped with Nut IDs: Lock

Nodes and Nuts. A Nut ID assigned to a Lock Node may be called a Lock ID. A Nut ID assigned to a Nut may be called a Nut ID. A Lock Node may be an internal building block of a Nut. A Lock Node may be a self contained, stand alone locking mechanism which may protect its payload known as a Bag. A Nut may be a data structure composed of one or more Lock Nodes. Therefore, a Nut may hold any parcel or parcels of data in whole or part thereof. Nuts may be used throughout the NUTS environment to identify in a practically unique way some or all associated software, data and/or hardware represented in binary form. A consequence of Nut ID stamping may be that every Nut may be uniquely identified implying that every data parcel stored within a Nut may be uniquely identified by that Nut ID regardless of where the Nut may be physically located.

[0391] FIG. 56 shows a simplified schematic of a Nut data structure. This diagram may highlight the usage and relative placements of Lock IDs and Nut IDs within the Nut data structure. The specific Lock IDs **5614-5622** may be assigned in this Nut and they may be different values. The Lock Nodes **5604-5612** may be respectively identified by Lock IDs **5614-5622**. In a typical Nut data structure formation such as this example, a Nut **5602** may be a group of Lock Nodes organized into a graph like data structure called a Lock Graph. A particular Nut **5602** may be identified by its Nut ID **5634** which may be stored in the Lock Node's **5606** Bag **5626** and the Nut ID may be considered the payload of this Lock Node which may be different from the payload of the Nut which may be stored in one or more of the other Lock Node Bags. Every Lock Node **5604-5612** structure may contain a payload area called a Bag **5624-5632**. This shows the relationship between a Nut and its Nut ID and where one may find these items stored in a typical Nut container.

[0392] FIG. 57 shows examples of relationships between Nut IDs, path names and/or payload data. There may be several Lock Nodes in a Nut which may be used to store metadata of the Nut, the metadata about the Nut payload and/or the Nut payload. The metadata portions may be stored in the Bags of various Lock Nodes in the Nut. **5702** shows a scenario where there may be two distinct Nut payloads D1 and D2 each stored in distinct Nuts identified by Nut IDs A3 and B1 respectively. Two character Nut IDs are used for illustrative purposes even though it may have been specified previously that a Nut ID may be a base64 encoding of a 512 bit hash which may produce a text string up to 86 characters long. Nuts A3 and B1 also have distinct pathnames in an NTFS file system. **5704** shows two distinct Nuts having the same filename but different pathnames. **5706** shows two copies of the same Nut with same filenames in different directories. **5708** shows two copies of a Nut with different filenames sitting in the same directory. This may not be an exhaustive listing of some or all the permutations of these attributes but it may show the flexibility of having a metadata item permanently associated with each payload such as a Nut ID.

[0393] Data embedded within a Nut file which may be identified by an associated Nut ID may give rise to a novel feature of this methodology: the ability to automatically create dynamic filenames based on parameterized rules in the metadata. The filename may be representative of the normal identifying string for the file as well as a formulated summary of its other attributes such as but not limited to modification date and time and/or number of writes for the day. This may give a more accurate and convenient way of identifying a file and its state in time without having to delve into normally hidden attributes such as having to look at the file properties in a directory browsing application. It also may allow the embedding of file and data attributes into the container holding the file rather than rely on the attribute capture capabilities of a file system which may vary from one file system to another. An example: a user may create a Nut with Nut ID #234 that may store a text document, the text document may always be identified by Nut ID #234 but the user may set up a dynamic filename comprising a base name+date of last modification+count of writes for the day such as "diary_20151115_1.txt". On the same day, when he saves to disk after modifying it a bit, the filename may show "diary_20151115_2.txt" and the old filename may no longer exist in the directory. This methodology may automatically create a new filename that may indicate some state information of the stored data. The properties of the Nut ID which may be practically unique and may be separate from pathname+filename designations may allow such a feature to be

implemented without any external references. One of the benefits of such a feature may be the oft used method of copying and archiving previous states of a working document with a date stamp. An author may find a directory stuffed with a file for each day that he may have worked on his document. Using the dynamic filename method, he may only have one Nut file in his directory with the date stamp of the last time he wrote to it. The history (state) saving aspect of the manual method may be preserved within the Nut itself using the Nut History feature presented in a later section. This concept of the Nut ID being the main identification of content may be used later by the NUTserver to perform replication and synchronization operations on dispersed Nuts.

Lock Graphs & Lock Nodes

[0394] NUTS technology may address the storage, protection and access control of data in a layered, integrated, modular and/or iterative approach which may be defined as Structured Cryptographic Programming (SCP). The overall design of a Nut's internals may be described and defined and then each defined structure may be subsequently described in detail. Some features may be described in a layered fashion and then an integration description may be provided to show how the individual features may work together. SDFT may be utilized throughout the NUTS design to improve the organization of complex cryptographic structures and the systematic embedding of attributes associated with each folded data structure. It may be shown in various embodiments how SDFT enables SCP designs to be implemented with relative ease compared to the equivalent manual methods.

[0395] There may be four different methodologies that may control access of a Nut: Keyhole, Variable Lock, Stratum Access Control (SAC) and/or Nut Access Control (NAC). Some or all of these methodologies in part or whole may be layered and/or integrated together in novel ways within a Nut which may provide the full functionality of a reference monitoring system in an internalized and/or independent manner. These four layers may be embodied in a complex data structure called a Lock Node which may be designed to be modular, insular and/or linkable.

[0396] A Keyhole may be a data structure that may accept any number of cipher keys each of which may have an associated Encrypted Key Map. The embodiment is not limited to the cipher key types it may currently recognize and accept: passphrase, symmetric key and asymmetric key pair. Any simple or complex method, or any process that may specify a sequence of bits as a secret key may be integrated into a Keyhole. The Encrypted Key Map may contain several sets of keys, one set for each layer of access control within the Nut: Variable Lock, SAC and/or NAC.

[0397] A Variable Lock may provide different types of locking mechanisms in a normalized structure which may protect data in a Lock Node. These Variable Locks may comprise ORLOCK, MATLOCK, SSLOCK, XORLOCK and HASHLOCK. This disclosure is not limited to these pre-defined lock types but may be expanded or contracted to accommodate any appropriate locking scheme that may be normalized into its structure.

[0398] The Stratum Access Control may regulate penetration access into individual Lock Nodes in a Lock Graph. This feature may give rise to a property in Nuts called Gradient Opacity which may be the ability for a Nut to allow various levels of metadata to be viewed given appropriate access attributes.

[0399] NUT Access Control or NAC may employ Role Based Cryptographic Access Control (RBCAC) techniques to finely control modifications and authentications of a Nut's internals.

[0400] Structured Cryptographic Programming may be the design of data structures which may allow facile and flexible interactions between different methodologies to express a variety of access models. The security mechanisms may be entirely embodied in ciphered data and their associated ciphers, therefore, there may be no external application dependencies on the access control of the Nut such as a reference monitor. In some embodiments, a Lock Node may be used individually to protect field level data in any part of a payload. The internals of the Nut container may potentially make use of a plurality of cipher keys to embody a particular security model.

[0401] A Nut may be a directed graph data structure called a Lock Graph composed of nodes called

Lock Nodes. Each Lock Node may be identified by a Lock ID which may be created by the same function for generating the Nut ID therefore they may both have the same characteristics. The Lock Nodes may be stored in a hashed array which may be referenced by their Lock IDs. Each Lock Node may have pointers linking to other Lock IDs or a null pointer. Using well established programmatic graph extraction and traversal techniques, a Lock Graph may be derived from the hashed array of Lock Nodes. A Lock Node which does not have other Lock Nodes pointing to it may be a Keyhole Lock Node (entry or External Lock Node). A Lock Node which may have a null pointer may be a terminal Lock Node of the Lock Graph and may store the Nut's payload or a reference to the payload. A Lock Node may have multiple Lock Nodes linking to it. Under most circumstances, a Lock Node does not link back to an earlier Lock Node in the Lock Graph or itself. A circular link reference may be unusual but may be accommodated through customized programming for custom Nuts if such a structure is warranted.

[0402] Some if not all data structure described herein to support the functionalities of a Nut may be implemented using complex data structures within the chosen programming language. If an SDFT functional library is available for the chosen programming language, it may be readily applied to fold and encapsulate any and all applicable complex data structures or subparts thereof to minimize data manipulation code, clarify the data manipulation methods, reduce the probability of coding errors, and take advantage of the implied SDFT features embedded in every folded data structure.

[0403] Note that due to the data centric nature of this disclosure, most flowchart type diagrams may be a mixture of traditional flowchart elements mixed in with data components which may be referred to as data flow diagrams or data flowcharts. Also, the intertwining nature of the Lock Node design layers may make it difficult to expose the logical operations of its components in a completely linear manner without making forward referencing statements therefore some re-reading may be required on the part of the reader.

[0404] FIG. 58 is an embodiment of a Nut or Lock Graph 5800 comprising two logical sections employing the multiple purpose aspects of modular Lock Nodes: Nut Lock 5802 and Nut Parts 5804. The Nut Lock 5802 section of the Lock Graph may allow for complex cryptographically linked locks to be constructed for a given Nut using one or more Lock Nodes. There are currently five types of Lock Nodes defined in this disclosure corresponding to the five types of Variable Locks mentioned: ORLOCK, MATLOCK, SSLOCK, XORLOCK and HASHLOCK. Each Lock Node type may be referring to the type of Variable Lock internal locking mechanism that may be utilized in the heart of a particular Lock Node to protect the encryption keys to the storage area and other Lock Node metadata and parameters. Lock transmutations as disclosed in FIG. 30 may be an embodiment of Variable Locks and may be used in the building of a Lock Node. Successfully unlocking and traversing the Nut Lock 5802 portion of the Lock Graph may lead to the Nut Parts 5804 section of the Lock Graph 5800. There may be several Lock Nodes that comprise the Nut Parts 5804: hair 5820, tick 5822, seal 5824, vita 5826, bale 5828, and/or tale 5830. The Nut Parts 5804 may contain the Nut payload 5830 and/or metadata 5820-5828. The number and type of Nut Parts for a Lock Graph may vary depending on the type of data the Nut may be storing and/or the design of the Nut for some desired behaviors and characteristics. In this example, unlocking the Keyhole Lock Node 5806 (5816) may result in proper cipher keys that may be inserted into the Primary Keyhole of the linked 5818 Lock Node 5820. Unlocking Lock Node 5820 may result in proper cipher keys that may be inserted into the Primary Keyhole of the linked Lock Node 5822. Unlocking Lock Node 5822 may result in proper cipher keys that may be inserted into the Primary Keyhole of the linked Lock Node 5824. Unlocking Lock Node 5824 may result in proper cipher keys that may be inserted into the Primary Keyhole of the linked Lock Node 5826. Unlocking Lock Node 5826 may result in proper cipher keys that may be inserted into the Primary Keyhole of the linked Lock Node 5828. Unlocking Lock Node 5828 may result in proper cipher keys that may be inserted into the Primary Keyhole of the linked Lock Node 5830. Lock Node 5830 may link to a null pointer therefore it may be the terminal Lock Node or the innermost layer of this Lock Graph

or Nut. The unlocking of a Lock Node may comprise the unraveling of a folded data structure representing the Lock Node using SDFT methods (unfolding). Each Lock Node may contain a plurality of folded data structures where the action of unlocking a Lock Node may be equivalent to the unfolding of the applicable data structures.

[0405] FIG. **59** shows a simplified Nut schematic **5900** of a Lock Graph embodiment comprising logical sections Nut Lock **5902** and Nut Parts **5904**. This example explores a Nut Lock **5902** comprising four Lock Nodes **5908**, **5910**, **5912** and **5916**. Lock Nodes **5908-5912** may be the Keyhole Lock Nodes **5906** of this Nut since some or all of them may be external facing nodes and may accept external cipher keys called Primary Keys. A user may have Primary Keys associated with one or more of these Keyhole Lock Nodes. The Nut ID of the Nut storing a Primary Key as its payload may act as a key ID that may be automatically matched up with the identifier marking the keyhole it belongs to amongst the Keyhole Lock Nodes **5906**. Passphrase keys may be identified by key IDs or a text string which may or may not hold a question as its identifier. Complex multi-level passphrases may be constructed using proper keyhole identifiers and clear text Nut metadata portions with the appropriate question lists. Linkages between Lock Nodes such as **5914** and **5918** may be opened in a similar manner where the successfully unlocked Lock Node may produce an output key(s) with an identifier. In this particular example, unlocking any one of the Keyhole Lock Nodes may reveal the proper cipher keys that may be inserted into the keyhole of the linked **5914** Lock Node **5916**. From this point forth, the unlocking of the nodes comprising the Nut Parts **5904** may proceed similarly to the process described for Nut Parts **5804**. This Nut Lock **5902** construction may convey the building block nature of Lock Nodes and the flexibility of its combinations by showing that three distinct pathways may exist to unlock the payload of the Nut **5900** with each pathway requiring different conditions to be met in order to proceed with the unlocking process.

[0406] In FIG. **60**, a Lock Node **6000** may be a data structure comprising the following sections: Parameters **6002**, Input **6006**, Key Maps **6008**, Variable Lock **6012**, Derived Key **6016**, Key Set **6020**, Bag **6024** and/or Output **6026**. The Parameters section **6002** may hold the Lock Node's metadata, Lock ID **6030**, and encrypted strings of the Key Maps **6010**, Derived Key **6014**, Key Set **6018**, Bag **6022**, and digns of the said encrypted strings created by the appropriate Access Role Keys (forward reference may be described in the discussion for FIG. **83** element **8334**) for the Lock Node. The design principles may be similar to the flow in a Lock Graph with the unlocking of each section which may lead to keys that may help open the next section but then each component within the Lock Node may provide a specific function. The digns on the encrypted strings may be used by readers (an Access Role) to authenticate a particular section prior to a decryption attempt. The digns may be created by the writers (an Access Role) of the particular section using the encrypted string of the section when there may be some modifications to preserve or to indicate that a proper writer access key holder generated the dign. Furthermore, each of the above mentioned encrypted strings may be embodied by the use of SDFT methods to fold data structures using TARs containing cryptographic transmutations. Given the number and variety of encrypted strings described in this section, SDFT methods may drastically reduce the burden of managing cryptographically related attributes by the programmer when coding.

Keyholes

[0407] In FIG. **61**, the Input section **6006** of the Lock Node may provide two different key holes: Primary Keyhole **6102** and Access Keyhole **6104**. Structurally, a Primary Keyhole **6102** may accept any number of cryptographic keys comprising four different key types: symmetric, asymmetric public, asymmetric private, and passphrase. The Access Keyhole **6104** may accept symmetric and/or passphrase key types. The Primary Keyhole and the Access Keyhole may internally utilize one or more KISS data structures as shown in FIG. **34** each operating in a keyhole mode (ima='keyhole') to represent the keyhole for each unique key that it may accept.

[0408] FIG. **62** shows a single cryptographic key **6202** which may have an associated key ID, key

type and key attributes **6206** and may be also designated as a Primary Key. The key ID may be any identifying string. The Primary Key and any other keys mentioned in this disclosure may each be internally represented by a KISS data structure as shown in FIG. **34** operating in a key mode (ima='key') with the key->value field populated with the key and other matching attribute fields filled in as needed. A Primary Keyhole **6204** may accept a Primary Key **6202** that may decrypt an encrypted Key Map **6208**. The decrypted Key Map **6240** may be a structure which may comprise three sections: Main Key **6210**, Stratum Keys **6212** and Access Key Set (AKS) **6214**. The Main Key structure **6210** may contain a symmetric key or time which may be called the Main Key, expiration date/time for the Primary Key **6202**, countdown timer for the Primary Key and/or action instructions upon expiration of the Primary Key. The symmetric key or time may be used by the Lock Node's Variable Lock. For a Keyhole Lock Node, the Key Map structure may additionally hold Stratum Keys **6212** and/or AKS **6214**. The Stratum Keys **6212** may hold a set of keys to be inserted into the Lock Graph's Strata Lock Nodes, those Lock Nodes which may be identified by its Stratum designation. The AKS **6214** may hold a set of keys to be inserted into its own Access Keyhole **6104** for a Keyhole Lock Node. The encrypted Key Map **6208** may be a SDFT folded data structure which may contain the Main Key **6210**, Stratum Keys **6212** and Access Key Set (AKS) **6214** structures.

[0409] FIG. **63**. shows a flowchart of the key insertion process for any Lock Node and for any cipher key. Step **6304** may be a search across some or all the listed Lock Nodes in a Nut for a given cipher key and its associated key ID. Once a cipher key is inserted into the proper keyhole **6304**, step **6306** may attempt to decrypt and unfurl the Encrypted Key Map for that key. The decryption and unfurling of the Encrypted Key Map may be equivalent to the unraveling of an SDFT folded Encrypted Key Map for such embodiments.

[0410] Upon a successful unlocking and unfurling of an Encrypted Key Map **6208** for a Keyhole Lock Node, 1) the Stratum Keys may be inserted into each Lock Nodes' Primary Keyhole matching the stratum designation found in each Lock Node's Parameters section, 2) the Access Key Set's (AKS's) Access Attribute Key Set Unlock Keys (AAKSUK) may be inserted into the Access Keyhole of the of the Lock Node. This Primary Key unlocking (or unraveling) may occur for as many Primary Keys may have been inserted into the Lock Node after which we may have a set of decrypted (or unfolded) Key Maps collectively making up a set of Main keys for possible use by the Variable Lock of the Lock Node.

[0411] FIG. **64** shows an example where three primary keys **6402-6406** may be inserted into the primary keyhole **6400**. Each key (.fwdarw.) may be matched up with its identifying key ID and may be inserted into a slot in a hashed array or KISS keyhole structure. The Key Type may indicate a key type such as but not limited to symmetric, asymmetric public, asymmetric private and passphrase. In some embodiments of a Nut, a user may specify any type of key that may have a corresponding cipher methodology suitably modularized for NUTS integration. These key cipher methodologies may include fingerprint scans, iris scans, palm prints, voice prints, hand writing patterns, facial recognition, DNA characteristics, physical key devices, hardware secured keys, software/hardware based Zero Knowledge Protocol keys and/or NFC keys. If an asymmetric private key is inserted such as may be used in RSA-2048, it may represent both the public and private portions, the public portion may be extracted from the private portion and may be used to encrypt the primary key's Encrypted Key Map therefore the decrypt operation may require a private asymmetric key to be presented. As plainly shown for one key (.fwdarw.) inserted into one keyhole **6402**, its Encrypted Key Map **6412** may be decrypted using the key type cipher methodology to reveal the Key Map structure **6430** which may contain three distinct sets of keys **6432**, **6434** and **6436**. This decryption step may be done for each key **6404** and **6406** to produce the respective corresponding key map sets **6440** and **6450**. Each decryption step may also be equivalent to unraveling an SDFT folded structure for such embodiments. For a passphrase key type, the key may be the passphrase and the key attributes may indicate the passphrase derivation function to use

and the appropriate parameters for the function comprising the number of iterations to perform to produce the symmetric key that may decrypt the Encrypted Key Map. For embodiments utilizing SDFT, such passphrase attributes may also be matched up with a corresponding TAR to access the appropriate derive transmutations with matching attributes. To put the example into perspective with the Lock Node diagram **6000**, the Input section **6006** may contain the Primary Keyhole **6400**, the Encrypted Key Maps **6010** may be represented by **6410** and Key Maps **6008** section may be represented by **6420**.

Variable Locks

[0412] The next part of the Lock Node may be the Variable Lock as shown in element **6012** of FIG. **60**. The Variable Lock may be the locking mechanism that may help protect the contents of the Lock Node stored in the Bag **6024**. The Variable Lock may allow a Lock Node to utilize any one of several different types of cryptographic locking techniques familiar to a person having ordinary skill in the art. For example, these different lock types may comprise an ORLOCK, MATLOCK, XORLOCK, HASHLOCK and/or SSLOCK. This may be accomplished by normalizing the inputs and/or outputs of each locking method to fit into a common data flow model thereby each locking method may be replaced with one another seamlessly. Similarly, the Primary Keyhole and the Key Map structures may act as data normalizers for the number of keys and key types flowing into a Variable Lock. A Lock Node may be imprinted with a set of parameters **6002** indicating what type of Variable Lock it may be implementing **6030**. Once this value is set, a Lock Node may rarely change this setting although it may be possible to rekey and/or reset Lock Nodes by the RAT (owner of the Nut). The SDFT library describes an embodiment of Variable Locks as listed in FIG. **30** and its accompanying specification which may be used in this section for convenience but the use of the Lock Transmutation is not a necessary requirement to fulfill this functionality of a Lock Node.

[0413] Continuing the traversal of the Lock Node in FIG. **64** where we ended up with three Main Keys **6432**, **6442** and **6452**. We may explore how its Variable Lock may operate in FIG. **65**. The Variable Lock **6502** may protect a Derived Key (DK) **6506** by encrypting it as the Encrypted Derived Key (eDK) **6504**. Some or all Main Keys **6432**, **6442** and **6452** may be symmetric or tine key types and may feed into the Variable Lock **6502**. Depending on the Variable Lock type which may be specified in the Lock Node Parameters section **6002** and **6030**, the appropriate Variable Lock function may be called to perform the cipher/decipher operation on the DK or eDK. FIG. **65** shows a decryption operation of eDK **6504** into DK **6506** by the Variable Lock **6502** which may use the Main Keys **6432**, **6442** and **6452**. FIG. **66** shows an encryption operation of DK **6506** into eDK **6504** by the Variable Lock **6502** using the Main Keys **6432**, **6442** and **6452**. In an embodiment using SDFT, the DK may be data that is protected by a TAR employing a Lock Transmutation by a data folding; therefore, unfolding such a structure reveals the Derived Key contained within.

[0414] The table in FIG. **67** summarizes some of the key characteristics the Variable Locks mentioned. As the term Variable Lock may imply, any locking technique that may be normalized into this model may be added as an additional Variable Lock type. Alternatively any locking technique may be excluded. The table in FIG. **30** may correspond to the table in FIG. **67** and shows how SDFT may embody the Variable Lock designs in its Lock Transmutations.

[0415] The metadata section **6030** of the Lock Node may be a common component that may be involved in some or all Variable Locks. There may be various digns (digital signatures) of Lock Node sections which may have been created by an appropriate Access Role Key (ARK) such as **6040-6048** (forward reference). Some of all of these digns may be created by a Nut owner who may be anyone holding a Root Access Tier (RAT) Access Role Key in particular the RAT private key through its AKS. Everyone with a valid Primary Key may have a RAT public key that may enable them to authenticate various RAT digns throughout the Lock Node to make sure the Nut components may not have been compromised. In the diagrams, sometimes the RAT public key may

be referred to as the RAT Reader key and the private key may be referred to as the RAT Writer key. Later in this document, further discussions concerning the Nut Access Control layer may explore, specify and/or clarify these features in more depth. As previously mentioned in the section on SDFT and TARs, the digns of encrypted data may be part of a folded data structure's TAR specification which may embed the protected data, its dign and the TAR which created it. It plainly implies that a systematic use of SDFT within the Lock Node may be advantageous to the programmers work load.

[0416] An ORLOCK in FIG. **68**, also known as an OR lock, is a Variable Lock that may accept any number of symmetric cipher keys called Main keys **6808** and may systematically attempt to decrypt **6814** the eDK **6806** using a symmetric cryptographic cipher such as AES-256 or alternative cipher. The Parameter section **6002** may indicate the cipher method to use for this logical operation or the preferred TAR when using SDFT methods. The first successful decryption of the eDK may produce the Derived Key (DK) **6816** and may result in the successful unlocking of the ORLOCK. Prior to a decryption attempt in any Variable Lock, the dign of the eDK **6804** may be authenticated using the eDK **6806** and the RAT Reader key **6802**. If the authentication is successful **6810**, then the decryption process may continue, otherwise an error **6830** may be raised and the attempt may be halted. The Main Keys **6808** may be identical keys such as but not limited to symmetric 256-bit keys. In this arrangement, the essence of an OR lock may be isolated and normalized into Keyhole and Variable Lock structures to make it modular. In a folded structure, the authentication step may be part of the TAR and may be implicitly attempted by the act of unraveling.

[0417] FIG. **69** depicts the encryption operation of an ORLOCK from the viewpoint of a RAT Writer or Nut owner. It may take any Main Key **6902** and may perform an encryption operation **6920** on DK **6906** using an appropriate cipher to produce an eDK **6908**. Then using its RAT Writer key **6904**, eDK **6908** and an appropriate digning algorithm **6922**, it may create a dign of eDK **6910** which may be stored in the Lock Node parameters section **6044**. SDFT methods may fold many of these attributes in a compact way along with the eDK into a single data object to be stored in the Parameters section. The encryption process for non-RAT members of a Lock Node may be simple; they may either erase the application memory contents of the Lock Node since they may not create an authentic dign on anything implying they may not successfully change its contents and dign it, or they may use the already decrypted DK **6908** and they may encrypt the relevant contents of the Lock Node but may leave the eDK **6910** untouched since nothing may be changed that may be relevant to the eDK dign. This may show that only RAT Writers may be able to replace the value of the DK **6906** or rekey it. When using SDFT methods, non-RAT members of a Lock Node may opt to leave the original folded data containing the eDK in the Parameters section and erase the unfolded structure holding the DK.

[0418] A MATLOCK in FIG. **70**, also known as a matroyshka lock, cascade lock or AND lock, is a Variable Lock that may accept a fixed number of symmetric cipher keys called Main keys **7006** and may successively decrypt the eDK **7022** using each Main key **7008** in ascending order using an appropriate cryptographic cipher **7014** such as AES-256 or alternative cipher. The Parameter section may indicate the exact cipher to use for this logical operation and the number of Main keys that may be required, or the preferred TAR when using SDFT methods. The successful ordered iterative decryptions of the eDK **7022** may produce the DK **7024** and may result in the successful unlocking of the MATLOCK. Prior to a decryption attempt in any Variable Lock, the dign of the eDK **7004** may be authenticated using the eDK **7022** and the RAT Reader key **7002**. If the authentication is successful **7010**, then the decryption process may continue otherwise an error **7030** may be raised and the attempt may be halted. In this arrangement, the essence of a matroyshka lock may have been isolated and normalized into Keyhole and Variable Lock structures to make it modular. In a folded structure, the authentication step may be part of the TAR and may be implicitly attempted by the act of unraveling.

[0419] FIG. **71** depicts the encryption operation of a MATLOCK from the viewpoint of a RAT

Writer or Nut owner. It may take some or all of the Main Keys **7102** presented and may sort them in descending order **7110**. Then, it may iteratively perform encryption operations **7112** on DK **7120** using an appropriate cipher to produce an eDK **7122**. Then using its RAT Writer key **7124**, eDK **7122** and an appropriate signing algorithm **7118**, it may create a sign of eDK **7126** which may be stored in the Lock Node Parameters section **6044**. SDFT methods may fold many of these attributes in a compact way along with the eDK into a single data object to be stored in the Parameters section. The encryption process for non-RAT members of a Lock Node may be simple; they may either erase the application memory contents of the Lock Node since they may not create an authentic sign on anything implying they may not successfully change its contents and sign it, or they may use the already decrypted DK **7120** and they may encrypt the relevant contents of the Lock Node but may leave the eDK **7126** untouched since nothing may be changed that may be relevant to the eDK sign. This may show that only RAT Writers may be able to replace the value of the DK **7120** or rekey it. When using SDFT methods, non-RAT members of a Lock Node may opt to leave the original folded data containing the eDK in the Parameters section and erase the unfolded structure holding the DK.

[0420] A XORLOCK in FIG. **72**, also known as a XOR lock, is a Variable Lock that may accept a fixed number (>1) of symmetric cipher keys called Main Keys **7206** and may produce a calculated key by successively applying XOR operations **7224** on each Main key **7208** in ascending order **7222**. Then it may attempt to decrypt **7228** the eDK **7210** using the calculated key from **7224** with an appropriate cipher such as AES-256 or alternative cipher. The Parameter section **6030** may indicate the exact cipher to use for this logical operation and the number of Main Keys that may be needed which may be no less than two keys, or the preferred TAR when using SDFT methods. The successful decryption of eDK **7210** may produce DK **7212** and may result in the successful unlocking of the XORLOCK. Prior to a decryption attempt in any Variable Lock, the sign of the eDK **7204** may be authenticated using the eDK **7210** and the RAT Reader key **7202**. If the authentication is successful **7220**, then the decryption process may continue otherwise an error **7230** may be raised and the attempt may be halted. In this arrangement, the essence of an XOR lock may have been isolated and normalized into Keyhole and Variable Lock structures to make it modular. In a folded structure, the authentication step may be part of the TAR and may be implicitly attempted by the act of unraveling.

[0421] FIG. **73** depicts the encryption operation of a XORLOCK from the viewpoint of a RAT Writer or Nut owner. It may take some or all the Main Keys **7302** presented and may sort them in ascending order **7320**. Then, it may iteratively perform XOR operations **7322** on the Main Keys **7304** to produce a calculated key which may be used to encrypt **7326** the DK **7306** to produce the eDK **7308**. The RAT Writer key **7310**, eDK **7308** and an appropriate sign algorithm **7328** may be used to create a sign of eDK **7312** which may be stored in the Lock Node parameters section **6044**. SDFT methods may fold many of these attributes in a compact way along with the eDK into a single data object to be stored in the Parameters section. The encryption process for non-RAT members of a Lock Node may be simple; they may either erase the application memory contents of the Lock Node since they may not create an authentic sign on anything implying they may not successfully change its contents, or they may use the already decrypted DK **7306** and they may encrypt the relevant contents of the Lock Node but leave the eDK **7312** untouched since nothing may be changed that may be relevant to the eDK sign. This may show that only RAT Writers may be able to rekey the DK **7306**.

[0422] When using SDFT methods, non-RAT members of a Lock Node may opt to leave the original folded data containing the eDK in the Parameters section and erase the unfolded structure holding the DK.

[0423] A HASHLOCK in FIG. **74**, also known as a hash lock, is a Variable Lock that may accept a fixed number of symmetric cipher keys called Main Keys **7406** and may create a calculated key by concatenating **7424** some or all the Main Keys presented in a particular order **7422** and then it may

apply a hashing algorithm **7426** on the string. Then it may attempt to decrypt **7428** the eDK **7410** using the calculated key with an appropriate cryptographic cipher such as AES-256 or alternative cipher. The Parameter section **6030** may indicate the exact cipher and hash to use for these logical operations, the number of Main Keys needed and/or the sorting order of the Main Keys, or the preferred TAR when using SDFT methods. The successful decryption of the eDK **7410** may produce DK **7412** and may result in the successful unlocking of the HASHLOCK. Prior to a decryption attempt in any Variable Lock, the dign of the eDK **7404** may be authenticated using the eDK **7410** and the RAT Reader key **7402**. If the authentication is successful **7420**, then the decryption process may continue otherwise an error **7430** may be raised and the attempt may be halted. In this arrangement, the essence of a hashing lock may have been isolated and normalized into Keyhole and Variable Lock structures to make it modular. In a folded structure, the authentication step may be part of the TAR and may be implicitly attempted by the act of unraveling.

[0424] FIG. **75** depicts the encryption operation of a HASHLOCK from the viewpoint of a RAT Writer or Nut owner. It may take the Main Keys **7502** presented and may sort them in ascending order **7520**, then may concatenate them **7522** and then may produce a calculated key by performing a hash operation **7524** on it. This calculated key may be used to encrypt **7526** the DK **7506** and may produce the eDK **7510**. The RAT Writer key **7508**, eDK **7510** and an appropriate dign algorithm **7528** may be used to create a dign of eDK **7512** which may be stored in the Lock Node parameters section **6044**. SDFT methods may fold many of these attributes in a compact way along with the eDK into a single data object to be stored in the Parameters section. The encryption process for non-RAT members of a Lock Node may be simple; they may either erase the application memory contents of the Lock Node since they may not create an authentic dign on anything implying they may not successfully change its contents, or they may use the already decrypted DK **7506** and they may encrypt the relevant contents of the Lock Node but leave the eDK **7512** untouched since nothing may be changed that may be relevant to the eDK dign. This may show that only RAT Writers may be able to rekey the DK **7506**. When using SDFT methods, non-RAT members of a Lock Node may opt to leave the original folded data containing the eDK in the Parameters section and erase the unfolded structure holding the DK.

[0425] A SSLOCK in FIG. **76**, also known as a secret sharing lock or Shamir's secret sharing scheme, is a Variable Lock that may accept k of n Main keys **7606** each of which may be a distinct tine or secret sharing share and where $1 < p+1 \leq k \leq n$ and $p+1$ may be the minimum number of keys required called the threshold. To recover the secret key, some or all tines from the decrypted Key Maps **7606** may be provided to an appropriate secret sharing cipher **7622** such as Shamir's Secret Sharing Scheme or alternative cipher. The recovery may be successful if some or all the tines may be valid and there may be a sufficient number of them. Then it may attempt to decrypt **7624** the eDK **7608** using the recovered secret key with an appropriate cryptographic cipher such as AES-256 or alternative cipher. The Parameter section **6030** may indicate the exact ciphers to use for the secret sharing and ciphering operations as well as the number of shares (n) and threshold count ($p+1$) for the secret sharing cipher, and/or the preferred TAR when using SDFT methods. The successful decryption of eDK **7608** may produce DK **7610** and may result in the successful unlocking of the SSLOCK. Prior to a decryption attempt in any Variable Lock, the dign of the eDK **7604** may be authenticated using the eDK **7608** and the RAT reader key **7602**. If the authentication is successful **7620**, then the decryption process may continue otherwise an error **7630** may be raised and the attempt may be halted. In this arrangement, the essence of a secret sharing lock may have been isolated and normalized into Keyhole and Variable Lock structures to make it modular. In a folded structure, the authentication step may be part of the TAR and may be implicitly attempted by the act of unraveling.

[0426] FIG. **77** depicts the encryption operation of a SSLOCK from the viewpoint of a RAT Writer or Nut owner who may be encrypting the Lock Node for the first time or who may be in the process

of rekeying the Variable Lock. A new secret cipher key, K, may be generated **7720** and then the desired number of shares (tines) may be created from K using an appropriate secret sharing methodology which may be specified in the parameters **6030**. These tines may be then stored as Main Keys **7702**. In step **7724**, the key K may encrypt DK **7704** producing eDK **7706**. The RAT Writer key **7708**, eDK **7706** and an appropriate dign algorithm **7726** may be used to create a dign of eDK **7710** which may be stored in the Lock Node parameters section **6044**. SDFT methods may fold many of these attributes in a compact way along with the eDK into a single data object to be stored in the Parameters section. The encryption process for non-RAT members of a Lock Node may be simple; they may either erase the application memory contents of the Lock Node since they may not create an authentic dign on anything implying they may not successfully change its contents, or they may use the already decrypted DK **7704** and they may encrypt the relevant contents of the Lock Node but may leave the eDK **7706** untouched since nothing may be changed that may be relevant to the eDK dign. This may show that only RAT Writers may be able to rekey the DK **7704**. When using SDFT methods, non-RAT members of a Lock Node may opt to leave the original folded data containing the eDK in the Parameters section and erase the unfolded structure holding the DK.

[0427] The descriptions of the Variable Locks and the illustrations of their various logical operations may show how a Lock Node may employ Primary Keyholes **6102** in the Input Section **6006**, Encrypted Key Maps **6010**, Key Maps **6008**, Variable Locks **6012**, Encrypted Derived Keys **6014** and/or Derived Keys **6016** to create a robust data structure that may allow for different locking techniques to be normalized and modularized so that substituting one for another may require some parameter **6030** changes and/or rekeying. The normalization of the different locking methods may assure that user Primary Keys for the Nut may be untouched and that a single user Primary Key may be employed in many different locking techniques in different Nuts unbeknownst to the user and which locking techniques may be deemed appropriate for the protection of the particular Nut payload. Sections were highlighted where SDFT methods may prove advantageous in the embodiment of some of these complex data structures. Here are some examples. An ORLOCK may allow multiple users to gain access to the Lock Node's Bag: this may be a form of group access or one of the keys may represent a master key. A MATLOCK, XORLOCK or HASHLOCK may assure that a certain number of keys may be present in order to unravel its Bag: a sensitive corporate secret may require two specific senior executives to supply their respective secret keys to view its contents. An SSLOCK may require a minimum number of secret keys may be present in order to gain access into its Bag: a corporate payment system may be accessed by a minimum number of authorized personnel but it may not be operated alone.

[0428] By compartmentalizing each Primary Keyhole with its corresponding Key Map, the Key Map may contain attributes for the Primary Key such as but not limited to expiration date/time, countdown timer and/or expiration action. If any of the expiration attributes have been set off, then a corresponding expiration action may be set to be performed upon Primary Key expiration. For example, a typical expiration action may be to delete the Key Map of the Primary Key. The deletion of a Key Map may not interfere with any other registered Primary Keys of the Keyhole Lock Node due to its compartmentalized design. Reinserting the expired Primary Key may no longer be recognized as a valid key because there may be no matching Key Map for it. Of course, such Primary Key deletions should be done carefully in regards to the type of Variable Lock being employed: deletions may be acceptable for ORLOCKS and some SSLOCKS but it may be counterproductive to MATLOCKS, XORLOCKS and HASHLOCKS since it may create a lock-out situation for that Lock Node.

[0429] The interplay of complex data structures which may utilize a plurality of cryptographic techniques for the purpose of protecting its contents in a variety of ways and layers may pose significant challenges in the implementation details due to the unusually large number of variable attributes required and/or produced per cryptographic operation. It is in such circumstances where

the utility and elegance of SDFT shines and may provide convenient organizing methods and structures to assist in overcoming such implementation challenges. For instance, a single authenticated ciphering of data may require the following attributes to be stored somewhere: key type, key length, cipher type, cipher mode, initialization vector, key ID, padding, padding type, padding length, block length, digital signature or keyed MAC string (digest), matching key ID for digest, digest length, digest key length, digest method. Multiply this by each ciphering operation described in the Lock Node specification thus far presented (the Lock Node has several more components to be discussed in later sections) and it may be an enormous number of attributes to keep track of. In many instances, application programmers and designers may be aware of such quandaries and challenges and may opt to simplify the coding process by selecting a handful of ciphering methods and associated attribute values and using them throughout their implementation in a global fashion. Such simplifications may lead to undesirable consequences such as but not limited to less security, less flexibility, less features, more incompatibilities, and computer code that may be harder to maintain or modify.

Stratum

[0430] FIG. **78** shows a block diagram of a Nut (Lock Graph) **7800** highlighting Stratum key usage. Each Lock Node in the Nut Parts **7804** section may be assigned a stratum ID. Lock Nodes **7820-7824** are stratum ID 'A', Lock Node **7826** is stratum ID 'B', Lock Node **7828** is stratum ID 'C' and Lock Node **7830** is stratum ID 'D'. The designation of strata may be arbitrary but may follow a pattern of grouping together various Nut Parts by privacy sensitivity: the deeper the stratum, the more sensitive the data contained in the Lock Node may be. By the precise use of Stratum Access Controls (SAC), one may implement a Gradient Opacity of a Nut. For illustrative purposes, the stratum IDs depicted in FIG. **78** are simple letters but in practice may be any set of identifiable strings such as but not limited to Nut IDs (as in the practically unique ID from FIG. **55**).

[0431] Any Lock Nodes comprising the Nut Lock **7802** may be assigned a stratum. When the Keyhole Lock Node of the Nut **7806** is properly unlocked or unraveled, it may reveal a Key Map **7840** which may comprise up to three key sets **7842** (similar to FIG. **62**). This section may concentrate on the Stratum Keys **7850 (6212)** and how they may function within a Lock Graph. In this example, we may find four stratum keys **7852, 7854, 7856, 7858** which may correspond to strata 'A, B, C, D' respectively. Each stratum key may be stored in the Stratum Keys **7850** section with the associated stratum ID. We may follow the flowchart presented in FIG. **79** that shows how each stratum key may be used. Once some or all the stratum keys may have been inserted into the Primary Keyholes of their matching Lock Nodes, the process may be finished and we may wait for the traversal of the Lock Graph to continue beyond the Nut Lock section **7802**.

[0432] The Stratum Keys may work in conjunction with a MATLOCK Variable Lock as shown in some or all the Lock Nodes in the Nut Parts **7804** section. When using SDFT methods, a MATLOCK may be indicated by a 'lock matlock' transmutation in the preferred TAR of the section involved. Each Stratum Key may be a mandatory key in a MATLOCK for the Lock Node in question (* in FIG. **79**). If either the Lock Node Output linking key or the Stratum key may be missing, then the particular Lock Node may not be unlocked as per definition of a MATLOCK. Therefore, some or all deeper strata beyond that level may not be opened as well. By controlling which Stratum Keys may be stored in a Key Map **7840** of the Primary Key, the Nut owner may explicitly control how far someone may penetrate the Lock Graph **7860** with precision. The Stratum Access Control layer may work independently from the Nut Access Control layer and it may work in conjunction with the Variable Locks method.

[0433] The methods by which SAC and Keyholes may work may imply that if multiple keys may be presented into a Keyhole Lock Node such as **7806**, there may be multiple Key Maps **7840** being revealed and possibly multiple Stratum Key sets **7850** that may get inserted into the various Lock Nodes. The stratum keys of a single stratum ID may be identical keys, thus inserting the same key

into a Lock Node that may utilize a MATLOCK may result in one key being inserted under that ID, basically the same key may be overwritten several times in the keyhole. This may be an additive access attribute property of Stratum Keys.

[0434] The Stratum Keys and Nut Access Control (discussed in the next section) both may exhibit an Additive Access Attribute property or characteristic. The insertion of Primary Keys of differing access levels into the Primary Keyhole of a Lock Graph may result in the access level of the Lock Graph that may represent the combination or union of the access levels of all the valid inserted Primary Keys. One powerful use of this property may be in the distribution of keys for a given Lock Graph in a segmented fashion where a combination of Primary Keys may be needed in order to gain a very specific level of access into the Lock Graph. This may contrast with a mode of operation where a Primary Key may present the complete picture of given access for that key holder.

Nut Access Control

[0435] Nut Access Control or NAC is an access control method using cryptographic data structures that may work independently from Variable Locks and Stratum Access Control. NAC may use a combination of Role Based Access Control (RBAC) and Cryptographic Access Control (CAC) which we may refer to as Role Based Cryptographic Access Control (RBCAC) or Key Based Permissions (KBP). NAC attribute key sets may be localized to a single Lock Node's internals, however, there may be mechanisms in a Lock Node to propagate the NAC attributes along the rest of the Lock Graph which may allow the key holder a consistent level of accessibility throughout the associated Lock Nodes. These NAC attributes may be found in an unlocked or unraveled Keyhole Lock Node for the Primary Key which may have been inserted from an external source. Similar to the Stratum Keys, NAC keys may exhibit an additive access attribute property.

[0436] KBP may be deployed using well known properties of Public-key cryptography such as creating digital signatures (dign) and authenticating them asymmetrically on a string of data using algorithms such as RSASSA-PSS (RSA probabilistic signature scheme with appendix based on the Probabilistic Signature Scheme originally invented by Bellare and Rogaway) or alternative algorithm. The basic premise of KBP may be that given a private/public key pair, the private key holder (writer) may create a digital signature (dign) on a parcel of data using the writer's private key and then the public key holder (reader) may use the writer's public key possessed by the reader to authenticate that the dign was created by the writer on the parcel of data. If the authentication fails then something may have been compromised such as the public key, the parcel of data or the dign or some or all of them. The writer may be responsible for creating an updated dign on the target data parcel upon every modification of it and the reader may be responsible for authenticating the dign and the target data parcel prior to "reading" or decrypting the data parcel. This process may reasonably assure the reader that he may be reading something that may have been created or modified by someone who may have the counterpart private key (writer). In Role Based Cryptographic Access Control (RBCAC), there may be an asymmetric key pair for each defined access role and the "writer" of the role may get the private part of the key and the "reader" of the role may get the respective public part of the key. By segregating the dataset by function and digning each functional dataset using different key pairs, access roles may be precisely defined and may be assigned to various key holders by distributing the appropriate key parts. NUTS' RBCAC may allow for the coupling of one or more symmetric keys with the defined role's asymmetric key pair to provide an additional layer of control over the target dataset. The holders of a coupled symmetric key may decrypt and read the target dataset for that role. This coupled symmetric key may encrypt the target dataset on top of the encryption by the symmetric key revealed by the unlocking of the Variable Lock and the subsequent keys in the eKS. Alternatively, the existence of a coupled symmetric key may override the use of the revealed encrypting key from the eKS and may be the only key to symmetrically cipher the target dataset. This alternative may be preferable for large target datasets since it will not be encrypted more than once. The coupled symmetric key

may be used to control the reading access to a target dataset.

[0437] The use of SDFT in an embodiment of NAC may significantly simplify the coding tremendously. The encryptions and digns may be embedded into logically cohesive TARs appropriate for the functions to be performed and the unraveling process of SDFT may automate much of the detailed processing of such operations. Any localized attributes associated with the TARs may be folded together with the target data or be further folded with another TAR to simplify its protection and storage.

[0438] The table in FIG. **80** shows an example of how Key Based Permissions may work with three defined roles, Readers, Writers and Verifiers, and five role players: A, B, V, X and Y. All role players in possession of the coupled symmetric key S may have the ability to encrypt or decrypt the data using the symmetric key S. The Class of Writers (COW), X and Y, may have the ability to create a dign on the encrypted data using asymmetric private key R. Using asymmetric public key U, the Class of Readers (COR), A and B, may have the ability to verify that the corresponding digital signature was created by someone from the Class of Writers on the encrypted data and they may have the ability to decrypt the data using symmetric key S. Therefore, the ability to create a valid dign may imply that you may have the ability to modify the data and all other Readers may authenticate that the dign may have been created by a valid Writer. The number of roles defined depends on the access control granularity desired by the owner but some or all defined roles may utilize the methodology as described for FIG. **80**. A role player who only possesses the asymmetric public key U may be known as a Verifier; the Verifier may have the capability to traverse an entire Nut but may be unable to decrypt the target data corresponding to the role class. For example, a COR Verifier may only authenticate that the payload of the Nut may have been properly modified by a proper COW role player by using the COW public key on the dign but she cannot decrypt the payload since she does not have a copy of the decryption key S.

[0439] The NAC may precisely affect and control the viewable and modifiable aspects of content thereby that of a Lock Node thereby that of a Nut. The table shown in FIG. **81** lists some parts of a Nut but may contain more or less parts as desired: hair, tick, seal, vita, face, tale and/or bale. There may be some forward references in the table to Nut Logs and Nut History which may be explained in detail later in the document. Each row may represent a Lock Node and the data defining the Nut Part may be held in the Bag of that Lock Node. The column titled Bag Opacity may show the cipher mode of the Lock Node's Bag which may be controlled by the Lock Node's metadata. The Bag may be encrypted or not (clear) based on the metadata settings which may be referred to as the Bag Opacity. If some or all of the Nut Parts in the table in FIG. **81** exist in a given Nut, then each Nut Part which may be represented by a Lock Node may be linked in sequence from the top down using Lock Node linking pointers and linking keys. The traversal down the column of this table with respect to the Bag Opacity of each Nut Part may be referred to as the Gradient Opacity of a Nut. Holders of a proper external Primary Key may gain access into a Nut by eventually unlocking the Variable Lock of the Lock Node. Depending on the SAC settings of the Primary Key, a key holder may be limited to how far they may traverse into a Nut. The NAC may affect which Primary Keys may be allowed the ability to read, modify and/or authenticate each Nut Part by the careful placement of coupled symmetric cipher keys, the precise use of asymmetric key pairs, and using digital signature methods.

[0440] FIG. **82** shows a table listing the Key Based Permission access roles that may be defined and available for a typical Nut. The access roles may not be limited by this list as there may be more or less access roles defined depending on the needs of the Nut owner. The table lists four sections of a Nut that may be identified but is not limited to it: Bale, Vita, Tale and All. The All section may refer to anything not covered explicitly by another access role. This may entail some or all the internal workings of a Lock Node such as but not limited to digns on Key Maps, eDKs and/or encrypted Bags not specified by a key pair representing a separate access role. For this discussion, a key pair may comprise the asymmetric key pair and a coupled symmetric key. The

existence of the coupled symmetric key may depend on the existence of an access class Verifier role. The holder of the All private key may be called a RAT (Root Access Tier) or the owner of the Nut. Each Primary Key may have a Key Map which may contain a copy of the RAT Reader public key for authentication purposes. The Bale may be held in the Bag of the Lock Node holding a payload of a Nut such as a document. This key pair may be specifically named Class of Writers (COW) and Class of Readers (COR) due to its frequent use. This key pair may control which Primary Key may have the ability to modify a payload of the Nut. In a similar fashion, the Nut Log may be held in the Bag of the Vita part of a Nut and may be controlled by a Logger/Log Reader key pair. The Nut History may be held in the Bag of the Tale part of a Nut and may be controlled by a Historian/History Reader key pair. A Verifier role for each access role class may have access to at least one public key associated with that role in order to authenticate the dign associated with it. A Verifier role may imply that there may be a coupled symmetric key associated with the access role class which it does not have access to. A maintenance process may be given access to a combination of defined verifier roles within a Nut to check the validity, consistency and/or authenticity of the Nut part but not be able to read the contents of the protected data. The key pairings are not limited to these sets and may be expanded or contracted based on requirements. Any encrypted and/or unencrypted strings of a Lock Node may have a dign created on it by its own specific key pair, and, all the Lock Nodes in a Lock Graph may employ this level of specificity which may lead to an extreme level of access control granularity; however, such an extreme level of access control granularity may mute the effectiveness of the access control on such Nuts.

[0441] The Parameters section of the Lock Node may specify the digital signature algorithm to apply and the length of the asymmetric key (defaults to a minimum of 2,048 bits for RSA-2048). Alternatively, SDFT usage may allow for a specific TAR to represent such preferences and the TAR label may be stored in the Parameters section instead. The encrypted Bag of the Lock Node that may be holding a payload of the Nut may not be digitally signed by a RAT Writer using the RAT Writer key but rather by a key holder having COW access which may include the RAT Writer. Primary Key holders may be given access to the RAT Reader key via their Access Key Set in their Key Map of the Keyhole Lock Node and a corresponding Access Attribute Propagation Key (AAPK); this RAT Reader key may allow any legitimate Primary Key holder to authenticate any dign within the Lock Node which may be in the province of RAT authority (exemplified by a Primary Key holder who may have access to the RAT Writer key). Any failure to authenticate any RAT dign may imply that the corresponding string or folded data may have been compromised, or the RAT Reader key may be invalid, or the Primary key may be no longer valid or some or all of the reasons mentioned. The application may show this warning and may not proceed beyond it since the integrity of the Nut may have been compromised and further decryption attempts may be unlikely to succeed or may result in showing compromised data.

[0442] FIG. **83** shows how a Nut attains its initial set of NAC access keys. Starting with the Keyhole Lock Node **8300**, a Primary Key **8304** may be inserted into the Primary Keyhole **8306** and may decrypt or unfold the Encrypted Key Map which may reveal the Key Map structure **8310**, there may be an Access Key Set (AKS) **8312** which may contain a set of keys composed of Access Attribute Key Set Unlock Keys (AAKSUK) **8314** which may be symmetric. Each individual AAKSUK symmetric key may correspond to an access role as shown in the table in FIG. **82**. Each AAKSUK in the AKS may be then inserted into the Access Keyhole **8320** in the same Input section **8302** of the same Lock Node **8300** as the initial Primary Keyhole **8306**; therefore the Key Map **8310** may hold a set of keys in the AKS **8312** that may feed its own Access Keyhole **8320**. This may be a special property of Keyhole Lock Nodes (external facing Lock Nodes) and may not be applicable to internal Lock Nodes in most cases. In the Access Keyhole **8320**, each properly inserted AAKSUK **8314** may decrypt or unfold to reveal a corresponding Access Attribute Key Set (AAKS) **8330** comprising access role description **8332**, Access Role Key (ARK) **8334** and/or Access Attribute Propagation Key (AAPK) **8336**. The ARK **8334** may specify a key pair part that

corresponds to the role being given: public (reader) or private (writer). The AAPK **8336** may be a symmetric key that may act as the AAKSUK into the next linked Lock Node's Access Keyhole. The set of AAKSUKs may make up a set of AAKSs which may define the Primary Key's NAC access attributes and ultimately its access in the Lock Node. In this diagram, the AAKS **8330** may specify the access attributes of a Nut owner because it contains the RAT private key and the COW key. The additive attribute property of AAKSUKs (thereby the additive attribute property of NAC) may be illustrated in this diagram; there may be an AKS **8312** for each Primary Key **8304** which may be inserted into the Primary Keyhole **8306** so every insertion of an AAKSUK **8314** into the Access Keyhole **8320** may be additive. Identical AAKSUKs may just overwrite the existing one by the Access Keyhole which may lead to a union of unique AAKS when some or all presented Primary Keys may have been processed. This may result in a cumulative access attribute effect when Primary Keys of differing access attributes may be inserted at the same time.

[0443] FIG. **84** illustrates how the AAPKs may be used to propagate the NAC attributes throughout the rest of the Lock Nodes in the Lock Graph. The Keyhole Lock Node **8400** may have been properly unlocked and some or all the AKS **8402** may have been inserted into the Access Keyhole **8404** which may result in the AAKS **8420**. The Access Attribute Propagation Keys (AAPK) **8424** may then be inserted into the Access Keyhole **8432** of the next linked Lock Node. Notice that this may be similar to the way the Keyhole Lock Node's Access Keyhole may have been populated but the keys come from a linked Lock Node rather than from the AKS which may or may not be found in its own Primary Keyhole. The Internal Lock Node's **8430** Primary Keyhole (not shown) may have an empty AKS in its Key Map except for RAT access level keys. By following this propagation methodology, the access attributes of the Primary Key may be present in every opened Lock Node in the Lock Graph. The Lock Node may isolate and localize some or all of its internal control mechanisms such as having different sets of AAKS being generated for its own use within the Lock Node even though the access role maybe the same such as COW. Even the AAKSUK and AAPK symmetric keys may be different as long as they may be mapped properly. It may be the premise of well-defined Nuts to assign the RAT with a complete set of AAKS for the entire Lock Node and for that to be propagated properly throughout its Lock Graph. For reference, there may be a complete set of AAPK and ARK which may be encrypted by the RAT public key and may be stored in the Lock Node's Parameters section so only the RAT may reveal it when it may need to rekey a Lock Node.

[0444] FIG. **85** illustrates the propagation of access attributes using AAPK from an External Lock Node **8500** to an Internal Lock Node **8530**. The diagram shows where the various keys may come from to feed the linked Node's Primary **8550** and Access Keyholes **8560**. The Output section **8510** may reveal a linking symmetric key **8512** for the Primary Keyhole **8550** of the linked Lock Node **8530**. The AAPK **8522** may be inserted **8524** into the Access Keyhole **8560** of the linked Lock Node **8530**.

[0445] FIG. **86** shows a flowchart for inserting keys into an Access Keyhole which may have been covered in detail using examples in the previous sections.

[0446] FIG. **87** shows a table of Key Based Permissions for an alternate embodiment. This table may expand upon the table presented in FIG. **80** by further defining a Write asymmetric key pair (U.sub.W, R.sub.W) and a per instance Data ciphering symmetric key S.sub.n. The three keys from FIG. **80** may be alternatively represented as a Dign asymmetric key pair (U.sub.D, R.sub.D) and a default Data ciphering symmetric key S.sub.0. The additional keys may allow the ARK to define a WriteOnly access role which may write into a Lock Node's Bag but may not read any other part of the Bag. A WriteOnly role may have access to the keys R.sub.D, U.sub.D, U.sub.W and S.sub.n. When a WriteOnly role wants to save message T.sub.n into a Lock Node's Bag, it may create a single instance symmetric encrypting key S.sub.n to encrypt T.sub.n producing encrypted message En. Then the single instance symmetric encrypting key S.sub.n may be encrypted using an asymmetric cipher with key U.sub.W to produce encrypted key K.sub.n. Both En and K.sub.n may

now be saved within the Lock Node's Bag. The WriteOnly role may also create a dign using the R.sub.D key and save it. Message authentication may be alternatively or additionally performed by the proper application of an authenticating SDFT TAR sequence which may embed and automatically fold such information for compactness and organizational simplicity. Such TAR sequences may allow for an alternative method of message authentication using any keyed MAC transmutations. Once the WriteOnly role player may be done writing and the in-memory instance of S.sub.n may be destroyed, the role player may no longer have access to the S.sub.n key to decrypt message En since the WriteOnly role may not possess the asymmetric private key R.sub.W. Only those access roles which may possess a copy of the asymmetric private key R.sub.W, such as the Reader and Writer roles, may decrypt the encrypted key K.sub.n to obtain S.sub.n and may use it to operate on encrypted message En to obtain the original message T.sub.n. The authentication methodology may additionally include hash or dign chaining similar to the way Merkle trees work to make the process of authenticating more efficient for payloads comprising numerous individual messages. WriteOnly role access may not prevent an unauthorized truncation or overwriting of previous messages on the Lock Node being operated on by the local system; however, the NUTS ecosystem may help prevent or highlight such occurrences by engaging its Nut history, replication and synchronization features in various cooperative ways. This will be discussed later in the section on NutServers and Revision Control modules.

[0447] The limited role capabilities of WriteOnly and Verifier presented by the table in FIG. 87 may help alleviate some of the issues associated with the pervasive “God Key” conundrum within computer systems security. This may be a well known class of problems where in one case a system administrator may be given the “God Key” or all access credentials to a system or set of systems in order to maintain, upgrade, repair, install and/or troubleshoot the system(s) at hand. There may be a tendency in the industry to automatically correlate technical ability with elevated security clearances due to the relatively small number of very capable and experienced system administrators with a proper security clearance check. This type of practice may fail to address the dynamic nature of trustful relationships where the trust level between two parties may change over time in a unilateral manner that may not be detectable by the other or may be intentionally hidden from the other. By the careful use of WriteOnly and Verifier access roles, payloads may be protected from unauthorized access at all times for data in transit or at rest. The application of these two access roles may allow an institution to separate the conjoined nature of technical ability and security clearance to fully manage each aspect more appropriately and independently. The WriteOnly role may allow persons and processes to add to the Log component of a Nut as evidence of handling but may not allow them to read the payload or edit the Log. Additionally, the WriteOnly role may have access to both Dign keys and may create authentication strings and verify them. The Verifier role may allow persons and processes to check a Nut for internal consistency and authenticity without allowing any access to the payload. Lock Nodes may be systematically modified, adapted and inserted within any database system such as but not limited to noSQL or RDBMS to enact such granularity of access controls at the field, record, table and/or database levels. The compactness, flexibility, features and/or independence may allow Lock Nodes to exist in computerized appliances as embedded access gateways into the appliance itself. This may be discussed in more detail in a later section on the Internet of Nuts.

[0448] NAC features may encompass a complete set of permutations on the actions that may be taken on a target payload. A simple cross reference matrix of permitted actions along with its NAC implementation may be shown as follows:

TABLE-US-00002 Actions Read Write Verify Read READER WRITER READER Write WRITER
WRITEONLY WRITEONLY Verify READER WRITEONLY VERIFIER

The READER and WRITER roles may have the implicit ability to Verify or authenticate the dign contained within the Lock Node's Bag.

[0449] To summarize the three methods of protection for a Lock Node: Variable Locks, Stratum

Access Control and/or Nut Access Control. The Variable Lock may primarily protect the Bag of the Lock Node which may be used to carry some data content. The Stratum Access Control may define how deep a user may penetrate into Lock Graph Strata. The Nut Access Control may specify which parts of a Nut may be modified, viewed, written and digitally signed by a user. Some or all of these layers may be controlled by embedded or folded key sets within the Keyhole mechanism of a Lock Node. The Keyhole mechanism may be a flexible entryway which may allow for a wide variety of cipher keys to be inserted and processed for a variety of functions. Some or all of these components may work together and/or separately to offer a rich set of access controls that may be customized on a per Nut basis and may be modularly constructed to exhibit the locking behavior that may be desired for the content to be protected. The Lock Node's modularity also may afford the simplicity of building many complex locking structures because of its iterative, compact and modular design. Although many different algorithms may be used to fully unlock and utilize a Nut, the information to initiate the mechanisms may be represented by ciphered data portions that may be stored entirely within the Lock Nodes of a Nut therefore its access control mechanisms may be portable and may travel with its payload independent of any external reference monitors. These mechanisms may further be embodied by various SDFT methods and structures to help simplify the implementation and better manage the complexity of the internal coding and/or data details.

[0450] A Nut's access control models may be a combination of Mandatory Access Control (centralized), Discretionary Access Control (user centric) and others. It may resemble the Discretionary Access Control model in the way it may store some or all of its access attributes within itself and the methods by which the owner may directly set the access levels per Nut in order to facilitate transportability. It may also accommodate some or all Mandatory Access Control models and may integrate into some or all such environments due to its flexibility provided by its Keyholes, Variable Locks and other mechanisms. Furthermore, it may exhibit other characteristics such as but not limited to Gradient Opacity, Additive Access Attributes and/or modular Lock Node linking which may be novel to NUTS.

Lock Node Traversal

[0451] Now we may traverse the entire Lock Node and see how things may be unveiled along the way. FIG. **88** depicts a simplified diagram which shows the decryption data flows within a Lock Node **8800**. References may be made to elements of other figures involved in this interwoven and integrated depiction of a Lock Node unlocking process such as FIGS. **62, 78, 83, 84, 85** and **88**. References may be made to the same Lock Node section numbered by different element numbers but may represent a different view of the same section being examined in a drill down type of approach. The sequencing of the logical operations which may be required in the Lock Node unlocking process may be further optimized for efficiency and/or other purposes. The process of unlocking a Lock Node, thereby eventually a Lock Graph or Nut, may involve these steps which may be described in this example such as but not limited to the use of Primary Keys to get access privileges and decryption keys to the Lock Node, the authentication of the Lock Node, the propagation of access privileges throughout the Lock Graph, the logical operation of a Variable Lock and/or the decryption of the stored data; these steps may be expanded, contracted or reordered as may be needed. If appropriate, certain mechanisms within the Lock Graph and Lock Node may benefit from an appropriate application of SDFT methods.

[0452] Primary Keys **8804** may be inserted into the Input section **8806** and each Primary Key may use its associated cipher method to attempt to decrypt its matching Encrypted Key Map **8810** and unfurl it into a Key Map **8808** structure. Each Key Map **6240** may produce a Main Key **6210** which may be used by the Variable Lock **8812**. Within each Key Map **7840** (equivalent to **6240**) may be a set of Stratum keys **7850** and each Stratum key (such as **7852-7858**) may be inserted into the matching Strata Lock Nodes (such as **7820-7830**) of the Lock Graph in the respective Input section's **8302** Primary Keyhole **8306** (in this example, a Stratum key such as **7852-7858** may be equivalent to a Primary Key in **8304**); Stratum designated Lock Nodes such as **7820-7830** may

employ a MATLOCK which may require a minimum of two keys to open it: the Stratum key such as **7852** and the Output linking key such as **8512** which may be found in the Output section **8510** or **8826**. For a Keyhole Lock Node **8300**, within each Key Map **8310** may be a set of Access Attribute Key Set Unlock Keys (AAKSUK) **8314** called the Access Key Set (AKS) **8312** and each AAKSUK key may be inserted into the Input section **8302** Access Keyhole **8320** of the current Keyhole Lock Node **8300**. Once a set of Access Attribute Propagation Keys (AAPKs) **8336** may have been attained in this manner, they **8522** (equivalent to **8336**) may be inserted into the next linked Lock Node's **8540** Access Keyhole **8560**. Now we may have an Access Attribute Key Set (AAKS) **8332** which may contain the Access Role Keys (ARK) **8334**. The ARK may define the Primary Keys' **8304** access roles for the entire Lock Graph. The Digns of the various Lock Node sections such as **8840-8848** may be authenticated using these ARKs. The Dign of the Lock ID & Metadata **8840** may be authenticated using the RAT Public ARK **8344** (this may be the public portion of a RAT asymmetric key pair as may have been described in the NAC specifications) and the authentication algorithm specified in section **8830**. To authenticate, section **8830** may be submitted into the authentication algorithm along with the corresponding dign **8840** and RAT Public ARK **8344**. If the authentication fails, then section **8830** may have been compromised and the Lock Node unlocking process may raise an error and may stop processing. If successfully authenticated, then each Dign of the Encrypted Key Maps **8842** may be authenticated for each Encrypted Key Map corresponding to a valid inserted Primary Key. To authenticate, each eKM **8810** string may be submitted into the authentication algorithm along with the corresponding dign **8842** and RAT Public ARK **8344**. If an authentication fails, then the eKM may have been compromised and the Lock Node unlocking process may raise an error and may stop processing. If all appropriate eKMs have been successfully authenticated, then each Dign of the Encrypted Derived Key **8844** may be authenticated. To authenticate, each eDK **8814** may be submitted into the authentication algorithm along with the corresponding dign **8844** and RAT Public ARK **8344**. If an authentication fails, then the eDK may have been compromised and the Lock Node unlocking process may raise an error and may stop processing. If all appropriate eDK have been successfully authenticated, then each Dign of the Encrypted Key Set **8846** may be authenticated. To authenticate, each eKS **8818** may be submitted into the authentication algorithm along with the corresponding dign **8846** and RAT Public ARK **8344**. If an authentication fails, then the eKS may have been compromised and the Lock Node unlocking process may raise an error and may stop processing. If all appropriate eKS have been successfully authenticated, then each Dign of the Encrypted Bag **8848** may be authenticated. To authenticate, each eBag **8822** may be submitted into the authentication algorithm along with the corresponding dign **8848** and COR ARK **8348**. If an authentication fails, then the eBag may have been compromised and the Lock Node unlocking process may raise an error and may stop processing. If all appropriate eBag have been successfully authenticated, then this Lock Node may be deemed fully authenticated. Note that the eBag may be authenticated using the Class of Reader (COR) Access Role Key **8348**. This may hold true for Lock Nodes holding a payload of the Nut, but for Lock Nodes holding Nut metadata in their Bags, the RAT Public ARK may instead be used to authenticate it. Then based on the Variable Lock type indicated in the Lock Node's Parameter section **8830**, an appropriate Variable Lock algorithm **8812** may be attempted on each encrypted Derived Key string (eDK) **8814** using the set of Main keys **7844** from the Key Maps **8808**. Successfully unlocking the Variable Lock **8812** by decrypting an eDK **8814** may result in one or more Derived Keys (DK) **8816**. Each Derived Key may decrypt a corresponding encrypted Key Set string (eKS) **8818** which may be stored in the Parameters **8802**. Decrypting an eKS may produce a corresponding Key Set **8820** structure which may hold an Output section **8826** structure and a Bag key. The Output linking key(s) which may be found in the Key Set **8820** structure may be stored in an Output section **8826** and it may function as a key that may be inserted into the Primary Keyhole of a linked Lock Node **8530** if any. The Bag key may decrypt the encrypted Bag string (eBag) **8822** that may be stored in the Parameters section using an appropriate cipher. A decrypted Bag may hold

data such as but not limited to a payload of the Nut (Lock Graph), metadata about the payload, metadata of the Nut, metadata of the Bag, any combination of these and/or other data. A Bag metadata may indicate whether the Bag **8824** holds a Nut Part or Nut payload. If a Bag holds a Nut Part, it may indicate which Nut Part it may represent and other appropriate Nut Part metadata and/or other data. If the Bag holds a payload of the Nut, it may indicate whether the stored data may be actual data or a reference to it and if so what type of reference it may be, what the reference may be and/or where it may be located.

[0453] This series of steps may be repeated for each Lock Node in the Lock Graph in order to unlock the Nut. FIG. **89** shows the general flowchart of Nut unlocking. Most of the steps may have been detailed in the previous example but some steps may need further clarification. Step **8902**—Organize Lock Nodes into proper traversal levels: since Lock Nodes may be stored in a row based form in a list type structure, the actual topology of the Lock Graph may be extracted and constructed using the linkage information which may be stored within each Lock Node. Once the graph may be constructed, then one or more additional passes may be done to properly assign graph levels so that Lock Nodes may be traversed in the proper sequence. Step **8908**—Prompt for some or all passphrase based keyholes: during the processing of an Input section, if a passphrase based keyhole is encountered with an empty key (passphrase), then it may prompt for the passphrase. This default behavior may be modified to call another function or bypass any empty passphrase keyholes. Any logical step or process in the flowchart may have errors that may be raised and may lead to the process being halted and these are not specified in detail because this is a higher level flowchart: for example, any process which attempts an operation may fail and may halt the algorithm. The rest of the flowchart may follow along the path of the previous example.

[0454] FIG. **90** illustrates how a NUTS based system may open a document contained in a Nut. A forward reference is introduced: NUTbook **9000** may be a data collections organizing application that may use Nuts and may essentially act as a personal PKI when it comes to storing and organizing collections of passwords, cipher keys and/or certificates. The file symbols such as **9002** and **9004** may be used throughout the diagrams to represent a Nut file. There may exist within a NUTbook system a Main NUTbook access key Nut **9002** which may be unlocked to get some minimal functionality from the application. The key may be stored within Nut **9002** and may be called the Main NUTbook key and the unlock mechanism into the Nut **9002** itself may comprise a passphrase. There may be a hierarchical key relationship with the Main NUTbook key **9002** and a Documents access key **9004** such that in order to access any document holding Nuts in this configuration, the Document access key may be needed. Therefore, the hierarchy may be set as needing the Main NUTbook key **9002** to open and access the Documents access key **9004**. The Nut holding the Documents access key may have Nut ID #33 **9016**. Therefore the key which may be stored in the payload **9020** may be referred to as Key ID #33: both the Documents access key and the Nut ID of the Nut holding it may be referenced by the same ID, in this case #33. The document **9040** may be stored in a Nut **9030** with Nut ID #44 **9036**. Similarly the document may be referred to as document #44. When a user decides to open document #44, one of the keyholes in the Primary keyhole **9032** may specify that it may need Key ID #33 to open it. Nut #33 **9004** may be requested from the NUTbook and in order to open that, it may need Nut **9004** to be opened. For that Nut to be opened, Nut **9002** may need to be opened. Suppose, the user may have already initialized his NUTbook with a passphrase to Nut **9002** and the NUTbook may have cached the Main NUTbook key in memory.

[0455] Then opening Nut **9004** only may require the additional passphrase for the Documents level access of the NUTbook and once it may be opened, the cascade of Nut unlockings may occur to eventually reveal the decrypted document #44 **9050**. The NUTbook may cache the Documents access key for a finite amount of time to expedite document fetching during a session but certain events such as inactivity, hibernation, screen lock, timeouts, and/or explicit locking may require the passphrase to be entered again for documents access. This section introduced the NUTbook

application and hierarchical passwords concepts which may be further discussed in a later section. The series of steps which may be needed to open a single document may be many but some or all the logic employed may be based on the Lock Nodes and its iterative processes and most of it may be hidden from the user. The end result may be that a piece of data may be stored in a Nut like **9030** and its security may be consistent in some or all environments.

[0456] FIG. **91** illustrates the common usage in NUTS parlance to refer to a Nut's payload by the Nut ID of the Nut holding it. Here it shows how a keyhole **9124** tagged for Key #33 may be actually looking for a Nut **9110** with Nut ID #33 **9112** and it may expect Nut #33 to be holding a single key **9114** that may be inserted into the keyhole **9124**. It may be interesting to note that in many of these diagrams and examples, the filename of a Nut file, if the Nut may be stored in a file, may be rarely referred to in most operations.

[0457] The next set of diagrams shows various example embodiments of a Lock Graph which may highlight the flexibility and expressiveness of the Lock Node and Lock Graph model using Variable Locks and Lock Node linking.

[0458] FIG. **92** shows a simplified embodiment of a list of recipients locking model: any one key **9210** may unlock the ORLOCK Lock Node **9220** which may reach the Lock Node carrying the Payload of the Nut **9290**. Note that for simplicity, a Lock Node may be pictorially represented as a padlock but in reality it is a fully functioning Lock Node which may be storing some metadata for the Nut.

[0459] FIG. **93** shows a simplified embodiment of an ordered locking model: key **9310** may be presented first then key **9320** second which may allow access to the Payload of the Nut **9390**. MATLOCK Lock Node **9312** may require a single key whereas MATLOCK Lock Node **9322** may require both key **9320** and the linking key from Lock Node **9312**.

[0460] FIG. **94** shows a simplified embodiment of an ordered locking model with master key: key **9410** may be presented first then key **9420** second which may allow access to the Payload of the Nut **9490**. Or, master key **9430** may be presented to ORLOCK Lock Node **9432** directly which may allow access to the Payload **9490**. The ORLOCK Lock Node **9432** may allow either the linking key or the master key to unlock it.

[0461] FIG. **95** shows a simplified embodiment of a locking model with master key: key **9510** or master key **9520** may be presented together or separately which may allow access to the Payload of the Nut **9590**.

[0462] FIG. **96** shows a simplified embodiment of a locking model with master key: key **9610** or master key **9620** may be presented together or separately which may allow access to the Payload of the Nut **9690**. A MATLOCK placement in the Lock Graph **9632** may indicate that there may be certain Stratum controls in place for this Nut and it may be a Nut Part storing some Nut metadata.

[0463] FIG. **97** shows a simplified embodiment of a safe deposit box locking model: key **9710** and bank key **9712** may be presented together which may allow access to the Payload of the Nut **9790**.

[0464] FIG. **98** shows a simplified embodiment of a secret sharing locking model with master key: from a set of keys **9810**, a number of keys meeting or exceeding the secret sharing threshold may be presented together which may allow access to the Payload of the Nut **9890**. Or, master key **9820** may be presented to ORLOCK Lock Node **9822** directly which may allow access to the payload **9890**. The keys **9810** may be any combination of passphrases, symmetric keys and/or asymmetric keys since the Keyhole/Key Map structures may hide the times that may be needed by the secret sharing scheme being utilized in the SSLOCK Lock Node **9812**.

[0465] FIG. **99** shows a simplified embodiment of a “PrivaTegrity” type locking model: user key **9920** may be presented to the ORLOCK **9922** which may allow access to the payload **9990**. Or, nine keys **9910** may be presented together to the MATLOCK **9912** which may allow access to the Payload of the Nut **9990**. The “PrivaTegrity” model may have been proposed in early 2016 by David Chaum to implement a text messaging system that may securely transmit messages using keys known to its users but it may have a collusionary backdoor system which may involve up to

nine different keys being held by nine international jurisdictions in order to give law enforcement access to specific messages if and only if all nine jurisdictions may agree that it may be vital and may be legally warranted.

[0466] FIG. **100** shows a simplified embodiment of a multi-Nut configuration where multiple payloads may be stored within a single Nut: user keys **10010** or master key **10012** may access one payload or both which may depend on their Stratum access controls. Master key **10020** may only access payload **10092** due to its traversal path through the Lock Graph. This Lock Graph may display the flexibility of the modular Lock Nodes and its access control layers working together. Separate data parcels may be protected in different ways some or all within this single Nut. If the master keys **10012** and **10020** may be the same, it may allow the key holder access to both payloads.

[0467] FIG. **101** shows a simplified embodiment of a multi-Nut configuration: any of the user keys **10110** may access some or all three payloads which may depend on their Stratum access controls. The keys **10120** for SSLOCK **10122** may only access payload **10194** due to its Lock Node linking. This Lock Graph may display the flexibility of the modular Lock Nodes and its access control layers working together and/or individually. Separate data parcels may be protected in different ways within this single Nut. The flexible nature of this disclosure may permit infinite variations of locking configurations.

[0468] FIG. **102** shows a simplified embodiment of a direct locking model with multiple payloads: this lock graph may show a flat topology for a Nut rather than the usual linear one. The ORLOCK **10212** may be the interesting node in that there may be several ways to implement the multiple linking keys needed to connect it to five different Lock Nodes. In one embodiment, the Output section of the ORLOCK node **10212** may contain five output keys. In another embodiment, an output linking key map may be embedded as a Key Map in the Keyhole and then may be propagated into the Output section. Furthermore, the Stratum Keys may also play a role as to which keys may access the various nodes.

[0469] FIG. **103** shows a simplified embodiment of an ordered message passing example: it may be a collusion resistant design using Nuts and Relationship Based Keys (RBK. A forward reference). Mr. Data **10310** may have a relationship each with Alice, Bob, Charlie and Danny. Some or all the participants may know each other. His relationship may be symbolized by having relationship based keys with each person. Mr. Data may want to send a set of secret instructions to each person but he may want the messages to be read in a certain sequence without the possibility of peeking ahead by collusion amongst the participants. Therefore, Mr. Data may construct these four Nuts with specific contents in each. The Nut sent to Alice **10320** may only be opened by Alice because it may be locked using the RBK set between Alice and Mr. Data. Inside **10320** may be a message for Alice and a key for Bob, K.sub.Bob. She may read her message and may send Bob the key K.sub.Bob. The Nut sent to Bob **10330** may employ a MATLOCK which may only be opened using two keys simultaneously: Bob's RBK key between Bob and Mr. Data and the key K.sub.Bob from Alice. Inside **10330** may be a message for Bob and a key for Charlie, K.sub.Charlie. He may read the message and may send Charlie the key K.sub.Charlie. The Nut sent to Charlie **10340** may employ a MATLOCK which may only be opened using two keys simultaneously: Charlie's RBK key between Charlie and Mr. Data and the key K.sub.Charlie from Bob. Inside **10340** may be a message for Charlie and a key for Danny, K.sub.Danny. He may read the message and may send Danny the key K.sub.Danny. The Nut sent to Danny **10350** may employ a MATLOCK which may only be opened using two keys simultaneously: Danny's RBK key between Danny and Mr. Data and the key K.sub.Danny from Charlie. Inside **10350** may be a message for Danny. He may read the message and Mr. Data's plan for ordering the messages may have worked.

[0470] In the cybersecurity field, a 'back door' feature may bring forth negative connotations in the various dialogues surrounding the topic. Traditionally, back door mechanisms may have been implemented at the application levels which may have allowed unfettered access to the data being

processed by that application. This type of application level access may have been construed as a severe compromise to the security of the data processed by that application depending upon which party gained access to that back door entry. The perception of compromise in such situations may have been well founded due to the prevalence of such applications mostly handling unencrypted data within its own application memory thereby potentially granting access to cleartext data to the back door user. In NUTS and in particular in a Nut's locking model, some may view the use of a Master Key as a type of back door into a Nut; however, technically it may be quite different because in all locking models of a Nut, all doors (keyholes) are front doors and requires the proper cryptographic key to gain access into the Nut. The NUTS API or any NUTS related application embodiment may not have an intended back door designed at the application level. There may be numerous legitimately good reasons to have Master Key entries available to Nuts, but all such entries may only be defined by a secret key and may be directly noticeable by a cursory examination of any Lock Node's Input Section. Therefore, any application attempting to install a back door type functionality within a NUTS related application may only do so after first gaining access to a Master Key for the target set of Nuts, and it may only be applicable to those Nuts where that Master Key is valid. This may illustrate the flexibility, compartmentalization, protection and/or resiliency of the data centric approach to the security of a Nut.

[0471] In some or all methods of access control in NUTS there may be involved a pattern of hiding cryptographic keys within encapsulated data structures whose unfolding may reveal other keys which may allow access to a target dataset. In the embodiments illustrated in this disclosure, most of these key hiding methods may use data encapsulation and or data folding methods. The method of hiding access keys may be a preference made by the implementer or it may be a parameterized setting withing each nut. These methods may comprise data folding, data encapsulation, attribute based encryption, functional encryption, authorization tokens from reference monitors, or any other method that may provide selective cryptographic revealing of subsequent access keys when provided with access material that decrypts or unlocks its cryptographic mechanism. The demonstrative embodiments in this disclosure may have been chosen for their simple and straightforward mechanics and their well known characteristics. Other equivalent mechanisms may streamline or make more efficient certain aspects of the embodiments but they may still essentially provide the same functionalities, that of controlling access to access attributes that may grant access to a target dataset with precision and may be independent of any reference monitors by default. Any equivalent access attribute revealing methodology may be substituted for the methods illustrated so far to provide the same level of protection for the contents of a nut.

[0472] This may conclude the section about the Nut container and its internal workings. The internal mechanisms may be embodied directly or by the usage of SDFT methods which may ease the coding and management of such an embodiment. The payload of the Nut may be what the Nut ultimately may protect which may be any storable digital data such as but not limited to a text file, a binary application, an image file, access keys to a remote system, executable scripts, credentials to establish a computer to computer connection securely, entire databases, operating systems, links to other Nuts, streaming data and/or text messages. Due to the Nut's ability to describe what it may be holding through its rich configurable metadata, the standard list of common file types may fall far short of its holding capabilities. The Lock Node architecture may allow for payloads to span Nuts thus it may result in unlimited logical container sizes. If solid state NUTS compatible chips or circuitry may be available, it may be possible to turn a physical device into a Nut itself thus the device may only be accessed by the key holder. A series of such devices may constitute entire networks and intranets that may be operable only with proper authentication. The flexible nature of the modular Lock Node design may permit infinite variations of locking configurations for a Nut. In the following sections, various systems and/or methods may be introduced which may use Nuts as the basis of secure storage to show how some common services and methodologies may be expanded, improved and re-designed to offer capabilities that may have seemed beyond the reach

of the average user.

Modular I/O

[0473] A significant amount of a programmer's efforts may be spent on making sure data may be properly brought into a program, transformed in its running memory space, calculated and/or edited and then may be properly stored persistently. A nasty byproduct of this mode of application development may be that of the eventual obsolescence of file formats and their various versions. Owning, possessing and controlling one's own data may be useful and admirable goals but of what use is it if you may not read it properly? The ability to read a format, write a format, act on the read data and/or display the data read may constitute some of the fundamental components of a typical program. Modular I/O (MIO) may be a system and/or method of modularizing these logical operations into a repository of modular components which may be used by anyone who may access it. A byproduct of MIO may be the ability to create file format conversion modules which may allow users to access past versions of file reading and writing routines so that their older data may be readable. This may be called backward compatibility. A concept of forward compatibility may be offered as well but the utility of this feature may be dependent on the skillfulness of the programmer who may design the application modules. It may be a preferred embodiment of a MIO system that some or all modules may be encapsulated in Nuts therefore the authentication, protection and/or access control of each module may exist by default. FIG. **104** shows the typical components in Modular I/O. Element **10450** may be a Modular I/O Repository (MIOR) which may be a server process that may store and organizes MIO components. A MIOR may be embodied as a local and/or remote server type application that may act as an intelligent cache for such modules. In other embodiments a MIOR may have a local cache on the local device so it may better facilitate commonly requested MIO modules. A typical application **10402** that may read and/or write to a file **10404** may be conceptually and programmatically broken up into modules to read **10406** and to write **10408** the file. File **10404** may be formatted in a specific format "A" that may be specific to application **10402**. Similarly, this figure shows two other applications **10410** and **10418** with corresponding data files **10412** and **10420** and their respective read and write modules **10414**, **10422**, **10416** and **10424** which may be stored in the MIOR **10450** for the specific formats that they may be in "B" and "C". The MIOR may contain other modules that may perform different tasks or procedures for the application. Depicted by **10426-10432** may be file conversion modules which may perform transformations from one file format to another as specified by its respective labels: module Convert_A_B **10426** may take data read into an application's memory from file format "A" by file reading module **10406** and then it may transform the memory structure to that resembling a memory structure that may be created by the file reading module File_Read_B **10414**.

Modular I/O: Reading and Writing

[0474] FIG. **105** shows a simple reading and writing operation using the MIOR **10500**. Application **10502** which may process files that may be stored in file format "A" may read the file F_A **10504** formatted in format "A" by requesting a file reading module File_Read_A **10506** from the MIOR **10500**. The module **10506**, if found, may be transmitted **10510** to App_A **10502** at which point App_A **10502** may install and may execute file reading module File_Read_A **10506** on file F_A **10504**. Module File_Read_A **10506** may perform the file reading on file F_A **10504** and may construct the internal memory structure that may represent the contents of file F_A **10504**. This memory structure that may represent the contents of file F_A **10504** may be then transferred to the calling application App_A **10502**. Once successfully transferred, App_A **10502** may continue to perform its functions with the contents of file F_A **10504** that may be present in its running memory space. In other embodiments, there may be no need to transfer the memory structure to App_A **10502** once the file contents may have been read by file reading module File_Read_A **10506** if there may be a facility whereby both the file reading module **10506** and application module **10502** may share the same memory space.

[0475] When application App_A **10502** is ready to store the modified contents of file F_A **10504**

back into file form, it may contact the MIOR and may request a file writing module for file format “A” called File_Write_A **10508**. Upon receiving **10512** module **10508**, App_A may install and may execute it using the same methodology for transferring application memory structures as the reading process. The writing module **10508** may perform the write operation to persistent storage which may create a modified file F_A **10520**. The requests to the MIOR for the reading and writing modules **10506** and **10508** may be done in any sequence that may be deemed appropriate by the application developer. In one embodiment, the application may request some or all relevant I/O modules up front before proceeding in order to be sure that some or all necessary I/O operations may be performed by the application which may prevent any undesirable failures later on. In another embodiment, there may be a locally cached MIOR of previously fetched modules by previously run applications that may be maintained in order to expedite the request and fetching procedures.

[0476] There may be many methods of transferring and/or sharing the memory structure between two or more logical processes to a person having ordinary skill in the art such as but not limited to shared memory segments, memory mapped files, databases, inter-process messages, binary memory dump files, and/or converted memory dumps. The preferred method of application memory transfer in a MIO system may be to use converted memory dumps between processes. JSON read and write functions may be modified to recognize binary data and automatically may convert them to and from base64 encoding or other binary-to-text encoding schemes. FIG. **106** shows the data transformations and transfers that may be involved in a typical MIO file reading operation. MIO reading module File_Read_A **10604** may read **10620** the file named F_A **10602** in format “A” into its running memory **10606**. Thus the relevant contents of the file **10602** may be represented **10630** in the application memory structure **10606**. The application memory may be stored into a JSON compatible data structure **10606** and may be marshalled into text form **10610** using a JSON write function call. Optionally, the JSON output may be embedded into a Nut container **10608** for added security. Thus the application memory **10606** may have been converted and stored **10608** outside of the reading module **10604**. The Nut **10608** may then be opened and read into memory by App_A **10612** and a JSON read may be performed on the data parcel **10610**. Thus reconstructing the data into App_A's **10614** running memory. The data transfer methods **10622** and **10624** may include but is not limited to command line arguments, inter-process messages, and/or data file(s). The read application and/or data processing application may be separate processes on different machines, the same machine, separate threads or separate cores; or the applications may be a single logical process on a local or remote machine with the dynamic capability to modify its running code on the fly.

Modular I/O: Backward Compatibility

[0477] Applications may undergo progressive changes over time by issuing version changes with enhancements throughout its lifetime. Many of these version changes may include format changes of the storage files used to save the user's works. Historically, this may lead to two issues: encumbrance and obsolescence. Encumbrance may be when software gets bloated due to adding backwards compatibility capabilities into every version for every format change for the life of the product line. This may involve quite a number of format version changes. Furthermore, if there may be other third party or open formats that the application may want to handle, then it may result in more software bloat. FIG. **105** illustrates how for any version of any format that the application may utilize, if modular read and write modules may be available in the MIOR, then the file may be read and processed without any excessive bloat. Furthermore, FIG. **105** illustrates how newer read and write modules may be independently added to the MIOR and every application that may communicate with the MIOR may now have access to the additional formatting modules without any program modifications. These newer modules may be the ability to read different versions of a file format for the same application product line or it may be compatibility modules to read and write third party file formats written by anyone including the application developer.

[0478] FIG. 107 shows a backward compatibility example where the version of the application App_B 10702 may be more recent and may use a corresponding newer format version “B” of the data file but the user may desire to read and write an older version “A” of the file format 10704. Data conversion modules such as 10708 and 10710 may be created and stored in the MIOR 10700 for such cases. A conversion module may be responsible for reading in one format and producing another format: in this example conversion module 10708 may read in an “A” formatted file and may convert it into a “B” formatted file; conversion module 10710 may read in a “B” formatted file and may convert it into an “A” formatted file. The file F_A 10704 may be presented to App_B 10702 where it may determine that the file may be in an incompatible format from its metadata and may proceed to make a request to the MIOR 10700 for the reading module sequence that may be needed to read “A” and may produce “B” files. The MIOR 10700 may respond by sending the following modules to App_B 10702: File_Read_A 10706, File_Write_A 10712, Convert_A_B 10708 and Convert_B_A 10710. App_B 10702 may invoke File_Read_A 10706 on file F_A 10704, File_Read_A 10706 then may invoke Convert_A_B 10708 and may transfer its memory structure in “A” form to the module 10708, then module 10708 may convert the data received into “B” form and may transfer it to App_B 10702. When App_B is ready to save the modified data back to file F_A in “A” format, then it may invoke Convert_B_A 10710 and transfer its memory structure in form “B” into the module 10710, then Convert_B_A 10710 may convert its memory structure to form “A” and may invoke File_Write_A 10712 and may transfer its memory structure in “A” form, then File_Write_A 10712 may write over file F_A 10714 with its modified contents in form “A” and may format the file writes in file format “A”. More complex examples may be where several conversion modules might be called in sequence to perform an iterative conversion process to the appropriate older format version or a developer may have added a frequently used version change converter modules such as Convert_B_F and Convert_F_B in order to streamline such requests.

[0479] The software bloat may be illustrated with a simple calculation: suppose a popular application may have undergone 5 major revisions, 3 file format versions across 3 operation systems with 3 major version changes each over 10 years. Let's also suppose that every one of these changes may have required a different version of the I/O routines for the applications. This may potentially lead to the most current version of the application to carry up to 135 versions of its I/O functions within itself. Granted that this may be an extreme case, one may understand the proliferation of program code that may be generated in order to maintain backward compatibility in an application over time. This characteristic may be referred to as the encumbrance property of software.

[0480] A properly maintained MIOR 10700 with consistently updated modules being added to its repository may act as a historical I/O format library and may allow users to access older versions of their data files at any time in the future: this may address the issues of software and data format obsolescence. When an application may be no longer produced, sold, and/or maintained, its useful life may be shortened drastically because newer versions that may allow it to run on newer operating system versions may not be forthcoming. When an application may no longer be run on modern computers due to incompatibilities, the data files formatted by the application may be difficult to access. Clever users and developers may have found various solutions to these issues but it may require much effort and/or specialized knowledge on their part. Using a MIOR may require that at least one developer may maintain the modules that may be associated with the now defunct application and he may make newer versions of the modules to be added periodically that may be compatible with newer versions of various operating systems. This type of routine maintenance may be automated using automated unit testing tools and auto-generating OS type and version appropriate modules in a timely manner. The updated modules may be inserted into the MIOR and everyone that may have access to the MIOR may benefit from the developer's work; if the particular MIOR may be accessible by everyone on the Internet, some or all users on the Internet may benefit from it automatically without requiring the user to be knowledgeable about the

lower level issues and those processes which may be invoked to automatically resolve them. Software backward and forward compatibility issues may be referred to as the obsolescence property of software.

Modular I/O: Forward Compatibility

[0481] A user sometimes may experience a situation where he may have bought, installed and/or used an application many years ago but he may have not purchased the subsequent upgrades to it over the years. However, the application may still be functional for him but it may only read and write file formats that may be compatible to his older version of the application. The newest version of the application may have introduced a newer file format with additional features at some point in the past. This situation may present two problems for the user: 1) his version of the application may not read files formatted in the latest format version, and 2) other programs that may read the latest format from this application may not be able to access his older formatted data. The solution to the first problem may be called a Forward Compatibility Read operation whereby his older application may directly load a set of modules from the MIOR that may perform progressive conversions on the data which may allow him to read files formatted in a newer version using his older program. The solution to the second problem may be called a Forward Compatibility Write operation whereby his older application may directly load a set of modules from the MIOR that may perform progressive conversions on the data which may allow him to write files formatted in a newer version using his older program. Programs built with forward compatibility in mind may make this type of transition easier and seamless using MIOR with minimal or no loss of functionality. Newer features offered in more recent format versions may be optimally mapped to less sophisticated application constructs or may be substituted with just the raw data and allow the user to modify it at a later time. FIG. 108 illustrates these two different logical operations with examples.

[0482] Forward Compatibility Read operation: App_A **10802** may be compatible with files formatted in version “A” but the user may want to read a newer file format “C”. This request may be conveyed to the MIOR **10800** and it may reply with a sequence of modules that may perform these regressive conversions: File_Read_C **10806**, Convert_C_B **10808** and Convert_B_A **10810**. The module File_Read_C **10806** may read the file F_C **10804** which may be formatted in version “C”. The module **10806** then may invoke the regressive conversion function Convert_C_B **10808** and may transfer its memory structure to it. Module Convert_C_B **10808** may perform the conversion on the data in memory and may produce a memory structure compatible with format “B”, a previous file format version of the application. Module **10808** then may invoke the regressive conversion function Convert_B_A **10810** and may transfer its memory structure to it. Module Convert_B_A **10810** may perform the conversion on the data in memory and may produce a memory structure compatible with format “A”, the desired file format version compatible with the older application App_A. Module **10810** may transfer its memory structure in format “A” to the calling application App_A **10802** and App_A may process it. Thus a newer version of a file format may be read by an older version of the application without modifications to the application.

[0483] Forward Compatibility Write operation: App_A **10840** may be compatible with files formatted in version “A” but the user may want to write a newer file format “C” which may be beyond its original capability. This request may be conveyed to the MIOR **10800** and it may reply with a sequence of modules that may perform these progressive conversions: File_Write_C **10816**, Convert_B_C **10814** and Convert_A_B **10812**. App_A **10840** may invoke Convert_A_B **10812** and may transfer its memory structure to it. Module Convert_A_B **10812** may perform the conversion on the data in memory and may produce a memory structure compatible with format “B”. Module **10812** then may invoke the progressive conversion function Convert_B_C **10814** and may transfer its memory structure to it. Module Convert_B_C **10814** may perform the conversion on the data in memory and may produce a memory structure compatible with format “C”. Module **10814** then may invoke the file write function File_Write_C **10816** and may transfer its memory

structure to it. Module File_Write_C C **10816** may write the file F_C **10818** which may be formatted in version “C”, the desired file format version. Thus a newer version of a file format may be written by an older version of the application without modifications to the application.

[0484] This disclosure is not limited by the two examples shown. Conversion modules may be produced to access some or all versions of file formats for an application on any operating system. Conversion modules may not be limited to conversions within its application product line but may be written to perform conversions across different application product lines. Conversions modules may include conversions of data to different formats such as but not limited to file to database, database to file, file to data stream, datastream to file, file to webpage, webpage to file, file to cloud storage, cloud storage to file and/or others.

Modular I/O: Display

[0485] FIG. **109** shows a diagram of a MIOR Display module in operation. Once an application App_A **10902** may have successfully read into its memory the data from a file F_A **10904**, it may proceed to use the functionality of module Display_A **10908** to display the contents of file F_A **10904** to the user. In display modules, the functional aspect of the module may vary greatly depending on the application, data content and/or developer's design. Some modules may be designed to use shared memory methods which may allow the display module to directly access the data in the application memory, others may transfer the data to the display module and which may allow it show it. Other embodiments of the display module may be screen formatting instructions and/or templates for the type of data to be shown and possibly may be edited. This modularization of display functionalities may allow for custom display modules to be created for various hardware and OS platforms while allowing the calling program to remain relatively unchanged.

[0486] The Catalog of Collections architecture discussed later in the NUTbook section may make use of the lightweight aspect of Modular Display. Instead of building ever larger monolithic applications to handle, display and/or edit different collections of datasets, NUTbook may make extensive use of the MIOR architecture which may allow it piecemeal customizations based on the type of payload in the Nut being examined.

Modular I/O: Application

[0487] In FIG. **110**, a MIOR **11000** may store modular application modules such as **11022**. A NUTbrowser **11020** (forward reference) may be an application that may be similar in look and behavior to most file and directory browsers but it may recognize Nuts and may act upon them by looking at the Nut's extensive metadata. Within a Nut's **11030** metadata **11002** may be information pertaining to the type of payload it may be protecting and storing. When a user selects a Nut from the NUTbrowser **11020** and double clicks to open it, the NUTbrowser may open the Nut and may read the metadata to figure out what modules may be required to open the file. The metadata may include data such as but not limited to application version, file format version and/or display version. Then the NUTbrowser may make a request **11004** to the MIOR **11000** looking for application App_A **11022**, File_Read_A **11024** and Display_A **11026**. The MIOR **11000** may return some or all modules and the application App_A **11022** may be invoked by the NUTbrowser **11020**. Once App_A is running it may invoke File_Read_A **11024** in order to read the contents of the Nut payload F_A **11028** which may be stored in the Nut **11030**. After transferring the memory structure from **11024** to the calling module App_A **11022**, it may invoke the display module Display_A **11026** to show the data F_A **11028** to the user.

[0488] Modular I/O Application modules may vary greatly in what they may hold and do: in some embodiments it may be a complex logical computational module; in another embodiment it may store an entire software installation package; in another embodiment it may contain some or all aspects of I/O, display and/or application functions; in another embodiment it may contain information containing a Genesis Nut which may kick start the reincarnation of a user's environment in a remote manner. The functionality of Modular I/O Application modules is not limited to these cases.

[0489] Modular I/O features such as Read, Write, Display and/or Application may be overlaid with access control mechanisms at the MIOR or container level so that only properly authorized users may access it. These access control mechanisms may include but is not limited to access control policies, ownership requirements, and/or DRM mechanisms for remunerative purposes. Most of the access controls may emanate from the properties of the Nut containers that the modules may be stored in. As this disclosure is discussed in detail further, it may be made clear as to the mechanisms by which these MIOR requests may be derived. When a data file or its contents may be encapsulated within a secure Nut container, there may be many levels of metadata available about the contents of the Nut, this metadata may specify the details of the data format such as but not limited to application version that created it, display version, file format version, size, create time, last modify time, author, type of file, and/or summary. Environmental attributes such as but not limited to OS version, application version, hardware make and/or version may be provided by the application that opens the Nut. With these pieces of information about the environment, data content and/or requested operation, the MIOR may look up the proper modules and may reply back with either a set of modules to satisfy the operation or an error message. These Modular I/O modules may run as a single or separate processes on the same machine, across different machines, across different chips or cores, across a network and other modes of running a logical process(es) on a computing device.

[0490] Through these modules the problems of obsolescence, encumbrance, adaptability, compatibility and/or flexibility may be addressed in part or whole.

Nut History

[0491] The Nut container may be structured to store the history of the payload. The form of the history may comprise periodic snapshots, progressive deltas, complete event sequences or any combination of the three or any other archiving methods. The form of the history may vary depending on the type of data being stored and the preferences and design of the application and/or data. The NUTS ecosystem may include the methods and systems to support these modes of data history archiving. These three methods of archiving may be well established methods known to a person having ordinary skill in the art. The physical location of the Nut history may be in the Nut Part called the Tale (FIG. 81) and, its opacity may be controlled by the Nut RAT.

[0492] FIG. 111 shows a simplified Nut schematic which illustrates the progressive changes to its history structure over three points in time covering two edit sessions of a document. At time T1, the Nut 11102 may hold data D1 11110 and its history may be empty 11112. The user may edit 11126 the data D1 11110 at time T2 and may produce a new version D2 11114. The application may employ a snapshot archiving method and may store the original version of the data D1 11110 as a snapshot of the data at time T1 11118 in the history section 11116 of the Nut. Subsequently, the user may edit 11128 the data D2 11114 at time T3 and may produce a new version D3 11120. The application may employ a snapshot archiving method and may store the older version of the data D2 11114 as a snapshot of the data at time T2 11124 in the history section 11122 of the Nut. At time T3, the history section 11122 now may hold two distinct snapshots 11118 and 11124 of the previous versions of the data D3 11120. The history 11122 of the Nut may be browsed and extracted by the user at will using simple history extraction methods at any time allowing for reversions or create entirely new documents from them. There may be Nut metadata parameters which may control the type, frequency and/or longevity of the history section in order to set reasonable history growth for the data at hand. For some textual documents, it may be practical to save some or all changes forever since its size may be relatively small when using a delta method of archiving. This may allow the Nut to produce some or all the saved versions of the document at any time thereafter. Binary documents may be archived as snapshots or deltas depending on the application. Certain event driven applications may archive the complete set of event sequences as its history. Note that Nut History may be archived within the Nut itself and may be independent of external programs and/or storage systems. As long as there may be an archival method available for the payload type

in the NUTS environment, any payload may be historically archived in this manner.

Nut Log

[0493] The Nut container may be structured to store the event log of the Nut. As computer processes may read, manipulate and/or write a Nut, they may generate and leave an audit trail of the logical operations done to the Nut within the Nut itself. The audit trail essentially may exist on a per object basis from the object's perspective. Therefore, between Nut history and Nut log, the chronicle of events since inception on the data object may be stored in a single container for further review at a later time. The accuracy, content and/or granularity of the Nut archives may be dependent on the disciplined and methodical usage of these features by the developers of the applications that operate on Nuts. The physical location of the Nut Log may be in the Nut Part called the Vita (FIG. 81), and its opacity may be controlled by the Nut RAT.

[0494] FIG. 112 shows a simplified Nut schematic which illustrates the progressive changes to its event log structure over three points in time covering two events occurring on the Nut. This example may continue the scenario from FIG. 111 for Nut History. At time T1, the Nut 11102 may hold data D1 11110 and its log 11212 may hold one log entry 11218 for event E1 which may indicate that the Nut 11102 was created at time T1. The user may edit 11226 the data D1 at time T2 which may create a new version of the data D2 11114 in the Nut 11104. The edit application may log an event log entry at T2 into the Nut log 11216 as may be indicated by element 11222.

Subsequently, the user may edit 11228 the data D2 11114 at time T3 and may produce a new version D3 11120. The edit application may log an event log entry at T3 into the Nut log 11230 as may be indicated by element 11224. At time T3, the log section 11230 now may hold three distinct event log entries 11218, 11222 and 11224. The log 11230 of the Nut may be browsed and extracted by the user at will using simple log extraction methods at any time which may allow audits on the Nut. There may be Nut metadata parameters to control the type, frequency and/or longevity of the log section in order to set reasonable and appropriate log growth for the Nut.

[0495] System administrators and application developers may know the work and effort that may be involved in tracking down bugs and errors on their systems when more than one application may be involved in modifying a data object because they may have to look through the event logs of some or all the contributing applications (if they may have access to these at all) and may be filter out those event log entries that pertain to the object in question and then perhaps manually reconstruct the events in the sequence in which they might have occurred on the object. Using a Nut Log, this gathering of event logs, filtering and reconstruction may be already done at the object level from the perspective of the object. Furthermore, the metadata of the Nut may specify to the working application the level of granularity of event log message details that may be desired by the object owner. This granularity may range from a terse to detailed debug levels in order to track down various lines of inquiries. A sensitive, top secret payload may require the most granular level of event log details in order to perform an audit trail on its access history. In short, this may be a consistent and customized method of controlling the auditable past of an object by any application on a per object basis per granularity level demanded by the said object. The term consistent may refer to the consistent design and operations of the logging feature available and the term customized may refer to the per object preferences that the design may accommodate.

Relationship Based Keys (RBK)

[0496] The description of how Relationship Based Keys (RBK) may be established should sound familiar to anyone who may have used encryption tools manually: Bob and Alice may want to communicate privately and thus they may trade randomly generated asymmetric cipher keys (public parts only) with each other and may use it in a tool such as PGP or its equivalent to exchange ciphered messages and documents. The protection and management of the key pairs by Bob and Alice may be left entirely up to them. This may tend to be a deliberate and laborious task for each relationship to be established, maintained and utilized properly perhaps requiring Alice and Bob to have a primer or two on ciphers, their proper usage and/or the protection of the keys.

This type of key exchange may occur when either Bob or Alice does not have an established Public Key certificate via a centralized directory or a web of trust. It may also happen if either participant feels that an added layer of privacy might be needed by creating a completely private communication channel.

[0497] What might happen if RBKs were the default method of communication for folks like Alice and Bob? What may be the consequences and what may be needed to make that happen in a painless way? The systematic aspects of the establishment, maintenance and/or usage of RBKs may be automated. It may be constructive to explore some of the properties and consequences of the consistent application of RBKs prior to delving into the details of how it may be accomplished systematically.

Characteristics of Relationship Based Keys

[0498] The trust level between two parties may be a dynamic adjustable parameter. This may be an observation of real life relationships between any two parties: trust may be relative. It may wax and wane over time based on events and communications.

[0499] Unilateral adjustment of trust levels. Either party in a relationship may unilaterally change their trust level of the relationship at will with or without informing the other party.

[0500] The relationship channel health may be determined from message context. Systems and keys may be compromised from time to time for anyone. The default usage of RBKs may allow either party to examine the content of communications and may determine the likelihood of the other person's systems or keys having been compromised. In the simplest case, a message coming from Bob without RBK ciphering may possibly be a sign of being compromised.

[0501] The true nature of a relationship may be assessed over time. If a message of unusual nature is transmitted via RBK and the sending party's key may have not been compromised, then the sending party may have changed the nature of the relationship.

[0502] Losing a relationship may be permanent and some or all history of the relationship may lose commercial and/or meaningful value. Unilaterally, either party may sever the relationship by blocking its messages or erasing their RBK set. This logical operation of a relationship channel may present each user with a deterministic unilateral message blocking ability.

[0503] Parties may strictly abide by mutually amenable ground rules or risk losing the relationship—ground rules which may vary over time. Violation of implicit ground rules may result in the unilateral severing of the relationship in a permanent way, digitally speaking.

[0504] It may allow closer expression of real world relationships in a digital cryptographic form. Public Key cryptography in its most widely used form may be a centralized model which may be contrary to how people form relationships. RBKs may be decentralized and may use Public Key cryptography in a private way.

[0505] Isolation of subversion. The subversion of RBKs on Bob's environment may be isolated to Bob and the RBK channels he may have established with his contacts, i.e. Alice. The damage to Alice's environment may be isolated to her channel with Bob and their mutual historical communiques. Some or all other relationship channels for Alice may be secure and may not be breached by the hackers who subverted Bob's environment.

[0506] A Personal Information Manager or PIM may be a well known application concept in computer software. It may be widely defined as an amalgam of various functions which may provide productivity and organizational tools for an individual's use. A PIM may offer such tools as but may not be limited to calendar, address book, contact management, password keeper, notes, email manager, chat function, project management, key manager, calculator, task lists and/or activity logger. A PIM may be a combination of any of these functions or it may just offer a single function. A PIM may be designed to operate locally in an isolated manner or solely in a PIM web server or in any combination thereof. In the discussions going forward, references to such functionalities of a PIM such as an address book or chat or email manager may be understood to be either a PIM that offers any of those functions as part of its offerings or it may be its sole function.

[0507] FIG. 113 shows how a digital address book entry between Alice and Bob may be structured to support RBKs in a consistent manner for some or all relationships in the address book. Alice's address book **11310**, which may be a function offered by her PIM, may have two entries: an entry for herself **11320** and an entry for Bob's information **11330**. In Alice's own entry **11320**, she may have listed her own basic contact data **11322** and some personal data **11324**. Alice's address book entry **11320** may be stored and may be secured as the payload in a Nut file on Alice's system. On Bob's contact card **11330**, Alice may have some contact information for Bob **11332** such as his name and email address. Bob's address book entry **11330** may be stored and may be secured as the payload in a Nut file on Alice's system. Bob's address book **11350**, which may be a function offered by his PIM, may have two entries: an entry for himself **11360** and an entry for Alice's information **11370**. In Bob's own entry **11360**, he may have listed his own basic contact data **11352** and some personal data **11354**. Bob's address book entry **11360** may be stored and may be secured as the payload in a Nut file on Bob's system. On Alice's contact card **11370**, Bob may have some contact information for Alice **11372** such as her name and email address. Alice's address book entry **11370** may be stored and may be secured as the payload in a Nut file on Bob's system. When Alice and Bob decide to setup RBKs with each other, they may decide to set up a private bidirectional communication channel between themselves. Alice may begin the process by generating an asymmetric key pair **11334** and **11336**, storing them under Bob's address card **11330** and transmitting **11344** the public portion of the key **11334** to Bob. The transmission process **11344** may be accomplished by a passphrase secured Nut, a message written in paper, a telephone call to Bob, a message using Bob's public key known to the world, or any version of secure key exchange protocols well known to a person having ordinary skill in the art. When Bob receives this message with the key **11334** inside, he may store it in Alice's address card entry **11370** as a key **11334** for sending messages to Alice privately. Bob then may generate an asymmetric key pair **11374** and **11376**, storing them under Alice's address card **11370** and transmitting **11346** the public portion of the key **11374** to Alice using the public key that Alice sent to him **11334** to encrypt the message. When Alice receives this message, she may decrypt the message from Bob using her private key for Bob's messages **11336**. She may extract the key **11374** inside, she may store it in Bob's address card entry **11330** as a key **11374** for sending messages to Bob privately. She may create a confirmation message for Bob encrypted with key **11374** from card **11330** and may send it to Bob through any working communication medium. Bob may receive the message, then he may decrypt it with key **11376** from card **11370** and may mark his RBK set to be established and active with Alice.

[0508] The steps in this RBK setup between Alice and Bob may be automated and may be initiated with a single action button or command. This may be the operational basis of how a NUTbook manages its Contacts Collection and may be discussed in the NUTbook section later in this document. The process may be repeated by either Bob or Alice independently for some or all the contact cards in their respective address books in their PIMs. In the end, each person may establish an RBK channel for each of their contacts which may be viewed as private communications channels for each of their relationships. If Cathy is a common friend between Alice and Bob, Cathy's RBK relationship with Bob may be different from Cathy's RBK relationship with Alice and the RBK configuration may reflect that reality.

[0509] Now that we may have defined the RBK and the context of its systematic use, what might it do for Alice or Bob? The consistent use of RBK to send messages between two entities may allow for the monitoring of their communication channel health. An example of a practical use may be SPAM email reduction. It may be estimated that a significant volume of global Internet bandwidth and data storage may be taken up by SPAM emails by both the malicious and/or commercial kinds. We may venture to assume that not many people may welcome such volumes of SPAM. Some of the usual methods of SPAM reduction may be by using filtering technologies based on content pattern recognition, domain exceptions, address exceptions and/or actually taking down prolific

SPAM servers by law enforcement. In a mode where RBK encryption may be the default way of communicating, SPAM may be detected in a more deterministic way.

[0510] One of the main obstacles in the way of automating processes such as RBK may have been the significant lack of user friendly, user accessible and/or user controllable personal Public Key Infrastructure (PKI) applications. The NUTbook along with the usage of Nuts may attempt to fill the PKI gap. It may provide flexible, secure and/or user controllable methods to store, manipulate and access such information in a seamless way.

[0511] FIG. **114** shows the flowchart to reduce SPAM between Alice and Bob who may have now established a RBK communication channel and it may be their default method of communicating and they may be using well known public email addresses. If the email is encrypted via RBK between Alice and Bob, then it may probably be valid email from Alice to Bob or vice versa **11414**. If either person receives an email from the other not encrypted with RBK, then it may most likely be SPAM and may be filtered out and may be stored in the SPAM bin **11412**.

[0512] FIG. **115** shows the flowchart to reduce SPAM between Alice and Bob who may have now established a RBK communication channel and it may be their default method of communicating and they may be using unpublicized, private email addresses—anonymous email addresses. If the email is encrypted via RBK between Alice and Bob, then it may probably be valid email from Alice to Bob or vice versa **11514**. If either person receives an email from the other not encrypted with RBK, then it may most likely be SPAM and may be filtered out and may be stored in the SPAM bin **11512**. This example may be assuming that the set of private email addresses may only be used between Alice and Bob to send each other RBK encrypted messages therefore expanding the RBK channel concept to the email address level as well. We may define this type of communication channel oriented email addresses as Anonymous Email Addresses.

[0513] A communication channel between Alice and Bob that may consistently use RBK via anonymous email addresses may exhibit certain characteristics that may be analyzed to determine the health of the relationship itself. We may have already removed some or all unencrypted SPAM messages from the channel by default as may be described in FIG. **115**. Now we may examine the context of the proper RBK encrypted messages. The table in FIG. **116** lists a Deterministic Context Based Status Matrix of the health of the Alice-Bob communication channel. It may require a qualitative assessment of the content by Alice to figure out what may be happening to their relationship. This shows a unilateral action matrix by Alice that may be based on Bob's behavior as may be evidenced by his messages to Alice.

[0514] The last symptom listed in FIG. **116** may pose an interesting scenario when the role of Bob may be substituted by a web vendor: i.e. Alice may have established an anonymous RBK communication channel with a vendor. The table in FIG. **117** shows the Deterministic Context Based Status Matrix of the health of the Alice-vendor communication channel. Now, Alice may have the ability to track down if this vendor may have sold her information to spammers through the channel identifiable aspects of the anonymous email addresses and the RBK sets. It may provide a level of transparency into the inner workings of the vendor's marketing department with a clear audit trail. This type of vendor accountability may be unprecedented in such a systematically detailed way by an average user. The consequence for violating Alice's trust by the vendor may be dire because the vendor may lose the means to contact her forever. In effect, the proper and consistent usage of anonymous email addresses and/or RBKs may allow for the digital equivalent of Alice walking out of a store and never coming back; this may serve as a deterrent for vendors to not abuse the personal information of their clients.

[0515] FIG. **118** shows the Deterministic Context Based Status Matrix of the health of the Alice-vendor communication channel from the viewpoint of the vendor. The channel characteristics may afford the vendor the same type of unilateral actions he may take to protect his business and possibly protect his clients. The use of this methodology by a vendor may possibly enhance his reputation for privacy and data security. It may also implicitly state that the vendor may not engage

in the wholesale indiscriminate reselling of client data.

[0516] FIG. **119** shows a graphic representation of how the use of RBKs may help isolate the compromise of sensitive data on a user's system. Bob **11900** may have RBK channels with Alice **11910** and Dave **11920**. Bob may have clicked on a Trojan horse website and may have gotten infected with a key logger or equivalent malicious program and subsequently the hackers may have been able to infiltrate his secure data store for RBKs such as his NUTbook. As a result, Bob's RBK sets with some or all his contacts may have been compromised **11900**. Bob may contact some or all his friends and he may notify them of this breach or some of his friends might have already deduced something was wrong with Bob or his system from SPAM messages which might have been sent to them using their private channels with Bob. If we look at Alice's NUTbook **11910** where she may store some or all her RBK sets, she may mark her RBK set with Bob **11912** as having been compromised and may generate a new RBK set whenever Bob gets his act together to remove the viruses on his system. That may be the extent of the damage to Alice and it does not spread to other RBK relationships that she may have established. This may be especially true if she consistently used anonymous email addresses with her contacts as well. Alice may receive SPAM from the hackers but the SPAM may be ignored automatically when she marks the channel as having been compromised or deleted. When Bob may be ready, Alice may generate a new set of RBKs and a new anonymous email channel and they may continue their digital dialogue privately. The process for Dave may be the same for his RBK store **11920**.

Anonymous Relationships

[0517] Digital relationship topologies and conventions that may have arisen and cemented on the Internet in the last few decades may be unnatural and unrealistic. Anonymity may be a powerful relationship construct and may be the level of relationship that we may enjoy on a daily basis with most casual interactions such as but not limited to going to the drug store to buy personal products, going to a restaurant to buy a meal, hailing a medallion cab for a ride and/or showing up at a protest rally. Contrary to this physical reality, almost every vendor on the Internet may want to know exactly who Alice may be including some or all the personal information they may get from her. Many vendors themselves may stay relatively anonymous by not publishing direct phone numbers and may service customers through emails, transaction systems and/or remotely outsourced customer service representatives in remote call centers. The most prevalent use of anonymity may be by those who may want to hide such as hackers. Currently there may be many fake persona generation websites for people who may want to stay anonymous on the Internet but they may have to keep track of anonymity in a very laborious fashion and may have to make conscientious decisions to be purposefully duplicitous. The use of RBKs and anonymous email addresses may bring some parity to this imbalance of anonymity on the Internet for the average user and may empower them to have a more meaningful bidirectional relationship with vendors and each other without having to resort to fake personas and casual duplicity.

[0518] FIG. **120** shows a simplified schematic of Pre-Packaged Personal Data Nuts. A Nut may store detailed personal information about a person **12000**. It may automate the pre-packaging of different subsets of this personal information for different purposes. **12010** may be a Simple Personal Data Nut which may contain just name and email address. An Anonymous Personal Data Nut **12020** may show only an alternate email address. A Shopping Personal Data Nut **12030** may include information fields typically needed for shopping websites to purchase items. The production of these data subsets from the master information **12000** may be done via simple rules and filters and may be produced on demand during a registration process on the Internet. Whether the vendor or service may accept data Nuts or not, the information may be made available for insertion into the correct fields when needed by other means. If the user takes advantage of an Anonymous email service (forward reference), data Nuts like **12020** may offer dynamically created anonymous email addresses for the specific relationship being established.

[0519] FIG. **121** charts the sequence of events in an Automated Registration process that may use

Nuts. A vendor on the Internet may use and accept personal data Nuts and may allow for RBK channels to be established with its customers in an automated manner. A user may visit the vendor's website and may want to register **12100**. The user may start the process by instructing her NUTbook to automatically register to the vendor's website and may input the URL of the registration site. The NUTbook may query the vendor to fetch the information that the vendor may need to get her registered **12102**. The NUTbook may compose a subset of her personal information that the vendor may be requesting and may show her a preview. She may decide the information requested for registration may be acceptable and that the NUTbook may have gathered the pertinent information and may continue the process **12104**. The NUTbook may extract and may create a pre-packaged Nut containing the previewed information and may send it to the vendor's site. The vendor may accept the registration request and may send a query to the user's email address specified in the pre-packaged Nut **12106**. The user may receive the vendor's query on her email asking her to provide evidence that she may not be a web bot which may be engaging in frivolous registrations by asking her to go to a specific URL to enter a captcha **12108** or other form of possible verification. Once the captcha is successfully entered, the vendor may be satisfied that the request may be from a person and may proceed to establish auto-generated login credentials, login key and/or RBK sets with the user. The user's NUTbook may automatically create an entry card for the vendor, its pertinent web information, login credentials, login key and/or RBK set **12112**. The registration process may be done with the user interacting at a few points in the process: initiation, personal data package review/edit, and/or human verification. The hassles of picking a login name, password, typing in personal information and/or creating an entry in a password keeper for the vendor may not be required and may have been automated. When her NUTbook is activated, she may have instant access to the vendor in a fully authenticated mode seamlessly because the NUTbook may automatically log her in when so ordered to do so. Note that this process may be done with any vendor adopting this methodology to the possible benefit of both the user and vendor. Less hassles for the user and the vendor may get more accurate information from the user for their database and perhaps the possibility of more transactions between them.

[0520] FIG. **122** charts the sequence of events in an Automated Registration process using Nuts and an Anonymous email address. A vendor on the Internet may use and may accept Nuts and may allow for RBK channels to be established with its customers in an automated manner using anonymous email addresses. A user may visit the vendor's website and may want to register **12200**. The user may start the process by instructing her NUTbook to automatically register to the vendor's website and may input the URL of the registration site. The NUTbook may query the vendor to fetch the information that the vendor may need to get her registered **12202**. The vendor may accept anonymous registrations so the NUTbook may contact the NUTmail service and may request a pair of anonymous email addresses under her account. The NUTbook may compose and may show a preview of the data to be sent to the vendor registration which may include the newly created anonymous email addresses. She may decide the information requested for registration may be acceptable and the NUTbook may continue the process **12204**. The NUTbook may create a pre-packaged Nut containing the previewed information and may send it to the vendor's site. The vendor may accept the registration request and may send a query to the user's new anonymous email address specified in the pre-packaged Nut **12206**. The user may receive the vendor's query on her anonymous email address asking her to provide evidence that she may not be a web bot which may be engaging in frivolous registrations by asking her to go to a specific URL to enter a captcha **12208** or other form of possible verification. Once the captcha is successfully entered, the vendor may be satisfied that the request may be from a person and may proceed to establish auto-generated login credentials, login key and/or RBK sets with the user. The user's NUTbook automatically may create an entry card for the vendor, its pertinent web information, login credentials, login key, anonymous email addresses and/or RBK set **12212**. The registration process may be done with the

user interacting at a few points in the process: initiation, personal data package review/edit, and/or human verification. The hassles of picking a login name, password, typing in personal information, creating email addresses and/or creating a new entry in a password keeper for the vendor may not be required and may have been automated. When her NUTbook is activated, she may have seamless access to the vendor in a fully authenticated mode because the NUTbook may automatically log her in when so ordered to do so. This process may require no personal user information and the email addresses that may have been created were specifically for this relationship implying only relevant emails between the user and vendor may arrive at these anonymous emails addresses. As various NUT based services may be discussed later, some or all of them offer anonymous registrations.

[0521] Communication channels which may be established using RBKs and anonymous email addresses may minimize SPAM in a deterministic fashion due to its default mode of ciphering everything via RBKs. Furthermore, it may give bidirectional control of the channel to the parties that may be involved so that there may be mutual respect for the relationship and its implied bounds. Deviations from these implied relationship boundaries may pinpoint relationship changing events and may invite a unilateral reaction ranging from inquiries to severing the relationship altogether in a deterministic way. For third parties attempting to subvert Bob or Alice's data, beyond the retrieval of the correct pair of anonymous email addresses the third party may have to crack the ciphered messages and documents as well.

[0522] Websites that may accept and may process automated registrations may add additional services such as but not limited to age filtering. Parents may deposit a pre-packaged Nut on the NUTserver of their child's device to indicate some generic identification features such as but not limited to sex, age and/or general location. This pre-packaged Nut may be automatically used to register the child on any child friendly or parentally pre-approved website that may accept Nuts. The vendor may accept or reject access attempts based on this information and the services they may provide such as but not limited to liquor sites, tobacco sites, movie preview sites, adult content sites and/or firearm sites. Furthermore, an internet activity logging Nut may be configured on the NUTserver of the child's device to monitor their activity and digital whereabouts. Limitations on internet use may also be administered by the parent by using such Nuts across some or all devices in the home so that device switching may be inconsequential to the child's cumulative internet usage per day. The blocking of, or admission to certain websites may be accomplished by using such child identification Nuts on the device itself and/or in conjunction with specific configuration settings on a NUTS based WiFi router (forward reference).

NUTS Core Applications

[0523] The table in FIG. 123 lists the applications that may comprise the NUTS Core Applications set. These applications may reside in most systems that may utilize NUTS technologies and they may handle Nut files as shown in this simplified diagram of an operational computing device in FIG. 124. As previously noted, some or all of these applications may have already been referenced by material discussed earlier in this disclosure. These applications could not be detailed any earlier in this disclosure due to their dependencies on some or all the core foundational functions and capabilities of NUTS such as but not limited to Lock Nodes, Lock Graphs, Nut Parts, Nut History, Nut Log, MIO, MIOR, Nut IDs, RBKs, Gradient Opacity and/or Anonymous Relationships. Some or all of these core applications may prefer to utilize the Nut as the basic unit of storage which may be embodied by an ordinary file but is not limited to it. This may imply that some or all the data that these systems touch, store and/or manipulate may come with a high degree of security and access control by default. Design philosophies, which may have been used in Lock Node design, that may assist the reader in understanding these Core Applications more fully may be the concepts of iteration, integration, independence and/or identifiability.

NUTS Core Application: NUTserver

[0524] A NUTserver may be depicted schematically in a simplified diagram of a user device in

FIG. 125. There may be several key functions that a NUTserver may perform in the background to organize and maintain a NUTS compatible environment. A NUTserver **12510** may run in the application space of a user computing device **12500**. The device may have some storage **12520** where Nut files **12522** may be kept. The NUTserver may be responsible for providing APIs and communication channels open with various applications comprising the NUTbook **12512**, NUTbrowser **12514** and/or other applications **12516** including the device OS. The NUTserver may be also responsible for maintaining external connections with other devices that may belong to the user who may be running NUTservers **12530** and possibly may be conversing with the NUTcloud **12540**. The NUTserver may not be a replacement for the file system of the user device **12500** but rather may work through the local Operating System and File System to access and process any Nut files.

[0525] FIG. 126 shows a simplified diagram of the principal internal parts of a NUTserver and its functionalities. The user device may have an Operating System **12600** managing the hardware and software. The device may have external communications serviced by network interfaces **12602** and its associated drivers running through the OS **12600**. The device also may have a File System **12604** that may be attached and may be managed by the OS **12600**. Stored on the file system may be data stores for the MIOR **12610** and user data may be contained in Nuts **12606** and **12608**. The OS **12600** also may act as an application environment where many applications may be run comprising those depicted in the diagram: NUTserver **12620**, NUTbook **12642**, NUTbrowser **12644**, MIOR Server **12646** and other apps **12648**. The NUTserver **12640** may be running on another device but the application interface **12636** may handle those communications as well.

[0526] Within the NUTserver **12620**, there may be a module **12622** that may perform authentications into the NUTserver and may maintain a key cache. When a NUTserver starts, it may not have any authority to peer into any secured layers in any Nuts. The user and/or the hardware may provide the authentication necessary which may allow the NUTserver authentication module **12622** to gain access to certain key sets. This may be as simple as having a passphrase protected Nut holding the key sets and asking the user to provide the passphrase, opening the Nut and caching into protected/unprotected memory the keys sets in its payload; or it may be secure hardware provided keys as found in many computing devices; or it may be a hardware token such as but not limited to a USB key that a user may provide. The key set may contain at a minimum a NUTserver authentication key and/or a key for each NUTS core application that may be installed on the local device. There may be a Cache **12624** that may be maintained by the NUTserver for organizational purposes and efficiencies. A part of the cache may be the Index **12626** of Nut IDs. This Index may contain some or all the Nut IDs that the user may want to keep track of locally and remotely. Looking up a Nut ID in the Index may indicate where the Nut ID may be found. Another part of the Cache **12624** may be relegated to keeping a Nut cache **12628** in memory for frequently accessed Nuts.

[0527] The NUTserver may be responsible for synchronizing the contents of two or more Nuts with the same Nut IDs **12630**. Once a NUTserver may be properly authenticated and it may have sufficient keys to access some or all the Nuts owned by the user, then it may open various Nuts to examine its contents and manage it. Each Nut may hold a version number and timestamp of last update or modification. If an update occurs on a Nut and the NUTserver may be notified of it or the NUTserver may notice it, then it may note the update and may look up the Index **12626** to see some or all the locations where a copy of this updated Nut might exist locally or remotely. It may then systematically begin to Propagate and Synchronize **12630** the changes to the affected Nuts. This process may be rather simple due to the metadata embedded within each Nut such as but not limited to Nut ID, version number, internal digns, history, and/or log. The newest version may simply overwrite the existing version if various modification criteria may be met. It may not be necessary that a NUTserver be able to peer into a Nut in part or whole since it may depend on the viewable metadata as may be allowed by the Gradient Opacity of the Nut as to whether a

synchronizing update may take place. Sufficient cleartext metadata may allow some Nuts to be synchronized by NUTservers with no keys to the Nuts in question. In cases where they may be a possibility of version forking or branching, the user may be involved to decide which version to make current. The Replication function **12630** may allow peer NUTservers to propagate these types of changes across user controlled devices automatically. The functionalities provided by **12630** may constitute a personal NUTcloud for a user when she may install and connect multiple NUTservers on her devices. She may enjoy synchronized and/or replicated Nuts on any of her devices in an automated fashion. When more complex version issues arise or a certain historical version of a Nut may be requested, the Revision Control module **12632** may handle those requests. It may utilize the specific version delta methods employed by a Nut and may perform a finer granularity of version control to produce the desired version of a Nut. These Nut specific version delta methods and the content read/write methods of Nuts may or may not exist in the local MIOR so there may be a MIOR interface **12634** to supply those functions when they may be needed.

[0528] An Access Nut may be defined as a secured Nut that may contain authentication credentials for other systems or containers such as but not limited to website logins, database logins, corporate systems, personal devices, software systems, other Nuts, NUTservers, email systems, chat systems, and/or any digital system requiring a secret passkey and/or login ID. The NUTserver may present an Application Interface **12636** for other applications to access its functions. The NUTserver may be identified by its application type and installation particulars, additionally it may be assigned a Nut ID as well. The NUTS configuration file for a user device may point to a configuration directory or area in the file system **12604** where it may find an access Nut holding information for each application it may need to know about such as but not limited to remote and/or local NUTservers. For example, the local NUTserver **12620** configuration directory may hold an access Nut containing the Nut ID, type and/or access keys for the remote NUTserver **12640**. Successfully opening such an access Nut may give the local NUTserver **12620** sufficient information to attempt to contact the remote NUTserver **12640** and authenticate with it so that it may open a trusted communication channel and send each other Nuts. In a similar fashion, there may be configuration Nuts for the various applications that the NUTserver may be interacting with. Since access Nuts are Nuts, they may be kept synchronized, replicated and/or propagated amongst peer NUTservers.

[0529] From this explanation of how a NUTserver may function, the iterative design approach of the Nut internals may extend to how applications and data associated to configure and authenticate them may be stored and accessed. Sensitive data may be stored in a Nut as much as possible. The consequences of such a simple statement become far reaching when one considers the built-in functions and features of a Nut and the functions provided by NUTservers. The unauthenticated NUTserver may provide enough functionality to replicate, propagate and/or synchronize Nuts that it may have no inner access to. This may be due to the Gradient Opacity property of a Nut: many Nut parts constituting non-revealing metadata may be saved as clear text and may provide sufficient information for many normal maintenance actions to be performed on a Nut by a NUTserver. Due to the security features which may be built into the Nut, the security of the communication channels for transporting Nuts between applications across the WAN or an intranet may have less significance.

[0530] This method of using access Nuts may solve numerous problems associated with software design, programming and/or use. For example, a bane of software developers may be when they hardcode logins and passwords into their code when in the process of developing their code in order to expedite the entry into a test system such as a test database or test app server. The transition to QA and Production modes of testing and development may be done by adding in the extra authentication procedures into the code right before that stage which may have been minimally tested. Using access Nuts, it may be possible to integrate it into the developing program at the earliest stages and the process may never have to change, only the access Nut might change. A manager may assign and create the appropriate access Nuts for a developer, QA engineer and/or

the production user. These access Nuts may seamlessly integrate into their respective NUTbook collections and may allow them to connect to their application resources without ever signing on separately. The manager may actually maintain ownership of the access Nuts and change it as needed and the NUTservers may eventually replicate and/or synchronize it so that the end users may never have to be bothered with it thereby the project manager may manage the relationships between users and their applications remotely and securely. The effective use of access Nuts may allow any user to configure their systems for single sign on (SSO): SSO on to their local NUTserver and everything else may be automatically authenticated when needed. Hierarchical passwords (forward reference) may allow for added security for certain subsets of access and information.

[0531] FIG. **127** is an alternate embodiment of a NUTserver where the Nut cache **12628** may be replaced by the functionalities of a NoSQL database **12728**. NoSQL databases may be considered by some to be a subset of object oriented databases and many of them may be very efficient in handling Nut-like containers which may be non-table structures. Some NoSQL databases such as CouchBase may offer built-in replication and other features which may be employed by the NUTserver to carry out some of its duties.

NUTS Core Application: MIOR Server

[0532] The Modular I/O Repository or MIOR may be a server based service as depicted in FIG. **128**. This may be a typical embodiment of the MIO systems and methods. A computing device **12810** may have a local MIOR Server running on the device with its own local MIOR Cache **12812**. If a request may not be satisfied by the local MIOR Server, it may reach out to well known Internet based MIOR Servers **12820** or their mirrors **12830**. Their respective caches **12822** and **12832** may be searched for the appropriated MIO modules in the request. If found, it may send it back to the originating MIOR server on the user's computing device. If the requested modules may not be found at the first MIOR Server **12820** on the Internet, the MIOR Server **12820** may reach out to other MIOR Servers on the Internet to look for it. The original request may have a timeout or cascade limit on the number of cascading requests it may make altogether. In some embodiments, the requests may be done asynchronously rather than in a blocking mode if appropriate.

[0533] A closer inspection of this process may be depicted in FIG. **129**. An application **12918** may be running on the local device **12910** which may need to read a Nut file **12908** into its memory. The Nut **12908** may indicate it may need a certain set of read and write modules for its payload from the MIOR Server **12914**. The application may contact its local MIOR Server **12914** and may request the read and write modules for this Nut. The MIOR Server **12914** may look in its local MIOR Cache **12916** to see if it may have those modules. If found, it may reply back to the application with the modules or information of the location of the modules on the local system or network. If not found, the MIOR Server **12914** may reach out across the WAN **12900** or other network of MIOR Servers to request it from a larger MIO repository such as **12920**. MIOR Server **12920** may be a dedicated server optimized to service requests from the Internet for various modules. Once MIOR Server **12922** receives the request from MIOR Server **12914**, it may check its local MIOR cache **12924** for those modules. If found, it may reply back to the MIOR Server **12914** with the modules in the request. If not found, it may contact other MIOR Servers in its peer group in search of these modules. In the meantime, it may send a "Failure to find but continuing search" message back to MIOR Server **12914**. When a remote request comes back with the requested modules, the local MIOR Server **12914** may authenticate it prior to storing it into its local MIOR Cache **12916**. As always, when the time comes for the application **12918** to instantiate and use the module, it too may authenticate the contents using the normal NUTS internal mechanisms.

[0534] FIG. **130** shows a flowchart for fetching MIO modules from a MIOR Server.

[0535] The authentication between the remote MIOR Server and local MIOR Server may be established via session keys or anonymous accounts if so desired. Higher levels of service may

include access to exclusive modules with custom keyed Nuts such as a corporation may wish to use the wide distribution of the MIOR network for their employees using custom developed software but the employees may only open and authenticate the custom modules if they have an access key possibly in an access Nut from the company thus proprietary information may be secured consistently on a relatively open service platform.

[0536] A typical embodiment of the internal organization of a MIOR Cache is shown in FIG. **131**. The Cache **13100** may have a set of indices **13110** that may contain reference to various modules that may be cross referenced and indexed. The structure of the MIOR is not limited to this embodiment but may contain some or all of these organizational structures and techniques. Since every module may be stored in a Nut, the master Nut ID index **13112** may contain some or all the Nut IDs of the modules and their locations in the Cache. The File I/O modules index **13114** may list some or all the modules of that type by description and Nut ID. The File Application modules index **13118** may list some or all the modules of that type by description and Nut ID. The File Display modules index **13120** may list some or all the modules of that type by description and Nut ID. The Collections modules index **13116** may list some or all the modules belonging to a Collection by description and Nut ID. There may be other indices built to allow for the efficient searching of the cache. The Collections groups (forward reference) **13130-13150** are depicted in the diagram to visually show how related modules may be grouped together. The Collections grouping method may play an important role in the operations of the NUTbook.

NUTS Core Application: NUTbrowser/NUTshell

[0537] FIG. **132** shows a diagram of a NUTbrowser application. The NUTbrowser may be essentially a graphical user interface (GUI) that may run on top of the functionalities of the NUTshell command line interface (CLI) application. Commonly known shell programs may be the bash shell, csh, cmd.exe, DOS shell, among others. Commonly known file manager programs may be Windows Explorer, Apple's Finder and others. The user facing behavior of these two programs may be very similar to their commonly known counterparts; however, a difference may be that the NUTbrowser and NUTshell may recognize Nuts and may process them more fully to take advantage of the rich metadata that may be stored in every Nut file. Every Nut file may be identified by two methods: a superficial '*.nut' file name extension and/or the deeper probing of the contents as a Nut. Most file systems may accommodate the file name extension method. The Nut read attempt might be used when attempting to confirm that a *.nut file may be actually a Nut or when introducing new files into the local system from an untrusted source.

[0538] Most popular operating systems such as Mac OS, Windows and/or Linux may use several methods to identify the type of file comprising file name extensions, magic numbers, uniform type identifiers (UTI), file system attributes and/or others. File name extensions may be the most superficial method since when a file name may be changed, the link between its content type and recognition may be severed. Magic numbers and UTI may be compact but limited forms of metadata embedded at the head of the file and may require access to an index of file types to cross reference what form the content may be. This index of file types may exist in the OS, file system, or other external system. File system attributes may be represented as attributes of the file object that may be attached to its instance within the indexing mechanism of a file system. This information may be only effective within the domain of the file system/operating system combination that may record and recognize it. The Nut metadata not only may specify the type of payload but how it may be read, written to, displayed and/or run it. It may specify some or all the versions of the various modules which may be necessary to successfully process the contents. In effect, it may remove some or all dependencies to any and all external reference tables for processing the contents such as but not limited to Windows registry entries and/or Mac OS property lists. This may allow the Nut to self describe and prescribe the necessary components that may be needed to access its contents and may allow the MIOR Server to auto-install any components which it may lack at the time of access.

[0539] The NUTbrowser/NUTshell may read the metadata of any selected Nut and may communicate with the various other NUT Core Applications to attempt to open, display and/or run the proper application on the contents of the Nut by accessing **13232** the MIOR Server **13250**. If the user has properly authenticated into the NUTserver **13240**, the NUTbrowser/NUTshell may have access **13234** to some or all the necessary access Nuts to properly open the Nuts even further. In effect, the NUTbrowser/NUTshell may act no differently from any application that may properly process a Nut.

[0540] Depending on the persistent store that may be used on the local system, the NUTbrowser/NUTshell may allow multiple Nuts of the same filename to exist in the same storage area as long as the Nut IDs may be different. Some storage systems such as databases and object file systems may not be sensitive to filenames. For most cloud based storage systems, the Nut ID method of identification may fit in more natively than the traditional pathname methods.

NUTS Core Application: NUTbook

[0541] A schematic of a NUTbook is shown in FIG. **133**. By now, the typical Nut processing application may look familiar with similar components; it may form the basis of a Nut processing framework more generalized in FIG. **134** and may function similarly to how the NUTbrowser application may work in FIG. **132**. The NUTbook may have requisite interfaces to the NUTserver **13334** and MIOR Server **13332**. It may process MIOR modules **13326-13330** as needed to provide the functionalities provided by them as indicated by **13322** and **13324**. The NUTbook's main function may be to maintain an organized set of caches **13336** called a card catalog. The NUTbook may be an electronic card catalog composed of Collections of data as shown in FIG. **135**. The NUTbook may offer some of the functionalities found in a typical Personal Information Manager. Why is NUTbook a card catalog? Here is a list of various reasons why it might make sense:

[0542] Users may have no easy way to collect, process and organize arbitrary sets of data

[0543] Usually it may be done informally in spreadsheets, text files or simple databases

[0544] There may be no easily accessible general utility to acquire, organize and/or catalog different collections of data in a secure way where the repository may comprise a data file per item in the collection.

[0545] PKI certificates, contact cards, RBK sets, web logins, baseball statistics, VPN logins and credentials, car history, DVD collections, stamp collections, book collections, children's medical records, etc These may be considered as different collections of data or cards.

[0546] A Nut may securely store each type of item in a secure way that may be easy to use and transport.

[0547] Therefore we may store some or all the encryption keys that may be needed to make NUTS work seamlessly into Nuts as well.

[0548] We may access these card collections by indexing their Nut IDs and any optional search index metadata within the NUTbook application.

[0549] NUTservers may be aware of certain important card types and may prioritize their processing in many of its tasks.

[0550] A Nut that may exist in a multi-NUTserver environment may have replication, synchronization, logging, full history, encryption and/or access control by default packaged into a single file per item for easy transportability.

[0551] The NUTbook may contain a Key Cache **13520** which may be in the form of protected or unprotected memory depending on available hardware. The Key Cache may store frequently used access keys with proper attributes attached such as but not limited to the number of times it may be used before expiration, expiration time and/or expiration events. Its main Catalog Cache **13550** may have a master Nut ID index of the Nuts it may be keeping track of. The cache may be composed of different Collections of data such as but not limited to PKI certificates **13562**, contact cards **13564**, NUTserver access cards **13566**, document control cards **13568** and/or any other defined Collections **13570**. These Collections may be stored in memory, in a database, on a file

system or other storage mechanism depending on the configuration of the NUTbook and available hardware. The database and file system storage may be remotely located as long as they may be locally accessible via a network interface. FIG. 136 may be an example of a layout of how the NUTbook Catalog Cache may be organized.

[0552] The data stored in the NUTbook may be an agglomeration of a PIM, password keeper, PKI certificate manager, key ring, address book, note taking app, recipe book, CD collection index, stamp collection index, book collection index, medical records and/or any other data sets that may be expressed as a Collection. The current state of the art for the average user may not offer many choices for them to digitally organize disparate pieces of their lives into a functional digital form. Address book apps may be numerous but seamless, effortless cross compatibility may be lacking. Most sensible users may not store sensitive passwords in their address books and might evaluate and make use of a password keeper app for that specific purpose. Even for just these two simple apps, address book and password keeper, if the user were to consider features such as operating system compatibilities, synchronization, cloud foot prints, backups, web browser integration among others, the decision making matrix may have expanded by several dimensions. And, there may be no guarantee of good integration between the password keeper and the address book. If the user wants to keep track of her family member's medical records, auto servicing records, home maintenance schedules, school logins related to children's classes, pet veterinary records, digital device information and/or other collections of data, they may have to do it in various different formats using different apps for each type of data. A common use of spreadsheets may be to organize such disparate sets of data and may act as a general purpose database for a user. A NUTbook may allow the user to systematically store some or all types of information into a Nut form and may integrate the use of the data into any Nut compliant application. Data that may be properly formed and identified may be made functional by apps that may take advantage of its defined structure. Some or all of the features of the NUTS environment may be available for every Nut in the NUTbook such as but not limited to security, synchronization, replication, backup and/or non-obsolescence.

[0553] Non-obsolescence and/or time compatibility may be an important characteristic of using the MIOR. By using Collections within a NUTbook along with the MIOR, the user may gain several advantages: the data they may produce may be theirs, it may be secure, and they may have a reasonable expectation to be able to access their data indefinitely (or as long as NUTS may be active and supported). The NUTbook also may act as a bridge between the world of the database user and the world of the file user. It may provide the benefits of a database in the form of records stored in a file format. A MIO module for read/write functionality for a particular Collection may be an organized specification set of fields related to capturing the details of the particular collection the user may have in mind but it may not be limited to this model. In some embodiments, the read/write modules may be interfaces to various databases and may provide field mapping and conversion functionality for the calling application. In other embodiments, it may be read/write modules that decipher proprietary binary formats of the payload using licensed keys from a software company. The variety of ways the modules may be used to access data may be very diverse and may have many permutations depending on the goals of the application developer. The basic structure of a specific Collection may be customized by a user with very little programming knowledge starting from simple pre-existing templates. New and useful Collections may be added to their local MIOR for their personal use and shared with others via Nut files. It may also be submitted to an internet MIOR Server for use by anyone after some approval process.

[0554] Now that we may have covered some of the motivations and design goals of the NUTbook, we may focus on how the NUTbook may act as a PKI and eventually may offer SSO level of service for the average user. FIG. 137 outlines the concept of Hierarchical Passwords. In NUTS parlance, passwords may be equivalent to passphrases because a Nut may accept both forms and in place of any password, a user may use hardware tokens, encoded binary keys or any other method

that may provide a secret key. The weed-like proliferation of passwords and their associated differentiators such as but not limited to two factor authentications, login rules, custom password rules, custom web pages and/or hard tokens may quickly spiral out of control and may leave the user in a mental state where they may resort to extremely easy to remember passwords across many web sites thereby the user may be counteracting the efforts of the individual vendors to make their systems more secure for their clients. The preferred solution for NUTS may be to use as few passwords as possible to allow effective SSO access and Hierarchical Passwords may embody this approach. There may be a Main password **13710** which may allow basic authentication into the NUTserver and NUTbook. The Main password may open a Nut containing a key that may be cached in the Key Cache **13520** and may be configured to auto-delete after the end of the session or a predetermined event. This Main key may be sufficient to effectively use most NUTserver and NUTbook functions. There may be second level passwords such as but not limited to Shopping **13714**, Work **13716**, Finance **13718** and/or Communications **13712**. These passwords may only be entered after successfully entering a valid Main password, therefore they may respect a hierarchy of passwords. This second level may allow the user to segregate and isolate different levels of security for different groups of data. Each password in the second level may be configured to have different lifetimes in the Key Cache **13520** so that the user may control their exposure. For example, a user may have an internet bank account login information in a Banks Collections card and may secure it with the Finance key that may have a single use lifetime. Then he may have to enter the Finance password every time he may want to access the bank website by accessing the login and password stored in the Bank card. Within each bank card, the website password may be created randomly to maximize entropy and stored for auto-login use by the NUTbook. There may be more levels added but it depends on the complexity of the user's information and how much she may want to memorize. There may be a Master password **13720** that may bypass some or all the hierarchical passwords. The Master password may be carefully chosen or randomly generated for maximum protection and may be kept in a safe place. Using this Hierarchical Password methodology, a user may just need to carefully choose a set of passwords that may be hard to guess but may be more easily memorized by the user just due to the reduction of the number of passwords she may need to memorize, and this may form the basis of her SSO access.

[0555] FIG. **138** walks through the password entry process for opening a personal document Nut **13806**. This document may be protected only by the Main level key so entering the Main password **13710** to access the Main key to authenticate into the NUTserver **13804** may be sufficient to open the Nut holding the personal document **13806**. In FIG. **139**, the Master password **13720** route may be exactly the same as Main password route.

[0556] FIG. **140** shows how a Work document protected by a second level Work password **13716** may be opened. The Main password **13710** may be supplied to access the Main key, then the Work password may be entered to gain access to the Work level key **14008** which may unlock the Work Document Nut **14010**. In FIG. **141**, the Master password **13720** unlocking route may stay the same as in FIG. **139**, it may still be a single step access, thus Master passwords may be created in a more secure fashion.

[0557] FIG. **142** shows a more detailed diagram of the NUTbook Key Cache **13520**. It may have a section partitioned for keys associated with the NUTservers **14210** and it may have a section partitioned for use on its various Card Collections **14230**.

[0558] FIG. **143** shows a process flowchart of how a NUTbook may view a Catalog Card.

[0559] Retained ownership is a concept which concerns the mingling of Nuts of different owners. Suppose Alice gets a new job with Acme Company and they both may use NUTS based applications to manage the minutiae of organizing their respective contacts and/or digital keys. Additionally, Acme may use Nuts to control access Nuts and carefully lock down corporate documents by department and/or by employee access level. When Alice gets hired, Acme's HR department may issue Alice a general corporate access Nut: it may be the access Nut that may

allow Alice to look up information such as internal corporate contact lists, client lists and/or various corporate documents. Acme's NUTS systems may have been customized and/or configured to give access to sensitive documents which may be stored in Nuts by wrapping a copy of the payload into a wrapping Nut locked by the employee's specific access Nut and a corporate master key. The ownership (RAT) of these corporate Nuts may always be Acme. Similarly, Alice's personal Nuts may always have her as the RAT. The ability to clearly define the owner in a cryptographic way may allow each Nut to be treated appropriately by each respective owner within their NUTS environments. This retained ownership characteristic of Nuts may allow Alice to comingle her Nuts with Acme's Nuts on any device she may use and maintain control over them. The same may apply to Acme's Nuts on Alice's devices. Both Alice and Acme may set the lifetimes of their respective access Nuts to be a relatively short period. For example, the lifetime may be set at 60 days on Nuts stored on foreign systems. Therefore, every 60 days, the keys may be renewed by each owner of the Nuts owned by them or they may be automatically deleted by the foreign NUTservers managing them. Deletions may occur forcibly if the appropriate NUTservers may be sent deletion commands in an appropriate access Nut and it may be encoded to systematically delete some or all affected Nuts of the owner. Thereby, each party may have the ability to maintain control over their Nuts in foreign systems either directly or indirectly. Thus if Alice leaves for a new job, she may know that her personal contact information that she may have left a copy of on her corporate desktop may automatically be deleted in 60 days or less. The same may apply for any Acme owned Nuts left on Alice's personal devices: if there is no renewed access Nut, no more associated Nuts on the system. This type of mingling of Nuts may be meant to solve the age old problem of juggling two or more separate contact lists and different sets of security measures for taking work home. Now Alice may always use her personal NUTbook as her main source of contacts in her personal and professional life and she may be reasonably be assured that it may be secure.

[0560] In another embodiment, a NUTbook contact card may carry references to or embed foreign Nuts that contain personal information for an acquaintance. The foreign Nut from Bob may not be owned by Alice but by Bob. Bob may send Alice a pre-packaged, limited detailed, contact Nut about himself and may maintain its ownership in Alice's NUTS environment. Alice's NUTbook entry for Bob may embed this Nut into her contact entry for Bob either directly or by reference. Whenever Bob changes some or all information about himself such as a new mailing address, a new work address, phone numbers or other affected information, he may send an update to his pre-packaged contact Nut to Alice by any available means and once Alice's NUTserver recognizes it, it may automatically update the appropriate embedded foreign Nut in the card for Bob in Alice's NUTbook. Then, Alice's NUTbook may run the contact application to process the updated card which may lead to the update in Alice's card for Bob. This last step may assure that Alice's card entry for Bob may never lose its past history on Bob's information and she may track down the various historical changes to Bob's information when she so may desire. Some or all of these steps may occur automatically without intervention on well established, trusted RBK relationships. This may mean some or all of Alice's trusted RBK relationships may have updated contact information with few or no manual interventions which may lead to a big savings in time and effort on Alice and each of her friends. If Alice has 99 RBK contacts and 50 updates may occur, then only 50 changes may have to be initiated by the affected people themselves and the rest may be handled automatically by each affected person's NUTservers. In a traditional address book setting, 50 updates may become 50 updates by the affected individual, 50 notifications to 99 friends informing them of the change, each of the 99 friends making up to 50 updates to their own address books along with some level of transcription errors within the nearly 10,000 events that the 50 updates may spawn let alone the collective time spent by the 100 people that may be involved. This embodiment may be solved alternatively by having a centralized service but such services may provide limited privacy, access, ownership and/or control. The NUTS solution may emphasize

decentralization as much as possible while attempting to maintain consistently high levels of privacy, history, audit trails and/or ownership.

NUTS Based Services

[0561] NUTS based services may extend Nuts usage to a wider network such as the internet so that Nuts may be utilized between multiple remote parties. The table in FIG. **144** lists examples of the various web based services that NUTS may support and offer and FIG. **145** shows a simplified network layout for these services. Some or all services may offer multi-tiered service packages with the lowest levels being offered for free with constraints. Payments for higher tiered packages may be made directly or anonymously via separately purchased service credit vouchers. Some or all of the services may be used anonymously to varying degrees.

NUTS Based Services: NUTmail

[0562] The NUTmail server depicted in FIG. **146** shows a web based email service that passes some or all its messages via Nuts among its registered users. Furthermore, it may support auto-registrations, anonymous registrations, anonymous channels and/or RBK based communications. The server may interact with NUTbook and/or NUTserver apps. The NUTmail may have a client component that may run on a user's device to enable them to manage, edit, display, compose, send and/or receive emails.

[0563] FIG. **147** shows the process for establishing an anonymous registration on a NUTmail server in an automated manner. A user may contact **14700** the server with a pre-packaged Nut which may contain a preferred pre-existing contact method such as but not limited to an email address, text capable telephone number and/or web browser. The server may accept the request **14702** and may send a request to the user using the preferred contact method **14704**. The user may enter the required information from the request and the server may create a randomly generated login ID and password which may employ the maximum entropy of the cipher method in **14706**. The server also may generate a RBK pair with the user which may be employed in some or all communications between the user and the server/administrator. The user may store the login credentials and RBK pair in her NUTbook in the card for her own contact information **14708**. Thus the user may have anonymously registered with the NUTmail server in a predominantly automated way **14710**.

[0564] The login ID and RBK which may have been created during the registration process may be only used by the user to communicate to the NUTmail server; in a way, it may be considered a private channel between the user and server. When a user wants to communicate with another person who may also use NUTmail, a communication channel may need to be established with that person on the NUTmail server as depicted in FIG. **148**. A communication channel may comprise a pair of randomly generated email aliases that may be attached to each user's registered accounts as aliases. The NUTmail server may not keep track of these alias pairs once the communication channel may have been established and verified in order to better preserve the anonymity of relationships. These aliases may be similar in function to the RBK in that it may only be used by the two participants in the channel. The random nature of the alias generation may give away no hints to the identities of the participants during email transit across the internet. The email contents themselves may be encased in a Nut protected by RBK methods further protecting the payload. This may provide two separate layers of relationship based methods and obfuscations that may minimize some or all unwanted SPAM and/or third party email sniffing. Once the communication channel may be properly established then the exchange of emails may be fairly standard as shown in FIG. **149**.

[0565] The security rationale behind a NUTmail server may be summarized as follows:

[0566] Anonymous registrations may mean a compromised server may reveal very little about the registered users and/or their email contents.

[0567] The encapsulation of emails within RBK encrypted Nuts may provide another independent layer of content security. Hacked servers may only reveal messages secured by Nuts.

[0568] NUTmail communication channels using alias pairs may obfuscate email metadata.

[0569] The server may not store alias pairing data permanently, only long enough for the channel to be verified.

[0570] The server may store email messages for a very short period of time. It may be configurable by the user but the default may be that messages may be expunged after it may receive information from the user's NUTmail client or NUTserver that at least 2 copies may exist outside the server or after a pre-configured duration.

[0571] A short history of emails may allow the server to have very small long term data storage requirements.

[0572] Randomly generated logins, aliases, passwords and/or RBKs may make full use of available data entropy which may lead to added security.

[0573] It may not be easy to use the NUTmail server without the integrated facilitation of a NUTbook although it may be possible. The login ID, password and/or aliases may be generated using maximum entropy methods and may look like a jumble of a long string of random characters. There may be a 1:1 correspondence between a relationship and an alias pair so the number of aliases that a user may have to keep track of may get numerous very quickly. A benefit of this communication methodology may be that data generated by the participants may be useless in and of itself and some meaning may only be extracted via targeted data surveillance and/or sophisticated reconstruction techniques.

[0574] The data storage requirements of a NUTmail server may be different from an ordinary email server: it may use much less space per user on an ongoing basis. When a user's NUTserver or NUTmail client may indicate that at least two copies of an email may exist outside of the NUTmail server, the NUTmail server may delete that email Nut permanently. This type of simple rule may allow each participant in a channel to establish two or more copies of their communicates at a minimum each. The NUTmail server may leverage the NUTservers of each registered client to offload as much long term storage as possible thereby reducing its own ongoing storage requirements per user. The NUTmail server may only have new email messages for registered users since each user may have downloaded and replicated previous emails on their own NUTmail client/NUTserver systems.

NUTS Based Services: NUTchat

[0575] NUTchat may be an anonymous chat service based on Nuts. It may offer the following chat features:

[0576] It may support anonymous registration, pairwise random aliases and/or RBKs

[0577] It may be able to provide local NUTchat hub phone numbers for anonymity [0578] It may

support simultaneous cell-phone & non-cell phone chats [0579] It may support SMS/MMS and

internet based chat sessions simultaneously [0580] It may support similar history features as

NUTmail server

[0581] Chat history may be saved within each contact entry storage, or it may be stored in a Nut and it may be referenced by the target contact entry rather than by just phone numbers or chat addresses.

[0582] Chat history may be permanently saved for personal use without the need of the NUTchat service.

[0583] NUTchat may be a specialized service for chat messages that may be contained in a Nut.

[0584] Randomly generated logins, aliases, passwords and/or RBKs may make full use of available data entropy which may lead to added security.

[0585] It may multiplex communication routes to ensure delivery of messages and show virtual chat sessions.

[0586] An example of a network diagram is shown for a NUTchat server in FIG. 150. Its registration procedures may be similar to methods employed by NUTmail servers and may offer some or all the anonymous features for its users. There may be a Nut based NUTchat client running on user devices and the basic data flow configuration is shown for chat sessions between three

participants in FIG. 151. This may be a standard text message passing topology with the NUTchat server acting as the coordinator in the middle 15100. Because NUTchat may be based on Nuts, the entire chat history of a session may be saved in a Nut and therefore may take advantage of the NUTserver replication/propagation/synchronization features automatically if properly configured. [0587] The NUTserver may be configured to prioritize NUTchat Nuts so that they may be handled in a more timely manner due to the nature of the real-time interactivity in a chat session. A close look at FIG. 151 shows that the same chat Nuts exist in multiple locations; it shows that a chat topology may be equivalent to a streamlined synchronization of data states in a plurality of physical locations. FIG. 152 is an example of the data flows of a process that may replicate NUTchat sessions using a user's NUTservers. Since each chat participant may store some or all of the chat session history in a Nut 15122-15126, the NUTserver 15238 may propagate changes to those Nuts across its peer NUTservers such as 15242. By properly synchronizing data in this methodical manner, when the user brings up a NUTchat client 15260 on his device #4 15240, he may see the same session history as he may have left it on device #2 and in no manner was the NUTchat server involved in bringing his device #4 up to date. When a chat session is initiated, and when the examination of chat Nuts on either side of the channel by the respective NUTchat clients may determine it to be unsynchronized, then a forced synchronization procedure may be automatically initiated to bring the session updated to the latest version (note the classification of chat history may be viewed basically as a newer state of the payload aka Nut history). For example, Alice may have a long standing anonymous NUTchat channel with Bob but somehow she may have lost or deleted her chat Nut storing this session history on her smart phone. When she resumes this NUTchat session with Bob and may make contact though the NUTchat server, the server may receive version numbers of the session from both Alice and Bob and it may show that Bob may have a later version of the session than Alice. At that point, a copy of Bob's chat Nut may be requested automatically and may be sent over to Alice via the NUTchat server and Alice's NUTchat client may accept Bob's session history as its own and the chat session may continue with a common view of its history and thereby its context. There may be very little storage being used in this scenario by the NUTchat server and some or all the session information may be stored by the end users under their control. Once chat session versions may have been synchronized, chat messages sent to each other thereafter may be contained in Nuts only holding the new chat message in the session rather than the entire history and the NUTchat clients on each end may be responsible for updating its cumulative chat session respectively thereby it may reduce the size of data transfers in an ongoing chat session.

[0588] Furthermore, Alice's NUTbook may make references in her contact entry for Bob to reference or point to chat Nuts and email Nuts so that some or all relevant historical communications with Bob may be indexed under Bob's information which may give rise to the systematic collation of context in a relationship stored under Alice's control.

[0589] NUTchat clients may engage in a Dialogue which may involve path agnostic chat sessions for reliability, redundancy and/or obfuscation. FIG. 153 shows a typical data flow pattern for three separate chat sessions between Bob and Alice which may be using up to three different chat services and/or chat IDs. Sometimes, this type of separation and segregation may be desired and convenient for the parties that may be involved. At other times, it may be forced upon the user by choices made by the other participant: for example, Bob may only want an account on the chat service B so Alice may be forced to create a login on service B to chat with Bob. However, to the extent that a NUTchat client may interface with other chat services, it may allow multiple separate chat sessions between the same two persons to be agglomerated into a path agnostic chat session as shown in FIG. 154 which may be called a Dialogue. Chat Nuts may be the basic medium of the messages so that some or all may have version numbers and a copy of the Nut may be sent on some or all three chat session paths simultaneously. Whichever chat Nut that may get to the other NUTchat client first may be processed and the others ignored (or may be merged by the NUTserver

Nut merge and then discarded). Sometimes due to the nature of the transport limitations, Chat Nuts maybe converted into concise, secured text messages appropriate for the transport platform. In this method, the conversation may be preserved over multiple pathways and only the most current version may be ever shown to each participant and the process may not rely on the storage and/or organizing functionality of the individual chat service providers, only their transport mechanisms. The redundant pathways may minimize or virtually eliminate transport failures for the Dialogue. The history that each transport service may store may be useless because it may be protected by a Nut on a per message basis therefore the contents may be opaque. The transport mechanisms may be any channel that may allow Nuts to be passed such as but not limited to email servers, ftp servers, networked file systems, point-to-point connections, Wifi protocols, Bluetooth protocols and/or any other digital transmission method. The synchronization properties of a Nut may allow for chat sessions to be engaged solely by using a shareable Nut configured to have at least two writers and common method for the users to access the Nut. This embodiment may show how relatively simple it may be to disintermediate the functionality of chat systems while protecting the user's data independently of the service and strengthening the overall reliability of the transmission mechanisms by the user.

NUTS Based Services: NUTcloud

[0590] The NUTcloud may be an internet based storage server available to any NUTS user as depicted in FIG. 155. The NUTcloud may support anonymous registration, pairwise random aliases and/or RBKs. It may seamlessly integrate with personal NUTservers to extend the reach and availability of a personal NUTS network. The NUTcloud may store Nuts and its storage and bandwidth limits may be affected by service tier levels and user configurable policies. NUTcloud accounts may interoperate with other NUTS based services to supply more permanent and/or accessible storage: i.e. it may backup NUTmail and/or NUTchat messages.

[0591] At the base level of service, it may offer a sufficient level of storage and bandwidth for general personal use. Its main purpose may be to facilitate the accessing of data stored in Nuts from any access point on the internet. It may seamlessly integrate with NUTservers to synchronize some or all of Alice's data at home and on the road.

[0592] The NUTcloud in conjunction with personal NUTserver may offer the same or better level of synchronization as any internet based centrally managed cloud service; however, unlike the popular freely available cloud syncing services, NUTcloud may offer complete anonymity, user controlled privacy, full history, full audit trail and/or secured data ownership.

NUTS Based Services: NUTnet

[0593] The NUTnet may be a Nut based webserver available to a NUTS user as depicted in FIG. 156. The NUTnet may support anonymous registration, pairwise random aliases and/or RBKs. The NUTnet may store Nuts and its storage and bandwidth limits may be affected by service tier levels and user configurable policy settings. NUTnet accounts may interoperate with other NUTS based services to access more permanent and/or accessible storage: for example, it may fetch Nuts from NUTcloud and/or NUTservers.

[0594] Sharing webpage content stored in Nuts may allow users to control who may view the content and it may be done on a cryptographic level. A person may have an RBK pair with the content owner in order to view the posted pages. One may say that this may be an anti-social social network, private social network and/or authenticated social network. None of the content may be mined by the NUTnet server or other unauthorized third party because it may not have any of the keys for the content. As long as the content may be stored and secured in Nuts, the owner may retain control over it. The owner may also view some or all history associated with her postings in her local Nut storage if it may be configured to replicate and synchronize the Nuts locally as well. There may be times when a person feels that sharing pictures and video amongst close friends and family may be a private matter and that no third party may have the right to own a copy of it for their use without knowledge and/or permission of the originator. NUTnet may be created for those

situations requiring privacy within a group of users.

[0595] Professional photographers may set up private webpages for potential clients to view copyrighted photographs with an immense amount of details and control over who may be issued the keys and for how long. The webpage Nuts may log some or all activity on the photographs to create an audit trail for the photographer. Project managers may set up private webpages for coordinating activity amongst members of the project. From a security perspective, the registration process may be unnecessary due to the access controls built into the Nut but it may serve as an organizing and compartmentalization function at the NUTnet server.

NUTS Based Services: NUThub

[0596] Currently, there may be no universally accepted standard on how the Internet of Things (IoT) may communicate and/or function. IoT may be a growing area of hardware products that may have built-in networking capability and may allow users to control and monitor the functions of the product remotely from various personal computing devices. Many IoT products may send a constant stream of data from their sensors back to the manufacturing vendor for them to collect and analyze, sometimes, unbeknownst to the user-owner of the product. The operational mode of some or all of these IoT devices may raise many invasion of privacy issues based on their data collection range and methods since the products may be intended for the most private areas of a person's home. The IoT frameworks to gain some use may be supplied by the IoT hardware vendors for their family of products. NUThub may be a packet forwarding service to facilitate the handling of Nuts based messages which may be created by NUTS compatible IoT-type devices called the Internet of Nuts (IoN). As depicted in the network diagram on FIG. 157, IoN may be a NUTS based standard for communicating securely and privately with your IoN compatible devices at home. The lowest tier of service on NUThub may be available to anyone that may have a registered account with any NUTS based service. The account may be anonymous. NUThub may work with Nuts and it may queue a certain amount of messages. NUThub may interface seamlessly with NUTcloud and/or NUTserver to access additional storage.

[0597] The NUThub topology may be configured to work in several ways. The direct topology is shown in FIG. 158 where every IoN device in the user's home may be making independent connections to the IoN vendor servers 15804, the NUThub 15802 and/or user control devices 15806, 15822 and 15824. This topology may allow the vendors to have more direct access to the devices in your home and the user may filter outgoing Nut packets only to the extent of the filtering capabilities of each device: this may be the predominant method of communications used by IoT devices today.

[0598] The preferred NUThub topology may be the indirect one as depicted in FIG. 159. Some or all IoN devices may communicate through a designated NUTserver hub 15930 before leaving the LAN 15920 and then traversing the NUThub 15902. This topology may allow for the fine tuning of filtering rules on IoN messages leaving Alice's home based on her comfort level. The NUTserver hub device 15930 may comprise a desktop PC, a special purpose appliance or even be part of the WiFi router 15920. If the designated NUTserver hub 15930 is off or unavailable, no IoN device may communicate with the outside world.

[0599] The configuration of a NUTserver hub is shown in FIG. 160. Within the familiar NUTserver 15930, there may be a component called the NUThub/IoN Interface 16032. This module may be responsible for communicating with the NUThub 15902, IoN devices 15922 and/or other NUTserver hubs 16052. The interface module 16032 may log, queue, forward, relay, process and/or filter IoN Nut messages from both the IoN appliances and the IoN control devices.

[0600] A closer view of the NUThub/IoN Interface is shown by FIG. 161. The interface 16032 may comprise some or all of these seven functions or other additional functions. The IoN device index 16112 may keep track of some or all the IoN devices registered by the user. IoN Device Authentication 16114 may authenticate and may cipher messages to and from IoN devices. The interface may keep track of the user's Message Filters and Rules 16116. The Message Logger

16118 may log some or all IoN messages to permanent storage. The Message Queue **16120** may temporarily store undeliverable messages. The Device Key Cache **16122** may store some or all the access keys for authenticating and ciphering IoN messages and it may be embodied within protected memory hardware if available. The Remote Control Interface **16124** may be the module that may allow for IoN device specific functions to be activated remotely.

[0601] A closer view of the NUThub/NUTserver/IoT interface on any IoN device is shown by FIG. **162**. The interface **16210** may comprise some or all these seven functions or other additional functions. The Nuts index **16212** may keep track of some or all the Nuts stored on the device relevant to administering and managing IoN devices. The Authentication module **16214** may authenticate and may cipher messages to and/or from the device to the vendor, NUThub and/or NUTserver hub. The interface may keep track of the user's Message Filters and Rules **16216**. The Message Logger **16218** may log some or all IoN messages to permanent storage. The Message Queue **16220** may temporarily store undeliverable messages. The Device Key Cache **16222** may store some or all the access keys for authenticating and ciphering IoN messages and it may be embodied within protected memory hardware if available. The Remote Control Interface **16224** may be the module that may allow for IoN device specific functions to be activated remotely. The IoN device may have a limited set of functionality for custom filtering due to its hardware limitations. It may also have storage limitations which may limit the amount of messages it may log and queue. Therefore, if history and audit trails may be important, the user may be strongly advised to use an indirect IoN topology as depicted in FIG. **159** which may allow him to access the enhanced functionalities that may be provided by a NUTserver hub. This interface **15922** is not limited to IoN/IoT specific devices, any computing device may have a similar interface if a developer may create one for it and follows the operational modes of an IoN device; additionally, any device that may have a version of NUTserver running on it may be capable of acting as an IoN device itself.

[0602] When Alice buys her new IoN device, she may need to add it to her network and configure it. The flowchart on FIG. **163** shows the steps that Alice may take to properly register her new IoN device to her NUTS based network. The method of configuring the IoN device may be to establish a RBK relationship with it through Alice's NUTbook. Steps **16302** and **16304** may allow the NUTserver hub to relay device specific information to her NUTbook and in turn the NUTbook may create a IoN/IoT device catalog card, fill in the model, version and/or serial numbers, generate RBK pairs and send it back to the IoN device via the NUTserver hub. The act of creating a catalog card for the IoN device may create a Nut which may create a Nut ID for that Nut; therefore, the IoN device may hereafter be imprinted with the Nut ID of its catalog card Nut. This step may be akin to picking an IP address for a new device on your home network but the potential advantages of using a Nut ID may be far reaching. The assigned Nut ID for the IoN device also may serve as a permanent way to reference the device irrespective of its actual IP address and/or location. The IoN device may be reset to factory conditions so that a new Nut ID may be imprinted on it by a new or same owner.

[0603] Once an IoN catalog card is saved in Alice's NUTbook, the configuration process may proceed to step **16306** and it may check if there may be MIO components necessary to decipher the device's configuration data, display it and/or set it. Once the proper settings have been made on the configuration screens, Alice may save the setting into her IoN catalog card for the device and may submit it to the NUTserver hub interface to be sent to the IoN device **16314**. The device may receive the configuration Nut, may authenticate it, may decode it, may validate it then may apply the changes to its internal system. Once complete, it may send back a Nut to the NUTserver hub indicating its status. Alice may be monitoring this device and she may see messages from it automatically.

[0604] IoN devices may operate in a mode where some or all the messages may be Nuts and therefore may be afforded the same level of privacy and control of Nuts by default. Since Nuts may

utilize MIO components, the software configurations, firmware and/or software updates to the devices may be submitted through the same MIOR mechanisms and the potential for being outdated may be low. The NUThub may be configured to may be assure the user that everything may be monitored, logged and/or controlled by her if necessary and that some or all outgoing information that may be collected by the IoN device may be filtered to honor the user's privacy preferences. In this embodiment, the NUTS core philosophy may extend into physical devices so that a device you own may be under your control at some or all times and some or all the data it may generate may be yours as well. The power of MIO and its functionalities may be apparent in this scenario because any data format with a proper MIO component may be inspected by the user unlike many proprietary protocols.

[0605] This may bring us to an important module called the Remote Control Interface shown in **16124** and **16224**. This may be the method by which a user or vendor may converse with a IoN/IoT device and may have it act on commands remotely which we refer to as Command Nuts. RBK authenticated command Nuts may be processed and the device owner (RAT) may execute any command available on it. This authentication requirement may allow a user to fully control its relationship with the vendor by adjusting the vendor's access rights. A user may allow the device vendor to have full access to it, a subset of it and/or no access. This may prevent unauthorized access to Alice's home network using IoN/IoT devices as entry points: each IoN/IoT access point may be now hardened by NUTS based security. As we may have mentioned the extensive nature of how Nuts may be propagated and may be sent along the intranet and/or internet, basically an IoN command Nut may be sent from anywhere there may be a proper route to the IoN device. The flowchart in FIG. **164** shows how the Remote Control Interface may process command Nuts.

[0606] The nature of the NUThub and its Remote Control Interface may give rise to Alice's ability to completely control some or all her NUTS compatible devices from anywhere there may be connectivity. It may present a secure protocol by which custom messages may be sent while being controlled by Alice's NUTbook relationships represented by RBK pairs. It may present a centralized view for Alice for all her IoN devices but it may be installed, configured and/or maintained in a decentralized manner. If Alice controls her Nuts, she may control some or all her devices. This may be another reason that when Alice may decide to use the SSO capability of NUTS she should choose her passphrases very carefully or use a hardware based key. In such embodiments, the vendor's role may be curtailed to that of the hardware manufacturer and not that of an uninvited remote administrator of a personal device that belongs to Alice and may be situated in a private area of Alice's home. The security of the NUTS environment may present a more unified, hardened and/or user controllable barrier than current IoT protocols which may be biased towards the manufacturer's (developer's) preferences and/or advantages.

NUTS Based Services: NUTS Certification Server

[0607] Since the integrity of the NUTserver processes and protocols may be essential to trusting that it may behave as expected, there may be a NUTS Certification Server (NCS) to validate NUTserver installations on an ongoing basis. As pictured in FIG. **165**, NCS may be available to any NUTS user and may support anonymous registration, pairwise random aliases and/or RBKs. It may have a tiered level of service with the highest level being official certification by the NCS company as being "NUTS Certified". The main functions of the NCS may be to monitor NUTservers for proper deletion of Nuts and/or detect unauthorized tampering with NUTserver protocols, behaviors and/or processes. Since clever programmers may identify probes and may circumvent it, the architecture of how anonymous registrations work may allow NCS probes into NUTservers to be virtually undetectable. It may be a voluntary level of service that a user may choose to activate on their NUTservers. There may be automated procedures initiated by the NCS to inject a target NUTserver with test Nuts and detect whether certain actions may have been applied to them according to NUTserver protocols. At higher levels of service, active participation by testers may allow even more thorough assessments about the state of a remote NUTserver.

[0608] Vendors may subscribe to NUTS Certification level testing to constantly maintain a level of NUTserver compliance that may be made known to their clientele and assure them that their Nuts may be being handled accordingly. The testing process may also highlight any unauthorized modifications to the client's NUTS environments unbeknownst to the client. From the client side, any vendor who may be using NUTS systems and methodologies but may not be "NUTS Certified" may require more inquiries as to their policies for handling Nuts. Users may configure their NUTservers and/or NUTbooks to interface with a lookup table on publicly available NCS databases to assess their certification status or lack thereof prior to engaging with an online vendor. [0609] In FIG. **166** the NCS **16520** may perform functions that may allow it to assess the behavior of remote vendor NUTservers (or personal NUTservers) **16620-16624**. The expiration integrity probing **16602** may be a method where Nuts may be injected **16604** into the system and may be probed by the Remote Control Interface **16610** for existence on that system after the expiration time. For example, if expired Nuts are found on the remote NUTserver, the NUTserver may be out of compliance and may not be "NUTS Certified". Long duration injection tests **16608** may test NUTservers for a longer amount of time and on an ongoing basis. Results analysis and certification **16606** may assess the adherence of the remote NUTservers to the various injection tests and may grade the NUTserver installation. Checking the versions of installed NUTservers and the patch versions may be integral to making sure that NUTservers may be updated and in compliance. A long outdated version may indicate lax maintenance of NUTS security protocols and/or unauthorized custom modifications may have been made therefore adoptions may be slower. The testing also may include but is not limited to checking various sensitive binary code segments' hash signatures and/or injecting from anonymous internet addresses. Anonymously registering a NUTserver to the NCS service may assure that RBKs may be set for deeper testing in a more secure way.

[0610] NCS may not guarantee that a NUTserver may have not been compromised since with enough knowledge and resources any person or group may eventually circumvent the testing by the NCS. On-site inspections may result in higher levels of NUTS Certification. For the average user, it may be good policy to not engage with any commercial NUTserver that may not have been certified at the highest levels. For engaging with personal NUTservers, a base level of automatic free testing from an NCS may be a minimal requirement prior to engaging with it.

NUTS Based Networking for WiFi/Ethernet Router

[0611] FIG. **167** shows an embodiment of a network layout for a personal NUTS based WiFi/ethernet router **16710**. The router may operate using the normal protocols that may be involved in WiFi communications as well as use Nuts based messaging as an alternate protocol. A NUTS WiFi router may be installed and configured like any IoN device whereby the owner establishes an RBK relationship with it and may store the information into her IoN catalog card via her NUTbook. During the configuration process, since the user may have most of her devices represented by catalog card entries, she may be able to register some or all the devices she may want to allow access to on the router by Nut IDs. Originating Nut messages may contain the sending device's Nut ID and may therefore be properly vetted against the registration list for access. The router may then be instructed to establish relationships between various devices and itself, thus it may allow secure communications for the contents of the Nut messages. The flowchart for processing messages at the NUTS WiFi router is shown in FIG. **168**. Some or all messages that may pass through the router by registered devices may be authenticated. Step **16818** shows an interesting feature that may be available on NUTS based routers. An unregistered device may contact the router for access not using RBKs. When this occurs it may look up the owner specified configuration settings for bandwidth allocations and limitations for different categories of WiFi accesses: registered, IoT and/or guest. Registered devices may be set with no limitations on the type of usage and bandwidth requested. IoT/IoN devices may have their own category and may require the same level of authentication as registered devices but may be separately managed as a

group. The table on FIG. **169** shows the defined categories and the type of access they may have through the router. Guest devices may be afforded access using normal protocols but with constraints. A sample configuration for category based attribute limits is shown in FIG. **170**. An owner may specify per device limits such as but not limited to expiry, bandwidth, aggregate bandwidth, maximum connections of category type, destinations and/or message modes. In this way, guest devices may have internet access through an unknown NUTS WiFi router within certain limits while the authenticated NUTS intranet may be protected by NUTS level secure methods. This methodology effectively may create separately manageable channel categories within the framework of WiFi communications.

[0612] Some or all the registered devices of the user may be now independent of internally assigned IP addresses for identification but rather by Nut IDs in a catalog card. This may be a property of NUTS to make data and hardware more tangible and functional across some or all networks in a more universal manner. The router may keep track of dynamic IP address assignments mapped against Nut IDs of registered devices. In future iterations and other embodiments, hardware manufactures may allow Nut IDs to be used alongside IP addresses and/or MAC addresses to access Ethernet interfaces on various devices. Device identifying Nut IDs may be thought of as the equivalent of assigning a system name to an OS installation on a PC but it may be systematic and practically unique therefore changing or adding an Ethernet card to a system may present new IP addresses and/or MAC addresses but it may not change the Nut ID associated with the device.

[0613] Parental oversight of their children's internet accesses may be monitored and limited at the router level using a NUTS based WiFi router rather than or in addition to at the device and user levels. The message Nut that may envelope the registered device traffic may include user identification information which may be used to further filter the traffic by parental preferences.

Application Wrapping with Nuts

[0614] The advent and development of cloud services, app stores and/or its associated apps may have allowed some form of modularization and/or transferability of apps across diverse devices. However, this may not be the case with desktop or laptop computers. Most applications that may run on them may require manual installations and/or maintenance. This may also be true for well maintained institutional environments where a mix of pre-selected app packages may be rolled up into a custom install pack by the system administrators for the ease of machine setups. Or, they may create cloned pre-installed applications on disks that may be swapped into computers. For a running environment, it may be very difficult and hard for individuals and/or administrators to monitor and authorize every program that might be installed on a particular device. Very strict account rules may lead to decreased productivity for the user or increased personnel requirements for the systems department.

[0615] An application wrapped in a well constructed Nut may solve many of these issues. Local operating systems may be modified to only allow Nut wrapped applications to run. The implications may be many. This may prevent some or all unauthorized installations and executions of unapproved and unvetted applications. Policies may be enforced by centralized administration of access keys in a managed institutional environment. Viral infection vectors that may involve the execution of a naked binary may be drastically reduced. The NUTserver replication and synchronization features may allow easy propagation of newer versions of installed software across some or all devices. Properly wrapped Nuts may be remotely commanded to self install using the Remote Control Interface upon successful synchronization. Device environment backups and duplication may be automated using NUTservers as depicted in FIG. **171**. Computing device **17100** may store a backup of Nuts for a device that may have failed. Upon getting a new device **17140** ready for installation, the application that may need to be installed properly may be the NUTserver **17144** and its access keys. Then a duplication command from either computing devices with the correct keys may initiate the copying of some or all relevant Nuts from Device **1** to Device **2** and

then may perform the necessary installations of some or all Nut wrapped applications.

[0616] Superficially, this method may not seem that different from cloning hard drives or having a well procured install script but there may be some significant differences. The Nut wrapped application may be a specification of the application and not the specific binary itself. The binary may be stored in the institutional MIOR and then the MIO mechanisms may take over during the opening process of the Nut wrapped application specification to fetch the correct version of the application for the current operating system of the device which may or may not be the same as the original device it may be replacing. This use of the MIOR may be a way to control application versions within a computing environment comprising heterogeneous operating systems and/or hardware. The use of NUTS technology actually may allow some or all of these processes to occur from anywhere in the internet therefore new machines may be installed and maintained on behalf of an institution in a remote manner.

[0617] An example of this may be a salesperson on a weeklong road trip may have his laptop stolen which may have contained 20 custom presentations and confidential client reports he may have wanted to use in client meetings. Assuming the company was utilizing NUTS, the salesperson may go to the nearest computer store and buy a replacement laptop under the guidance of a system administrator. He then may install a standard NUTserver downloaded from the internet on that laptop. The administrator may send him a specially encoded access/install Nut called a Genesis Nut via email and the salesman may download this Genesis Nut on to his new laptop from a web browser based corporate email page. The administrator may call him and tell the salesman the secret passphrase that may unlock the Genesis Nut. Once unlocked using the local NUTserver/NUTbrowser, the Genesis Nut may initiate some or all the processes necessary across the internet to duplicate the applications and data from the salesman's lost laptop from its most recent synchronizations with the corporate servers. In a matter of a few minutes to a few hours depending on the amount of data in the backups, the salesman may be fully operational with some or all his contacts, apps and/or data Nuts reinstalled on his new laptop and it may be done on different brands of laptops and different operation systems as long as the corporate MIOR may be properly seeded and maintained. Parallel to this duplication effort, the administrator may send self delete commands to the stolen laptop for some or all company owned Nuts stored on it just in case the thief starts up the laptop with a connection to the internet. This may be a precautionary measure since the Nuts on the laptop may be already individually secured with corporate Nut expiration policies.

[0618] In another embodiment, a hardware embedded NUTserver may be integrated into an uninitialized computing device that may have a connection to a network harboring accessible source NUTservers and MIOR servers. The Genesis Nut may be loaded onto the device and accessed which may initiate the processes which may lead to the complete installation of a computing environment onto this uninitialized computing device including the OS, drivers, applications, application configuration data and/or user data. The choice of OS may be left up to the user upon examination of the device and contents of the accessible MIOR caches. Applications may be installed incrementally as the user accesses different Nuts or all at one time by querying the source NUTserver for a complete list of needed applications for accessing the user's Nuts.

Event Processing Service (EPS)

[0619] The NUThub may allow Nut based communications with IoN/IoT devices and NUTservers. An Event Processing Service (EPS) may function as a coordinator for archiving events that may be produced by IoN devices and applications that may want to generate an event or react to it as depicted in FIG. 172. Since some or all events may be contained within Nuts, any event may be communicated across any network as long as there may be a traversable route between devices. This may allow a user to monitor for desired events in local and remote IoN/IoT devices and/or NUTserver systems. It may allow a user to trigger scheduled or adhoc events on local and/or remote devices. Events may be replicated across some or all of the user's devices if so desired. The

EPS may work with the Remote Control Interface to allow for device specific commands to be initiated based on events. FIG. 172 embodies a scenario where a local calendar application 17208 on device 17200 may trigger a timed event through the local EPS 17204 to be executed on IoN device 17220 that may be reachable by NUTserver 17212 on device 17210. The local EPS 17204 may relay the event to another EPS 17214 that may have access to the target IoN device 17220. The EPS 17214 then may relay the event/command to its local NUTserver 17212 and then it may use its IoN/IoT interface to pass the event/command Nut to the IoN device 17220. Upon receipt of the event/command Nut, the IoN device 17220 may authenticate and then may execute the command via its Remote Control Interface. Examples of such events may be as varied as but not limited to starting up remote servers on a schedule, sending emails on a schedule, sending chat messages concerning system statuses, brewing coffee in the morning on an IoN compatible coffee machine, changing the temperature setting on a smart thermostat and/or warming up a car on a cold winter morning twenty minutes after the coffee may have finished brewing.

[0620] The EPS may store past events it may have received and generated on each device it may be running in an Event Nut Storage area 17216 and 17206. This may act as an event repository as well as an event queue for communication and device failures. The user or admin may browse these events at a later time and may analyze it for any use thereafter. A user with a NUTcloud account may also have her events replicated to it so that events may be viewed from any internet access. Some or all events may be Nut protected and may be owned by the user. The NUThub may interface with it seamlessly to take advantage of the queuing capability of the EPS.

[0621] An example of an application taking advantage of the EPS and its repository may be when a home alarm system starts warning that some of its battery operated sensors may be low on battery charge. The home alarm system may generate a low battery event specifying the unit that may be involved and may request a service call with the alarm maintenance company. The alarm company may suggest various times it may service the problem to the user via email and the user may make a different time suggestion or accept their suggested time. Upon acceptance, both calendars on the alarm company and user devices may be updated with the appointment information automatically. The alarm system may have a limited RBK relationship with the alarm company thus it may do diagnostics with the implicit approval of the homeowner in a secure manner.

Contextual Computing with App Nuts

[0622] There may be an unabashed land grab for some or all facets of a user's digital detritus by web companies such as but not limited to search habits, search history, device specifications, web viewing habits, shopping tendencies, blogging content, social networks, business networks, email content, texting messages, photos and/or even the digitized analysis of their DNA. The overwhelming majority of this user generated data may not be owned, accessed, reviewed, changed, deleted and/or controlled by the user who may have generated it. NUTS technology may make it easier for app developers to store user generated data and may make it easier to give a copy to the user for their own use and archiving. It may provide a common secured container which may vary on content formats via the MIO to allow for customizations. Very few web service vendors may be general enough to cover most aspects of a user's digital footprint; for example, Amazon may only know some of your shopping preferences and Google may know only some of your search history. Thus web vendors typically may assemble partial slices of a person's habits based on the service they provide. The best vantage point to collect some or all the digital whereabouts and activities of a user may be by the user for the user. A typical network layout for a vendor and the user app is shown in FIG. 173 where a vendor might use local browser based cookies to tag the user or his current session and may use Big Data gathering servers to record some or all the activities from and on the App.

[0623] If a user interfaces with apps that may provide a complete record of their sessions in a Nut for their own archives and use, then the user may eventually be able to gather the various facets of her digital excursions as depicted in FIG. 174. These session histories may provide a context upon

which analysis may be done by context sensitive apps to provide more conveniences to the user as shown in FIG. **175**. An application may save its session histories in an App Nut **17414** and this in turn may be used by some or all other apps the user may have installed to benefit the user appropriately. The proper analysis of context may derive the essence of the task the user may want to accomplish. An accounting app **17524** may record its sessions in an app Nut **17414** for some or all the bill paying and checking account activities the user may have done. A pattern recognition app **17520** that may read such a session history may analyze it and recommend the historical steps taken to pay the monthly bills and may present a preview of the actions it may take on behalf of the user. If the user agrees with its analysis, it may execute these steps to pay some or all the relevant bills automatically using the various accounts under the user's name. This app Nut may be available to the user across the internet if she synchronizes her Nuts via the NUTcloud **17500**.

[0624] Another useful aspect of the context saved by app Nuts may be that of repeatable procedures. This may be a common feature among Command Line Interfaces that developers may be fond of where previous commands may be saved for optional re-execution on demand. App Nuts may provide the same type of procedural recalls on demand for the user on virtually any compatible app. A context storing travel app may provide the essence of the requirements for a proposed trip in an app Nut after the initial search on the web by the user. At a later time, the user may resume this search to some or all her preferred travel outlets automatically by re-executing the distilled requirements on them using a context sensitive travel search app. This may alleviate the time spent on re-entering varying forms on each travel website and may produce an automatic summary of some or all her options. Furthermore, since the process may be entirely controlled by the user and some or all sensitive information may be stored by her NUTbook, the queries to vendors she may have mileage privileges and/or memberships with may be applied properly by the context sensitive travel search app to obtain the most personalized and meaningful results to her. This type of deep context sensitive searches may be virtually impossible to accomplish by a single vendor unless the user wholeheartedly may give unfettered access to some or all her sensitive digital information at some or all times to that vendor and trusts it completely; this may be a highly doubtful proposition for the average digitally sensible user.

[0625] In another embodiment, FIG. **176** shows the network layout for a user's IoN/IoT devices and the various utilities and services she may subscribe to for her daily life at home. No single company may be able to collect the user's entire home life in a digital manner. However, the user may accomplish this if some or all her devices produced app Nuts and she had an app that may analyze her various digital contexts. An energy saving context sensitive App may analyze the electricity use by various electronic appliances in her home and may merge it with the electric company's peak and off peak rates to suggest energy saving measures that may be automatically enacted by the app on her behalf. It may analyze her personal use habits of each device to coordinate convenient combinations for her when it recognizes a set of circumstances from the past. IoN/IoT devices may inform her of maintenance requirements if periodically run self-diagnostics reveal failing parts or sub-optimal operational readings.

[0626] There may be security concerns with IoT devices containing various environmental sensors which may not be entirely controlled by the owner of the device but rather by the manufacturers and/or potential malfeasant hackers. FIG. **177** shows an example of a network layout of two IoN devices and their respective manufacturers. When app Nuts **17734** and **17744** may be produced by each IoN device **17730** and **17740**, it may be locally archived by a NUTserver **17722** in local storage **17724**. These archived app Nuts may later be reviewed and filtered by the user before sending them on to the manufacturers to remove any sensitive information that the user deems inappropriate for a third party to collect. In FIG. **178**, a contextual analysis app **17808** may offer specialized routine filtering of some or all her IoN/IoT produced messages to minimize unknowingly exposing her privacy to third parties. In this manner, third parties may still gather some data from each device sold only to the extent of which each owner may allow; therefore, they

may deduce what personal information the average buyer may be willing to give to them.

CONCLUSION AND PHILOSOPHY

[0627] The various embodiments and scenario examples which have been detailed may be based on the core NUTS philosophy that data belongs to the user who generated it and that the user may have the means to control its exposure with precision. The design may be flexible enough to accommodate variations and/or alternatives such as but not limited to alternate cipher methods, keys of different lengths, different data transmutations, and/or different locking mechanisms. SDFT provides a useful toolset for the programmer with which to transmute data at the lowest levels and may help in the enabling of Structured Cryptographic Programming to construct NUTS structures and other complex cryptographic structures. SDFT allows a portability of data paired with its transmutation commands in a flexible and generalized way. NUTS' various embodiments may be customized to fit into existing organizational and security infrastructures or it may be stand-alone installations for a single user. The tangibility of data may be an important philosophy that NUTS proposes and may implement, the ability for users to store, manipulate and/or review the data that they may generate in simple ways while offering features befitting the most sophisticated managed systems. In conclusion, NUTS may give individual users an alternative to current methods of organizing their digital works and data.

Claims

1. A method of processing data comprising: operating, by at least one processor, a variable locking function on data using a plurality of main keys, at least one of the plurality of main keys performing one of a plurality of defined operations on the data, thereby producing at least one encrypted derived key; wherein, the one of the plurality of defined operations performed on the data using any one of the plurality of main keys produces at least a first encrypted derived key, the one of the plurality of defined operations performed on the data using the plurality of main keys produces at least a second encrypted derived key, and the one of the plurality of defined operations performed on the data using at least a required threshold number of the plurality of main keys produces at least a third encrypted derived key.
2. The method of claim 1, wherein the plurality of defined operations comprise transmutation commands configured to be performed in a specified sequence on at least one input data object.
3. The method of claim 1, wherein each of the plurality of defined operations decrypts the at least one encrypted derived key to produce the original data; the one of the defined operations performed on the encrypted derived key using any one of the plurality of main keys decrypts at least a first derived key; the one of the defined operations performed on the encrypted derived key using the plurality of main keys decrypts at least a second derived key; and the one of the defined operations performed on the encrypted derived key using at least a required threshold number of the plurality of main keys decrypts at least a third derived key.
4. The method of claim 3, wherein the plurality of defined operations include at least one of an ORLOCK operation, an XORLOCK operation, a MATLOCK operation, a HASHLOCK operation, and an SSLOCK operation.
5. The method of claim 1, wherein the at least one encrypted derived key includes at least one of an encrypted symmetric key, an encrypted asymmetric key part, and an encrypted string.
6. The method of claim 1, wherein a parameter sets the defined operation to be performed by the variable locking function.
7. The method of claim 1, wherein a parameter sets the required threshold number of the plurality of main keys.
8. The method of claim 5, wherein the encrypted string comprises a passphrase, a password, a binary string, and any string used as a cryptographic key material.
9. A data processing system comprising: a memory configured to store a data storage structure, the

data storage structure containing at least one data field including at least one input data object; and at least one processor configured to operate a variable locking function on the at least one input data object using a plurality of main keys, at least one of the plurality of main keys being configured to perform one of a plurality of defined operations on the data, thereby producing at least one encrypted derived key; wherein, the one of the plurality of defined operations performed on the at least one input data object using any one of the plurality of main keys is configured to produce at least a first encrypted derived key, the one of the plurality of defined operations performed on the at least one input data object using the plurality of main keys is configured to produce at least a second encrypted derived key, and the one of the plurality of defined operations performed on the at least one input data object using at least a required threshold number of the plurality of main keys is configured to produce at least a third encrypted derived key.

10. The system of claim 9, wherein the plurality of defined operations comprise transmutation commands configured to be performed in a specified sequence on at least one input data object.

11. The system of claim 9, wherein each of the plurality of defined operations is configured to decrypt the at least one encrypted derived key to produce the original data; the one of the defined operations performed on the encrypted derived key using any one of the plurality of main keys is configured to decrypt at least a first derived key; the one of the defined operations performed on the encrypted derived key using the plurality of main keys is configured to decrypt at least a second derived key; and the one of the defined operations performed on the encrypted derived key using at least a required threshold number of the plurality of main keys is configured to decrypt at least a third derived key.

12. The system of claim 11, wherein the plurality of defined operations include at least one of an ORLOCK operation, an XORLOCK operation, a MATLOCK operation, a HASHLOCK operation, and an SSLOCK operation.

13. The method of claim 9, wherein the at least one encrypted derived key includes at least one of an encrypted symmetric key, an encrypted asymmetric key part, and an encrypted string.

14. The method of claim 9, wherein a parameter is configured to set the defined operation to be performed by the variable locking function.

15. The method of claim 9, wherein a parameter is configured to set the required threshold number of the plurality of main keys.

16. The method of claim 13, wherein the encrypted string comprises a passphrase, a password, a binary string, and any string used as a cryptographic key material.
